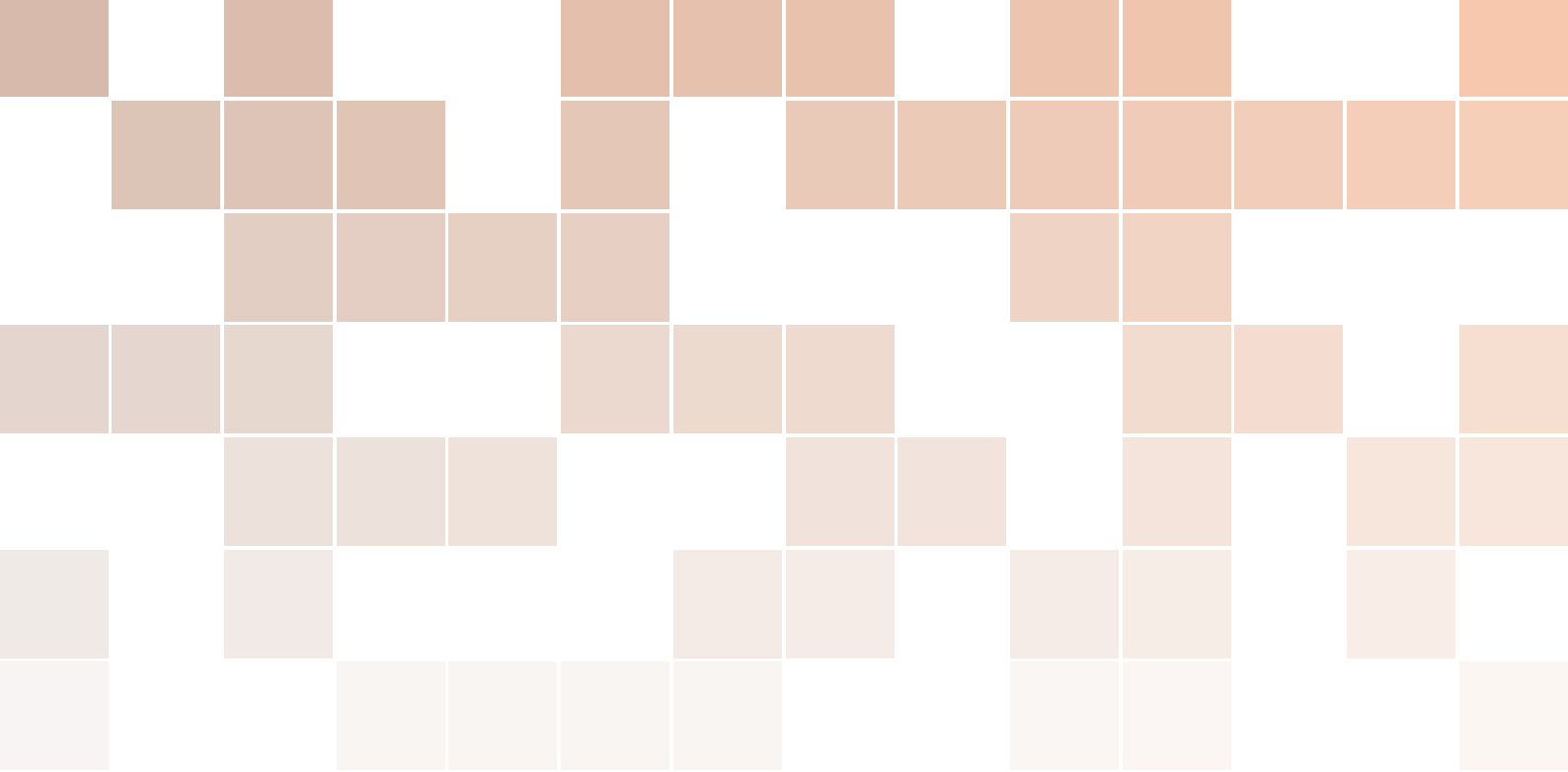
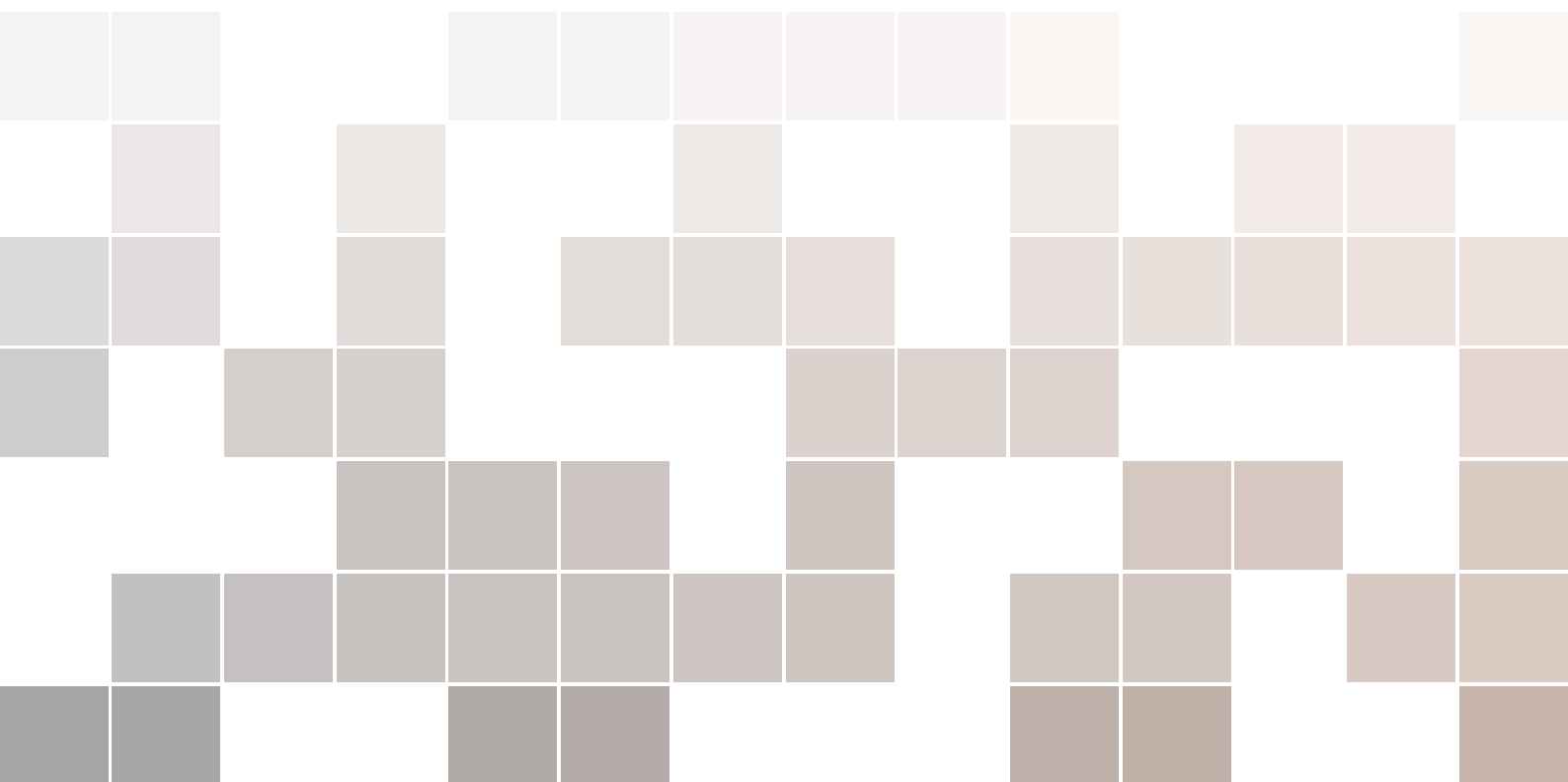




# Distributed Systems

Simple Introduction Guide

**Dr. Gehad Taher**







# Contents

## I

## Part One: Effective DS

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction to Distributed Systems</b>                                    | <b>9</b>  |
| 1.1      | Introduction                                                                  | 9         |
| 1.1.1    | What Is a Distributed System?                                                 | 9         |
| 1.1.2    | We Live in a World of Data                                                    | 11        |
| 1.1.3    | Store and Process Data at Scale: Vertical vs Horizontal Scaling               | 12        |
| 1.1.4    | Historical Context                                                            | 12        |
| 1.1.5    | Features of Distributed Systems                                               | 12        |
| 1.2      | Why Distributed Systems?                                                      | 13        |
| 1.3      | Motivating Case Study: Netflix – How Distributed Systems Make Streaming Work? | 16        |
| 1.4      | Exercises                                                                     | 17        |
| 1.5      | Answers                                                                       | 17        |
| 1.6      | Multiple-Choice Quiz                                                          | 18        |
| <b>2</b> | <b>Communication Paradigms</b>                                                | <b>21</b> |
| 2.1      | Communicating Entities                                                        | 21        |
| 2.2      | Communication Paradigms                                                       | 22        |
| 2.3      | Middleware Layers in Distributed Systems                                      | 22        |
| 2.4      | Introduction to Remote Invocation Concept                                     | 24        |
| 2.5      | Exercises                                                                     | 35        |
| 2.6      | Answers                                                                       | 36        |
| 2.7      | Multiple-Choice Quiz                                                          | 37        |
| <b>3</b> | <b>Architectures Models in Distributed Systems</b>                            | <b>41</b> |
| 3.1      | Architectural Models                                                          | 42        |
| 3.1.1    | Master-Slave Architecture                                                     | 42        |
| 3.1.2    | Peer-to-Peer Architectures                                                    | 42        |

|            |                                                                 |           |
|------------|-----------------------------------------------------------------|-----------|
| <b>3.2</b> | <b>Architectural Patterns: Tiering vs. Layering</b>             | <b>46</b> |
| 3.2.1      | Tiering: Horizontal Splitting                                   | 46        |
| 3.2.2      | Layering: Vertical Organization                                 | 46        |
| <b>3.3</b> | <b>Excercises</b>                                               | <b>48</b> |
| <b>3.4</b> | <b>Answers</b>                                                  | <b>49</b> |
| <b>3.5</b> | <b>Multiple-Choice Quiz</b>                                     | <b>49</b> |
| <b>4</b>   | <b>Naming Systems in Distributed Systems</b>                    | <b>51</b> |
| <b>4.1</b> | <b>Introduction to Naming</b>                                   | <b>51</b> |
| <b>4.2</b> | <b>Naming Systems in Distributed Systems</b>                    | <b>54</b> |
| 4.2.1      | Flat Naming                                                     | 54        |
| 4.2.2      | Structured Naming and Naming Graphs                             | 62        |
| 4.2.3      | Attribute-Based Naming and Directory Services                   | 65        |
| <b>4.3</b> | <b>Questions</b>                                                | <b>67</b> |
| <b>4.4</b> | <b>Answers:</b>                                                 | <b>68</b> |
| <b>4.5</b> | <b>Multiple Choice Quiz</b>                                     | <b>69</b> |
| <b>5</b>   | <b>Synchronization in Distributed Systems</b>                   | <b>71</b> |
| <b>5.1</b> | <b>Introduction: The Problem of Time in Distributed Systems</b> | <b>71</b> |
| <b>5.2</b> | <b>Physical Clock Synchronization</b>                           | <b>72</b> |
| 5.2.1      | Cristian's Algorithm                                            | 72        |
| 5.2.2      | Berkeley Algorithm                                              | 73        |
| 5.2.3      | Network Time Protocol (NTP)                                     | 73        |
| <b>5.3</b> | <b>Logical Clock Synchronization</b>                            | <b>74</b> |
| 5.3.1      | Lamport Timestamps                                              | 74        |
| 5.3.2      | Vector Clocks                                                   | 74        |
| <b>5.4</b> | <b>Mutual Exclusion in Distributed Systems</b>                  | <b>75</b> |
| 5.4.1      | Centralized Algorithm                                           | 76        |
| 5.4.2      | Decentralized Approach                                          | 77        |
| 5.4.3      | Token Ring Algorithm                                            | 78        |
| <b>5.5</b> | <b>Election Algorithms in Distributed Systems</b>               | <b>79</b> |
| 5.5.1      | Bully Algorithm                                                 | 79        |
| 5.5.2      | Ring-Based Election Algorithm                                   | 80        |
| <b>5.6</b> | <b>Questions</b>                                                | <b>81</b> |
| <b>5.7</b> | <b>Answers</b>                                                  | <b>82</b> |

## II

## Fast & Reliable or Efficient DS

|            |                                                                   |           |
|------------|-------------------------------------------------------------------|-----------|
| <b>6</b>   | <b>Replication &amp; Caching</b>                                  | <b>85</b> |
| <b>6.1</b> | <b>Caching Architecture</b>                                       | <b>86</b> |
| 6.1.1      | Fetching Policy: What Data Should Be Cached and When?             | 86        |
| 6.1.2      | Consistency Policy: How are Updates Propagated to Cached Copies?  | 87        |
| 6.1.3      | Replacement Policy: Which Cached Data Should Be Evicted and When? | 94        |

---

|            |                                              |            |
|------------|----------------------------------------------|------------|
| <b>6.2</b> | <b>Ordering of Updates</b> .....             | <b>99</b>  |
| 6.2.1      | Total Ordering in Distributed Systems .....  | 99         |
| 6.2.2      | Sequential Ordering .....                    | 100        |
| 6.2.3      | Causal Ordering .....                        | 102        |
| <b>6.3</b> | <b>Questions on Replication</b> .....        | <b>104</b> |
| <b>6.4</b> | <b>Answers of Replication</b> .....          | <b>105</b> |
| <b>6.5</b> | <b>Multiple Choice Quiz</b> .....            | <b>106</b> |
| <b>7</b>   | <b>Fault Tolerance</b> .....                 | <b>109</b> |
| 7.1        | What is a Failure? .....                     | 110        |
| 7.2        | Types of Failures .....                      | 110        |
| 7.3        | Reliability vs. Availability .....           | 110        |
| 7.4        | Techniques for Fault Tolerance .....         | 111        |
| 7.5        | Two-Phase Commit Finite State Machines ..... | 112        |
| 7.6        | Questions: .....                             | 116        |
| 7.7        | Answers: .....                               | 117        |



# Part One: Effective DS

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction to Distributed Systems</b>                                    | <b>9</b>  |
| 1.1      | Introduction                                                                  | 9         |
| 1.2      | Why Distributed Systems?                                                      | 13        |
| 1.3      | Motivating Case Study: Netflix – How Distributed Systems Make Streaming Work? | 16        |
| 1.4      | Exercises                                                                     | 17        |
| 1.5      | Answers                                                                       | 17        |
| 1.6      | Multiple-Choice Quiz                                                          | 18        |
| <b>2</b> | <b>Communication Paradigms</b>                                                | <b>21</b> |
| 2.1      | Communicating Entities                                                        | 21        |
| 2.2      | Communication Paradigms                                                       | 22        |
| 2.3      | Middleware Layers in Distributed Systems                                      | 22        |
| 2.4      | Introduction to Remote Invocation Concept                                     | 24        |
| 2.5      | Exercises                                                                     | 35        |
| 2.6      | Answers                                                                       | 36        |
| 2.7      | Multiple-Choice Quiz                                                          | 37        |
| <b>3</b> | <b>Architectures Models in Distributed Systems</b>                            | <b>41</b> |
| 3.1      | Architectural Models                                                          | 42        |
| 3.2      | Architectural Patterns: Tiering vs. Layering                                  | 46        |
| 3.3      | Excercises                                                                    | 48        |
| 3.4      | Answers                                                                       | 49        |
| 3.5      | Multiple-Choice Quiz                                                          | 49        |
| <b>4</b> | <b>Naming Systems in Distributed Systems</b>                                  | <b>51</b> |
| 4.1      | Introduction to Naming                                                        | 51        |
| 4.2      | Naming Systems in Distributed Systems                                         | 54        |
| 4.3      | Questions                                                                     | 67        |
| 4.4      | Answers:                                                                      | 68        |
| 4.5      | Multiple Choice Quiz                                                          | 69        |
| <b>5</b> | <b>Synchronization in Distributed Systems</b>                                 | <b>71</b> |
| 5.1      | Introduction: The Problem of Time in Distributed Systems                      | 71        |
| 5.2      | Physical Clock Synchronization                                                | 72        |
| 5.3      | Logical Clock Synchronization                                                 | 74        |
| 5.4      | Mutual Exclusion in Distributed Systems                                       | 75        |
| 5.5      | Election Algorithms in Distributed Systems                                    | 79        |
| 5.6      | Questions                                                                     | 81        |
| 5.7      | Answers                                                                       | 82        |







# 1. Introduction to Distributed Systems

## 1.1 Introduction

Imagine you're using Google Maps to find the fastest route to a café. You type in the destination, and within seconds, your phone shows traffic updates, estimated arrival time, and alternate routes. Behind the scenes, dozens—maybe hundreds—of computers are working together to deliver that result. This is the magic of **distributed systems**.

**Distributed systems** are the invisible engines behind the digital experiences we rely on every day. From streaming a movie on Netflix to conducting a real-time financial transaction across continents, these systems quietly coordinate vast networks of machines to deliver seamless performance. As the world becomes increasingly data-driven, centralized computing models are reaching their limits. The sheer volume, velocity, and variety of data—generated by billions of devices and users—demand architectures that can scale, adapt, and recover gracefully from failure.

### 1.1.1 What Is a Distributed System?

A **distributed system** is a group of independent computers that work together to appear as a single system to the user. Figure 1.1 shows that these computers:

- Communicate over a network
- Share resources and data
- Coordinate tasks to achieve a common goal

These systems are designed to handle massive workloads, scale across geographies, and remain resilient even when parts fail. They are the backbone of modern computing, enabling everything from cloud platforms and social media to scientific research and global commerce. Figure 1.2 presents a powerful idea that says: We are living at the edge of a technological revolution. Fields like astronomy, biotechnology, sensor networks, and ubiquitous computing are generating unprecedented amounts of data and complexity. **These breakthroughs are not isolated—they are deeply interconnected, and they all rely on distributed systems to function.**

- **Astronomy:** Telescopes like JWST produce terabytes of data daily, requiring distributed storage and processing.
- **Gene Sequencing:** Modern labs analyze entire genomes using distributed computing pipelines.
- **Sensors:** Smart cities and industrial systems use distributed sensor networks for real-time monitoring.
- **Ubiquitous Computing:** Devices embedded in everyday life (phones, appliances, vehicles)

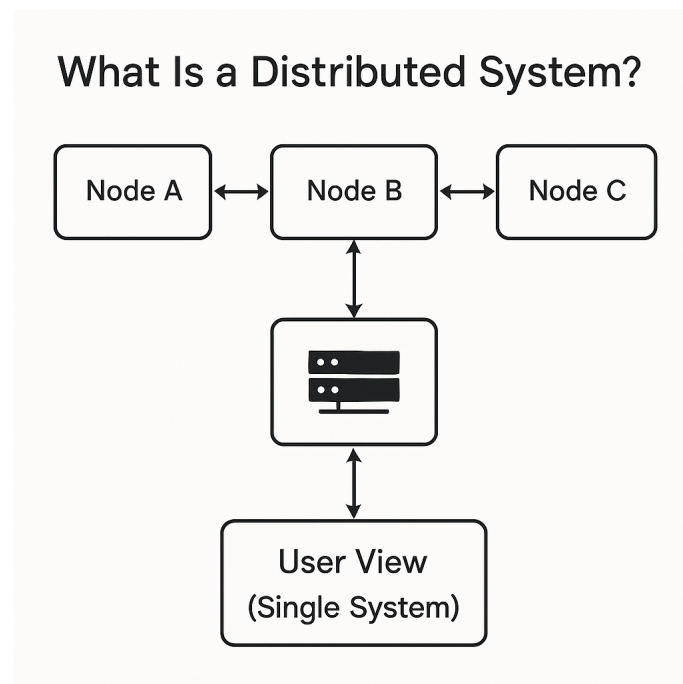


Figure 1.1: Distributed Systems Idea

## On the Verge of A Disruptive Century : Breakthroughs

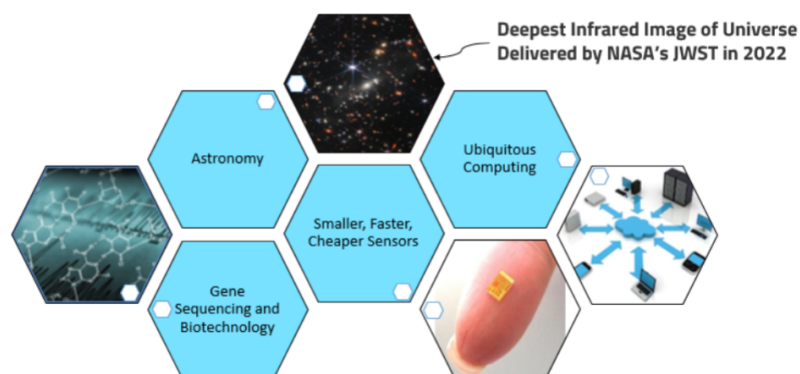


Figure 1.2: Foundations of Distributed systems

rely on distributed coordination.

## 1.1.2 We Live in a World of Data

We are surrounded by data—generated, transmitted, and consumed at staggering rates. Every click, swipe, transaction, and sensor reading contributes to a digital ecosystem that is growing exponentially. In 2025 alone, global data creation is projected to exceed 180 zettabytes you can see this in Figure 1.3, a figure so vast it defies intuitive comprehension. This data is not confined to tech giants or research labs—it flows through our daily lives: from smart thermostats adjusting room temperature to autonomous vehicles navigating city streets, and from personalized healthcare diagnostics to real-time financial trading platforms.

The implications are profound. Data is no longer just a byproduct of computation—it is the raw material of insight, innovation, and decision-making. Distributed systems make this possible by enabling scalable, fault-tolerant infrastructures that can store, process, and analyze data across geographies and domains. Understanding how data moves, transforms, and informs is essential for any computer scientist in the modern era.

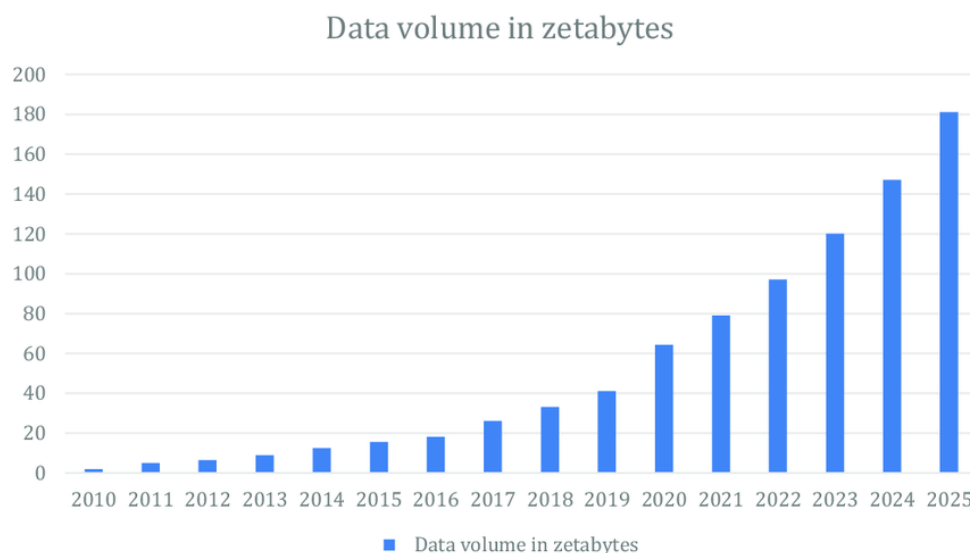


Figure 1.3: Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2025

Yet, as the volume of data grows exponentially, so does the complexity of managing it. Storing and processing data at scale is one of the defining challenges of distributed systems. Traditional monolithic architectures crumble under the weight of petabytes of information, necessitating a shift to decentralized, fault-tolerant infrastructures. Distributed file systems like HDFS and object stores like Amazon S3 allow us to persist massive datasets reliably. Processing frameworks such as Apache Spark, Flink, and Hadoop enable parallel computation across clusters, turning raw data into actionable knowledge. Stream processing engines like Kafka and Pulsar handle real-time ingestion and transformation, ensuring that insights are not just accurate but timely.

This scale demands more than just technical prowess—it requires architectural foresight. **Data must be partitioned intelligently, replicated for resilience, and processed with latency and throughput in mind.** Systems must be elastic, adapting to fluctuating workloads, and secure, safeguarding sensitive information. As we move deeper into the era of machine learning and AI, the ability to store and process data at scale is not just a technical requirement—it is a strategic imperative.

### 1.1.3 Store and Process Data at Scale: Vertical vs Horizontal Scaling

As systems grow and user demands increase, scalability becomes a critical concern. Two primary strategies address this challenge: **vertical scaling** and **horizontal scaling**. Understanding their differences is essential for designing resilient, high-performance distributed systems.

**Vertical scaling, also known as scaling up**, involves enhancing the capacity of a single machine. This could mean upgrading the CPU, adding more RAM, or increasing disk throughput. It's straightforward and often requires minimal architectural changes. For example, a database server might be upgraded from 32 GB to 128 GB of RAM to handle more queries.

However, vertical scaling has limitations: hardware upgrades have diminishing returns, there's a physical ceiling to how much a single machine can be enhanced, and it introduces a single point of failure. If that machine crashes, the entire system goes down.

**Horizontal scaling, or scaling out**, takes a different approach. Instead of relying on a single powerful machine, it distributes the workload across multiple machines. These machines can be commodity servers, each handling a portion of the traffic or data. This strategy is foundational to distributed systems. It offers fault tolerance—if one node fails, others can take over—and elasticity, allowing systems to grow or shrink based on demand.

However, horizontal scaling introduces complexity: data must be partitioned, consistency must be managed, and communication between nodes becomes critical.

In practice, vertical scaling is often used for quick performance boosts or legacy systems, while horizontal scaling is preferred for modern cloud-native architectures. Distributed databases, microservices are all designed with horizontal scaling in mind. **You can see this video to let you visualize what we mean [Horizontal vs Vertical Scaling](#)**

#### Advantages and Disadvantages

- **Vertical Scaling**
  - *Advantages*: Simple to implement, no need to redesign architecture.
  - *Disadvantages*: Limited scalability, single point of failure, expensive hardware.
- **Horizontal Scaling**
  - *Advantages*: High fault tolerance, cost-effective, scalable on demand.
  - *Disadvantages*: Complex architecture, requires load balancing and data distribution.

### 1.1.4 Historical Context

The roots of distributed computing trace back to the 1970s see [Figure 1.4](#), when researchers began exploring how multiple computers could work together to solve problems more efficiently. Early systems like ARPANET laid the groundwork for networked communication, while the rise of UNIX and remote procedure calls (RPCs) enabled basic distributed interactions. Over time, the need for fault tolerance, scalability, and global reach led to the development of more sophisticated systems—culminating in today's cloud platforms, blockchain networks, and edge computing infrastructures.

### 1.1.5 Features of Distributed Systems

Distributed systems are distinguished by a set of core features that shape their architecture, behavior, and challenges. These features are not merely technical attributes—they define how distributed systems operate, scale, and recover in real-world environments. Understanding these characteristics is essential for designing systems that are robust, efficient, and responsive see [Figure 1.5](#).

#### 1. Geographical Separation

Distributed systems consist of components that are physically separated across different locations. These components may reside in different data centers, cities, or even continents. This separation



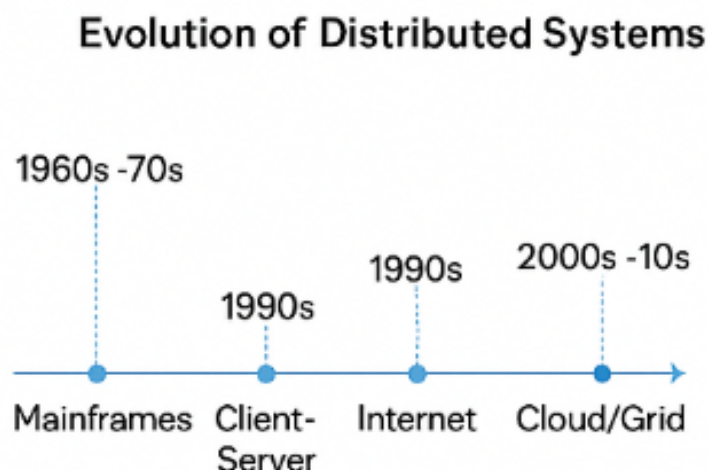


Figure 1.4: Evolution of Distributed Systems

enables global scalability and redundancy, but also introduces latency and network reliability concerns. For example, a content delivery network (CDN) caches data in multiple regions to serve users faster, but must synchronize updates across all nodes.

## 2. No Common Physical Clock

Unlike tightly coupled systems, distributed systems lack a shared global clock. Each node maintains its own local time, which can drift due to hardware differences or network delays. This absence of a common clock complicates event ordering and synchronization. To address this, distributed systems use logical clocks—such as Lamport timestamps or vector clocks—to infer causality and maintain consistency.

## 3. No Shared Physical Memory

Each node in a distributed system operates with its own local memory. There is no shared memory space accessible to all components, which means that communication must occur through message passing. This design promotes modularity and fault isolation, but requires careful coordination to ensure data consistency and coherence across nodes.

## 4. Autonomy and Heterogeneity

Nodes in a distributed system are **autonomous**—they can operate independently and may be managed by different organizations or teams. They are also **heterogeneous**, meaning they can run different hardware, operating systems, or software stacks. This diversity enhances flexibility and scalability, but also demands standardized communication protocols and robust interoperability mechanisms.

## 1.2 Why Distributed Systems?

The motivation for distributed systems stems from practical constraints and ambitious goals. Vertical scaling—adding more power to a single machine—faces diminishing returns due to hardware bottlenecks in memory, disk I/O, and processor throughput. Horizontal scaling, on the other hand,

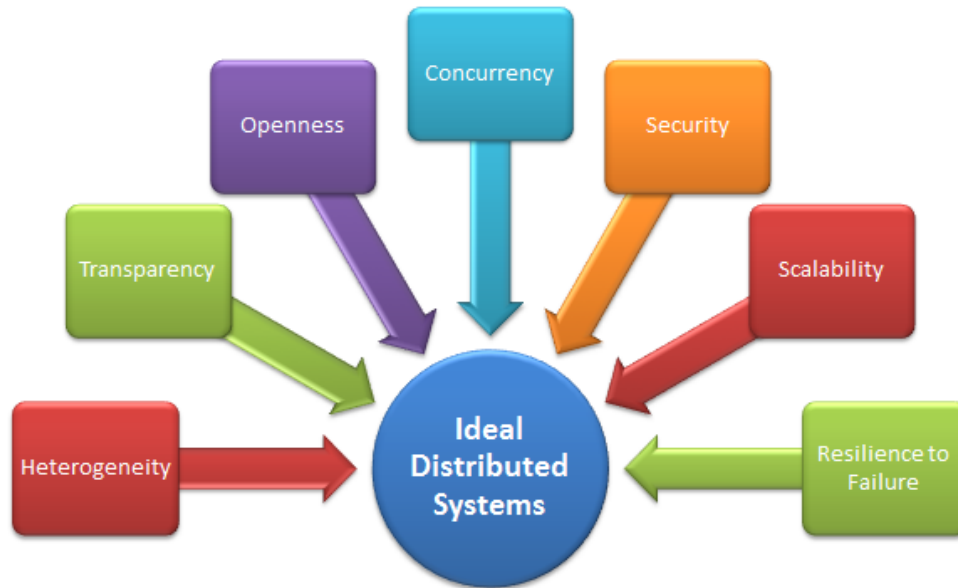


Figure 1.5: Illustration of Key Features in a Distributed System

distributes tasks across multiple machines, offering elasticity, fault isolation, and cost efficiency. But this shift introduces complexity: how do we synchronize processes, maintain consistency, and ensure availability when components can fail independently? why we don't use parallel systems and how they differ from distributed systems?

### Parallel vs. Distributed Systems

Parallel and distributed systems both aim to improve computational efficiency, but they differ fundamentally in architecture and design philosophy. **Parallel systems** consist of multiple processors working simultaneously on a shared task, often within a single machine or tightly coupled environment. These systems rely on shared memory and typically operate under a unified clock, making synchronization straightforward but limiting scalability.

In contrast, **distributed systems** comprise independent nodes that communicate over a network. Each node has its own memory and clock, and coordination is achieved through message passing and consensus protocols. This design enables distributed systems to scale across geographic regions and tolerate faults more gracefully. While parallel systems excel in high-performance tasks like simulations and matrix computations, distributed systems are ideal for applications requiring resilience, scalability, and decentralized control—such as cloud platforms, microservices, and peer-to-peer networks see Figure 2.1 and Table 1.1 .

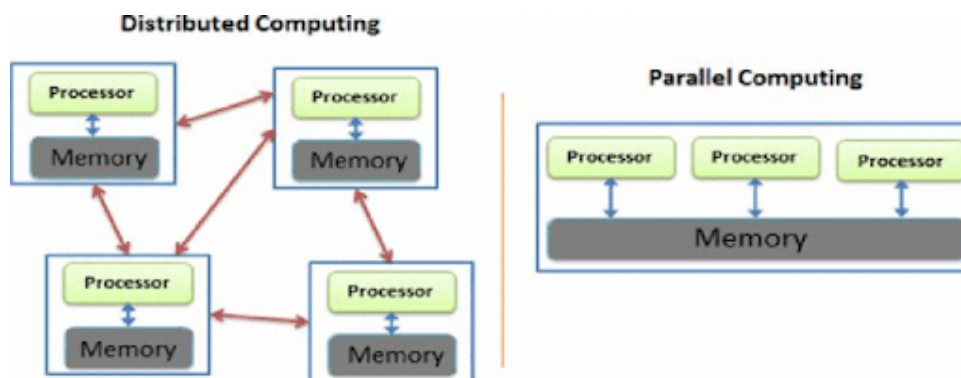


Figure 1.6: Parallel Vs. Distributed Systems

Table 1.1: Comparison Between Parallel and Distributed Systems

| Aspect                 | Parallel Systems                                        | Distributed Systems                                  |
|------------------------|---------------------------------------------------------|------------------------------------------------------|
| Architecture           | Tightly coupled processors sharing memory               | Loosely coupled nodes with independent memory        |
| Communication          | Shared memory or high-speed interconnects               | Message passing over a network                       |
| Clock Synchronization  | Typically share a global clock                          | No global clock; use logical clocks                  |
| Location of Components | Usually located in the same physical machine or rack    | Geographically dispersed across multiple locations   |
| Fault Tolerance        | Lower fault tolerance; failure may affect entire system | Higher fault tolerance; nodes can fail independently |
| Scalability            | Limited scalability due to shared resources             | Highly scalable across heterogeneous environments    |
| Use Cases              | Scientific computing, simulations, real-time processing | Web services, cloud computing, distributed databases |
| Resource Sharing       | Shared memory and CPU resources                         | Independent resources coordinated via protocols      |

## 1.3 Motivating Case Study: Netflix – How Distributed Systems Make Streaming Work?

### What Is Netflix Doing Behind the Scenes?

Netflix lets millions of people around the world watch movies and shows anytime they want. But how does it work so smoothly—even when millions are watching at the same time?

**The answer: distributed systems.** Netflix uses many computers working together across the globe to make sure videos load fast, recommendations are accurate, and the service doesn't crash.

### Key Ideas in Netflix's System

Here are some simple examples of how Netflix uses distributed systems:

- **Content Delivery:** Netflix stores copies of videos in many places around the world. When you press play, it sends the video from a nearby server so it loads quickly.
- **Microservices:** Instead of one big program, Netflix is made of many small services. One handles your login, another gives you recommendations, another tracks what you've watched. These services work together like a team.
- **Handling Failures:** Netflix tests its system by pretending things go wrong—like a server crashing. This helps them make sure the system can recover and keep working.
- **Finding Services:** Netflix uses tools to help its services find each other, even when they move or change. This keeps everything connected.
- **Balancing Traffic:** If too many people use one server, Netflix spreads the load to others so no one gets stuck waiting.

### Smart Recommendations

Netflix suggests shows you might like based on what you've watched. To do this, it collects data and runs smart algorithms across many computers. This helps it give you good suggestions quickly—even while millions of others are watching too.

### Staying Strong Under Pressure

Netflix doesn't wait for problems to happen—they test their system by creating fake failures. This helps them build a system that can handle real problems without going down.

Netflix is a great example of how distributed systems help real-world apps:

- They make things faster by using many computers in different places.
- They keep services running even when something breaks.
- They let companies grow and serve millions of users without slowing down.



## 1.4 Exercises

- Q1.** Define a distributed system in your own words. What makes it different from a centralized system?
- Q2.** Why is horizontal scaling generally preferred over vertical scaling in modern distributed architectures?
- Q3.** Describe one real-world application that relies on distributed systems. What challenges does it face?
- Q4.** Explain the role of message passing in distributed systems. Why is it necessary?
- Q5.** What are some historical milestones that contributed to the development of distributed computing?

## 1.5 Answers

- A1.** A distributed system is a network of independent computers that work together and appear as a single system to users. Unlike centralized systems, distributed systems rely on coordination and communication across multiple nodes.
- A2.** Horizontal scaling allows systems to grow by adding more machines, which is more cost-effective and fault-tolerant than upgrading a single machine. It supports elasticity and better handles large-scale workloads.
- A3.** Netflix is a prime example. It faces challenges like ensuring low-latency streaming, handling global traffic, and maintaining service availability during failures. Distributed caching and microservices help address these issues.
- A4.** Message passing enables nodes in a distributed system to communicate and coordinate actions. Since there's no shared memory or clock, messages are the only way to share state and synchronize behavior.
- A5.** Key milestones include the development of ARPANET, the introduction of RPCs, the rise of UNIX networking, and the emergence of cloud computing platforms like AWS and Azure.

## 1.6 Multiple-Choice Quiz

- Q1.** What is a key driver of the current data explosion?
- A. Decline in global internet usage
  - B. Advances in gene sequencing and sensor technology
  - C. Reduction in cloud storage prices
  - D. Decrease in computing power
- Q2.** Which scaling method involves upgrading hardware on a single machine?
- A. Horizontal scaling
  - B. Vertical scaling
  - C. Distributed scaling
  - D. Modular scaling
- Q3.** What is a major limitation of vertical scaling?
- A. It requires multiple machines
  - B. It cannot improve cache performance
  - C. It is constrained by physical hardware limits
  - D. It eliminates the need for concurrency
- Q4.** What is the main advantage of horizontal scaling?
- A. It reduces the need for distributed systems
  - B. It allows data to be loaded faster using multiple machines
  - C. It eliminates the need for concurrency models
  - D. It simplifies memory management
- Q5.** Which of the following is NOT a requirement for building distributed systems?
- A. Naming protocols
  - B. Synchronization mechanisms
  - C. Shared physical memory
  - D. Communication paradigms
- Q6.** What does virtualization help achieve in distributed systems?
- A. Reduced latency
  - B. Increased heterogeneity
  - C. Portability and flexibility
  - D. Elimination of fault tolerance
- Q7.** What is a distributed system?
- A. A single computer with multiple cores
  - B. A group of tightly coupled processors
  - C. A collection of independent computers that appear as one system
  - D. A cloud-based storage solution
- Q8.** Why is horizontal scaling generally preferred over vertical scaling in modern data-intensive applications?
- A. It reduces the need for software updates
  - B. It allows scaling beyond hardware limits and improves fault tolerance
  - C. It simplifies cache design
  - D. It avoids concurrency issues
- Q9.** What challenge arises when scaling vertically with many cores on a chip?
- A. Increased power consumption
  - B. Difficulty in delivering input data fast enough to all cores
  - C. Reduced memory latency
  - D. Elimination of interconnects
- Q10.** Why is fault tolerance a critical requirement in distributed systems?
- A. Because distributed systems use unreliable hardware
  - B. Because partial failures are inevitable and must be handled gracefully

- C. Because all nodes share the same memory
  - D. Because it improves encryption
- Q11.** How does the processor-memory speed gap affect system design?
- A. It encourages the use of slower CPUs
  - B. It necessitates better caching and parallelism strategies
  - C. It eliminates the need for distributed systems
  - D. It simplifies vertical scaling
- Q12.** What architectural implication arises from the need to load 4TB of data in minutes?
- A. Use of larger disks
  - B. Use of more CPU cores
  - C. Use of distributed machines with parallel I/O channels
  - D. Use of deeper cache hierarchies
- Q13.** Which feature is common to distributed systems but not to parallel systems?
- A. Shared physical memory
  - B. Common physical clock
  - C. Geographical separation
  - D. Homogeneity
- Q14.** Which of the following is a communication paradigm in distributed systems?
- A. Shared memory
  - B. Message passing
  - C. Direct disk access
  - D. Cache coherence



## 2. Communication Paradigms

### Course Map

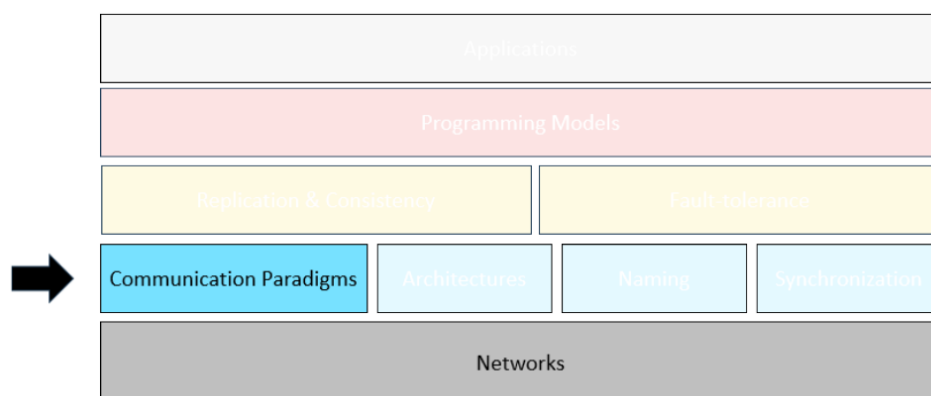


Figure 2.1: Communication Paradigms

Distributed systems consist of multiple independent computers that collaborate to perform tasks. Communication between these entities is essential, and **Remote invocation** is a powerful abstraction that allows a process to execute code on another machine as if it were local.

This chapter explores the concept of remote invocation, its role in distributed systems, and the foundational ideas behind **Remote Procedure Calls (RPCs)**. We begin by examining the types of entities involved and the communication paradigms that enable their interaction.

### 2.1 Communicating Entities

Entities in distributed systems can be broadly classified into:

- **System-oriented entities:** These include processes, threads, and nodes. They represent the underlying execution units and hardware components.
- **Problem-oriented entities:** These are higher-level abstractions such as objects in object-oriented programming. They encapsulate behavior and state relevant to the application domain.

Understanding how these entities communicate is crucial for designing robust distributed

applications.

## 2.2 Communication Paradigms

**Communication paradigms** define the methods by which entities exchange information. They can be categorized based on the location of the communicating entities:

1. **Same Address Space:** Entities share memory and can communicate via global variables or direct procedure calls.
2. **Same Computer, Different Address Spaces:** Communication occurs through files, signals, or shared memory.
3. **Networked Computers:** Entities communicate over a network using sockets or remote invocation mechanisms.

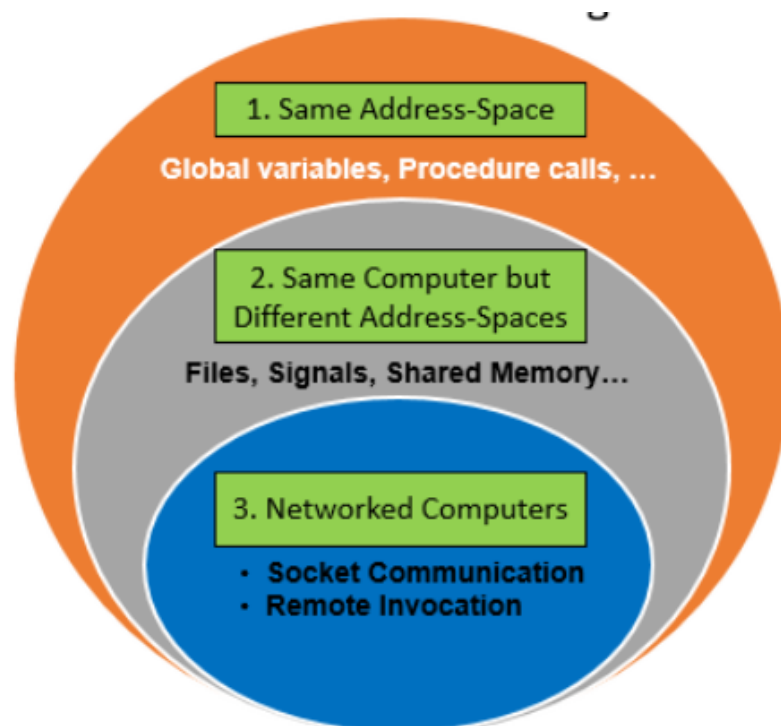


Figure 2.2: Communication paradigms based on entity location

This chapter focuses on the third category—communication between entities on networked computers.

## 2.3 Middleware Layers in Distributed Systems

Middleware is a software layer that resides between the operating system and distributed applications. It abstracts the complexity of network communication, heterogeneity, and resource management, enabling seamless interaction among distributed components.

### Overview of Middleware Architecture

The middleware stack typically consists of several layers, each responsible for a specific aspect of distributed communication and coordination:

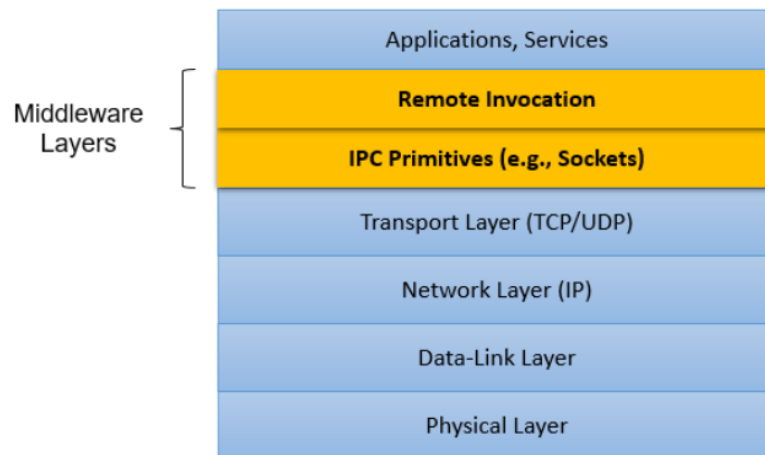


Figure 2.3: Layered middleware architecture in distributed systems

## Layer Descriptions

1. **Application Layer**
  - Contains user-facing applications and services.
  - Relies on middleware to access remote resources and services.
2. **Remote Invocation Layer**
  - Provides abstractions such as Remote Procedure Call (RPC) and Remote Method Invocation (RMI).
  - Handles marshalling, unmarshalling, and stub generation.
3. **Communication Services Layer**
  - Offers messaging, queuing, and event-driven communication.
  - Examples include message brokers, publish-subscribe systems, and service buses.
4. **Resource Management Layer**
  - Manages distributed resources such as databases, filesystems, and compute nodes.
  - Supports load balancing, replication, and fault tolerance.
5. **Security and Policy Layer**
  - Enforces authentication, authorization, and encryption.
  - Manages access control policies and audit trails.
6. **Transport Layer**
  - Provides reliable or best-effort data transmission.
  - Protocols include TCP, UDP, and QUIC.
7. **Network Layer**
  - Handles routing and addressing across distributed nodes.
  - Protocols include IP, ICMP, and routing algorithms.
8. **Data-Link and Physical Layers**
  - Responsible for physical transmission of data over hardware.
  - Includes Ethernet, Wi-Fi, and optical links.

## Benefits of Layered Middleware

The layered architecture of middleware in distributed systems offers a structured approach to managing complexity. By separating concerns across distinct layers, developers can build scalable, maintainable, and interoperable systems. Each layer encapsulates specific functionality—ranging from transport protocols to application-level abstractions—allowing independent development and



optimization. This modularity not only simplifies system design but also enhances flexibility when integrating diverse technologies and platforms.

- **Modularity:** Each layer can evolve independently, enabling focused improvements and easier debugging.
- **Interoperability:** Middleware supports communication between heterogeneous systems, programming languages, and operating environments.
- **Scalability:** Layered design facilitates dynamic resource allocation, load balancing, and horizontal scaling.
- **Maintainability:** Clear separation of concerns simplifies system upgrades, testing, and long-term maintenance.

### Examples of Middleware Technologies

- **CORBA:** Common Object Request Broker Architecture
- **Java RMI:** Remote Method Invocation in Java
- **gRPC:** Google's high-performance RPC framework
- **Message Brokers:** RabbitMQ, Apache Kafka
- **Enterprise Middleware:** .NET Remoting, EJB, WebSphere

## 2.4 Introduction to Remote Invocation Concept

Remote invocation is a fundamental abstraction in distributed systems that allows a process to execute a procedure or method on a remote machine as if it were local. This concept simplifies distributed programming by hiding the complexities of network communication, data serialization, and transport protocols.

In traditional local procedure calls, both caller and callee reside in the same address space. However, in distributed systems, the caller and callee may be located on different machines, possibly running different operating systems and architectures. Remote invocation bridges this gap by providing a seamless interface for invoking remote functionality.

The key idea behind remote invocation is that the programmer does not need to explicitly manage sockets, message formats, or network protocols. Instead, the middleware handles these details transparently. This abstraction enables developers to focus on application logic while relying on the system to manage communication.

Remote invocation is typically implemented in two forms:

- **Remote Procedure Call (RPC):** Allows invoking procedures on remote systems using a procedural interface.
- **Remote Method Invocation (RMI):** Extends RPC to object-oriented environments, enabling method calls on remote objects.

### Advantages of Remote Invocation

- **Transparency:** Developers write code as if invoking local procedures.
- **Modularity:** Promotes separation of concerns and reuse of components.
- **Interoperability:** Supports communication across heterogeneous platforms.
- **Scalability:** Enables distributed deployment and load balancing.

Remote invocation is a cornerstone of modern distributed systems, forming the basis for service-oriented architectures, microservices, and cloud-native applications.

**Example:** Imagine a weather app that fetches temperature data from a remote server. Instead of manually opening a socket and formatting a request, the app simply calls `getTemperature("Cairo")`—and the middleware handles the rest.

## A. Remote Procedure Call (RPC)

RPC allows a client to invoke a procedure on a remote server using a familiar function-call syntax.

**Example:** A client wants to add two numbers remotely:

```
int result = add(5, 3); // Looks local, but runs on server
```

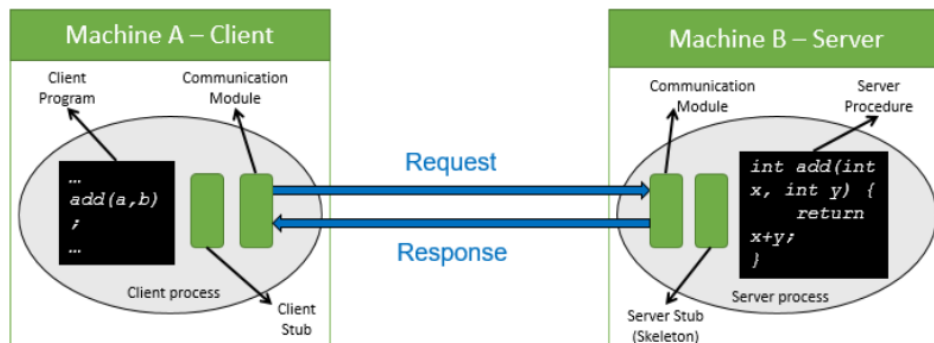


Figure 2.4: RPC add two numbers Example

## RPC Architecture

- **Client Stub:** Marshals parameters and sends request.
- **Server Stub:** Unmarshals request and invokes procedure.
- **Transport Module:** Handles message delivery.

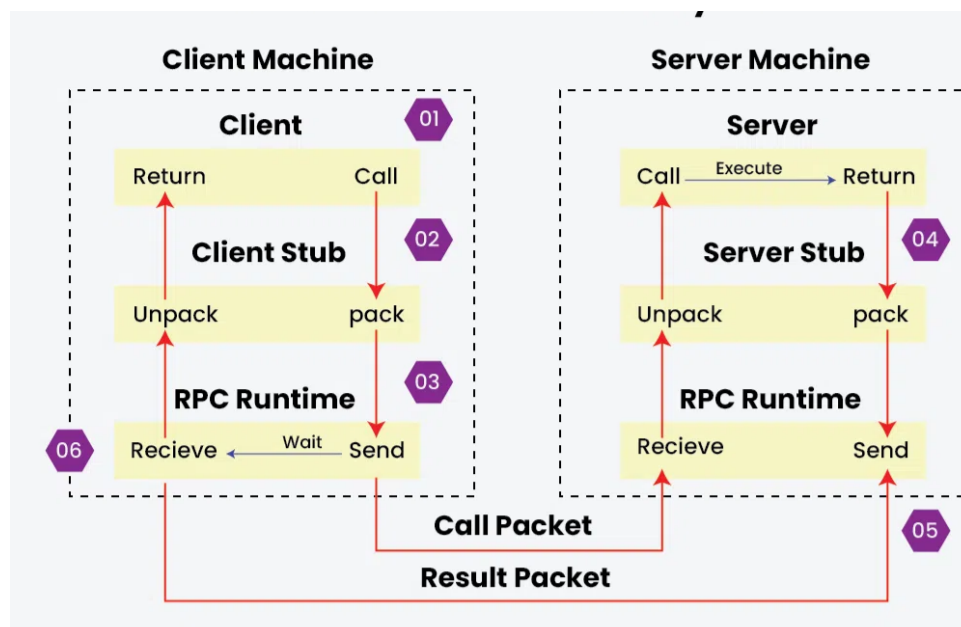


Figure 2.5: RPC architecture showing client and server interaction

## Client Stub Workflow in RPC

In a [Remote Procedure Call](#) (RPC) system, the client stub plays a pivotal role in abstracting the complexity of remote communication. When a client initiates a procedure call, it does so as if the procedure were local. The client stub intercepts this call and begins by *marshalling the procedure*

name, arguments, and metadata into a structured call packet. This packet is then handed off to the client-side RPC runtime, which transmits it across the network to the server. Upon receiving a response from the server—typically in the form of a result packet—the client-side RPC runtime delivers it back to the client stub. The stub then *unmarshals* the result, converting it into a format suitable for the client application, and returns it as though the procedure had executed locally. This seamless workflow allows distributed systems to maintain transparency and modularity, enabling developers to invoke remote services without needing to manage low-level networking details.

### Server Stub Workflow in RPC

On the server side of an RPC system, the server stub is responsible for receiving and processing incoming procedure calls from remote clients. When the server-side RPC runtime receives a call packet, it forwards the request to the appropriate server stub. The stub begins by *unmarshalling* the packet, *extracting the procedure name and arguments*. It then invokes the corresponding procedure implementation on the server, passing in the extracted arguments. Once the procedure completes execution, the server stub *marshals* the result into a response packet. This packet is then sent back through the server-side RPC runtime to the client. The server stub thus acts as a bridge between the network layer and the actual server logic, ensuring that remote calls are correctly interpreted and executed within the server's address space.

### Challenges in RPC Systems

Remote Procedure Call (RPC) simplifies distributed programming by allowing procedures to be invoked across machines as if they were local. However, this abstraction introduces several challenges that are critical to understand, especially for beginners. These challenges include parameter marshaling, data representation, and failure independence.

#### 1. Parameter Passing via Marshaling

Marshaling is the process of converting procedure parameters and return values into a format suitable for transmission over the network.

For example, when a client calls `add(3, 5)`, the integers 3 and 5 must be packed into a message, sent to the server, and then unpacked for execution. This becomes complex when dealing with reference parameters, which are valid only in the client's memory space.

To preserve semantics, RPC systems use a *copy-restore* strategy: the referenced data is copied to the server, and any changes are copied back to the client after execution.

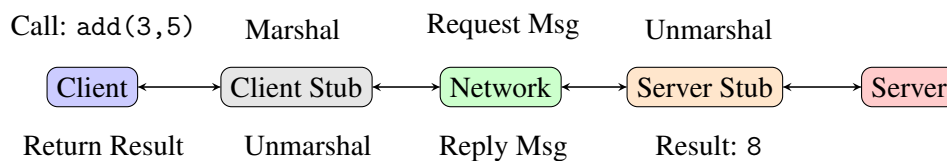


Figure 2.6: RPC marshaling and unmarshaling workflow

#### Another Simple Example:

Sending an integer over the network:

```
int x = 42;
char* msg = serialize(x); // Marshalling
send(msg);
```

#### 2. Data Representation Across Architectures

Clients and servers may run on different architectures. For example, Intel uses little-endian format, while SPARC uses big-endian. RPC systems must standardize data formats to ensure correct interpretation.

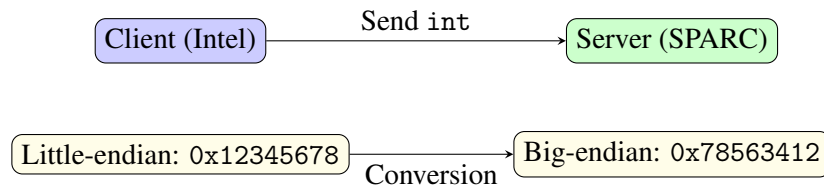


Figure 2.7: Data representation differences across architectures

**3. Failure Independence** In RPC, the client and server are separate entities. If the server crashes or the network fails, the client must handle these failures gracefully.

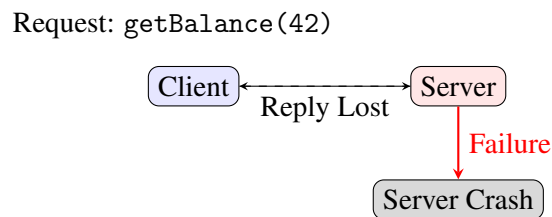


Figure 2.8: Failure independence in RPC systems

**Example** Consider a banking application where a client invokes `getBalance(accountID)`. If the server crashes after processing the request but before sending the reply, the client may retry the call. This could accidentally repeat the operation—such as charging a customer twice—unless the system is designed to safely handle repeated requests. To avoid such problems, RPC systems often use timeouts, retries, and unique request identifiers to ensure safe and consistent behavior.

These challenges highlight the importance of designing RPC systems with careful attention to serialization formats, error handling, and communication protocols. For beginners, understanding these pitfalls is essential to building reliable distributed applications.

## Remote Procedure Call Types

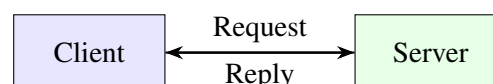
Remote Procedure Calls (RPCs) allow a client to invoke a procedure on a remote server. Depending on how the client and server interact, RPCs can be classified into three main types.

### 1. Synchronous RPC

In synchronous RPC, the client sends a request and waits (blocks) until the server processes the request and returns the result.

**Explanation** This is the most common type of RPC. It behaves like a regular function call: the client pauses until the server responds. While simple, it can be inefficient if the server takes time to respond.

**Example** A banking app sends a request to check account balance. The client waits until the server replies with the balance.



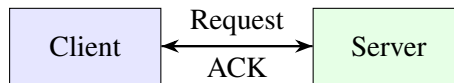
**Client behavior:** Blocks until reply is received.

### 2. Asynchronous RPC

In asynchronous RPC, the client sends a request and receives an immediate acknowledgment (ACK). The client does not wait for the result and continues execution.

**Explanation** This is useful when the client does not need the result immediately or at all. It improves responsiveness and resource usage.

**Example** A sensor device sends temperature data to a server. It doesn't wait for confirmation—it just sends and moves on.



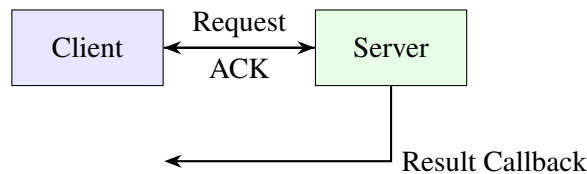
**Client behavior:** Continues execution after ACK.

### 3. Deferred Synchronous RPC

In deferred synchronous RPC, the client sends a request and receives an ACK. Later, the server sends the result back using a second RPC call.

**Explanation** This is useful when the client wants the result but doesn't want to block. It splits the interaction into two phases: request and callback.

**Example** A user submits a video for processing. The server acknowledges the request immediately. Once processing is done, the server notifies the client with the result.



**Client behavior:** Continues execution and receives result later.

## Mini-Project: Simple Chat Using RPC

**Objective** Create a basic RPC system where a client sends a chat message and the server replies with a confirmation.

**Setup** Install the required Python packages:

#### Install Dependencies

```
pip install grpcio grpcio-tools
```

### Step 1: Define the Chat Service

Create a file named `chat.proto`: This file describes a `ChatService` that allows a client to send a message and receive a response.

- The service interface (`ChatService`)
- The RPC method (`SendMessage`)
- The message formats (`ChatRequest` and `ChatReply`)

**chat.proto**

```
syntax = "proto3";

service ChatService {
  rpc SendMessage (ChatRequest) returns (ChatReply);
}

message ChatRequest {
  string user = 1;
  string message = 2;
}

message ChatReply {
  string response = 1;
}
```

Compile the proto file:

**Compile Proto**

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. chat.
proto
```

After running the command, you'll get two files:

- (a) **chat\_pb2.py**: Contains Python classes for your messages (ChatRequest, ChatReply)
  - (b) **chat\_pb2\_grpc.py**: Contains gRPC service classes and stubs for ChatService
- These files are the foundation for building both your server and client in Python.

**Step 2: Server Code**

Create a file named `server.py`:

**server.py**

```
import grpc
from concurrent import futures
import chat_pb2
import chat_pb2_grpc

class ChatServer(chat_pb2_grpc.ChatServiceServicer):
    def SendMessage(self, request, context):
        print(f"{request.user} says: {request.message}")
        return chat_pb2.ChatReply(response=f"Message received from {
            request.user}")

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=2))
    chat_pb2_grpc.add_ChatserviceServicer_to_server(ChatServer(), server)
    server.add_insecure_port('[::]:50051')
    server.start()
    print("Chat server running...")
    server.wait_for_termination()

if __name__ == '__main__':
    serve()
```

## Explanation of Server code

- Step 1:** `import grpc`  
Imports the gRPC library, which provides tools for building and running gRPC servers and clients.
- Step 2:** `from concurrent import futures`  
Imports the futures module to create a thread pool for concurrent request handling.
- Step 3:** `import chat_pb2` and `import chat_pb2_grpc`  
These are generated from the `.proto` file.
- `chat_pb2` contains message classes.
  - `chat_pb2_grpc` contains service classes and stubs.
- Step 4:** `class ChatServer(chat_pb2_grpc.ChatServiceServicer):`  
Defines the server class that implements the service interface from the proto definition.
- Step 5:** `def SendMessage(self, request, context):`  
Implements the RPC method `SendMessage`.
- `request` contains the client's message.
  - `context` provides RPC metadata and control.
- Step 6:** `print(f"{request.user} says: {request.message}")`  
Logs the incoming message to the server console.
- Step 7:** `return chat_pb2.ChatReply(...)`  
Returns a response to the client using the `ChatReply` message type.
- Step 8:** `def serve():`  
Defines the function that starts the gRPC server.
- Step 9:** `server = grpc.server(...)`  
Creates a gRPC server with a thread pool of 2 workers for concurrent handling.
- Step 10:** `chat_pb2_grpc.add_ChatserviceServicer_to_server(...)`  
Registers the `ChatServer` implementation with the gRPC server.
- Step 11:** `server.add_insecure_port('[::]:50051')`  
Binds the server to port 50051 on all available network interfaces.
- Step 12:** `server.start()`  
Starts the server and begins listening for requests.
- Step 13:** `print("Chat server running...")`  
Prints a confirmation message to the console.
- Step 14:** `server.wait_for_termination()`  
Keeps the server running until manually stopped.
- Step 15:** `if __name__ == '__main__': serve()`  
Ensures the server starts only when the script is executed directly.

### Step 3: Client Code

Create a file named `client.py`:

`client.py`

```
import grpc
import chat_pb2
import chat_pb2_grpc

def run():
    channel = grpc.insecure_channel('localhost:50051')
    stub = chat_pb2_grpc.ChatServiceStub(channel)

    response = stub.SendMessage(chat_pb2.ChatRequest(user="Alice",
        message="Hello!"))
    print(response.response)

if __name__ == '__main__':
    run()
```

### Explanation of Client code

**Step 1:** `import grpc`

Imports the gRPC library, which provides tools for creating channels and calling remote procedures.

**Step 2:** `import chat_pb2` and `import chat_pb2_grpc`

These modules are generated from the `.proto` file:

- `chat_pb2` defines message types like `ChatRequest` and `ChatReply`.
- `chat_pb2_grpc` defines the service stub `ChatServiceStub`.

**Step 3:** `def run():`

Defines the main function that executes the client logic.

**Step 4:** `channel = grpc.insecure_channel('localhost:50051')`

Creates a communication channel to the server running on `localhost` at port `50051`. This is an insecure channel (no SSL/TLS), suitable for local testing.

**Step 5:** `stub = chat_pb2_grpc.ChatServiceStub(channel)`

Creates a stub object that allows the client to call RPC methods defined in the service.

**Step 6:** `response = stub.SendMessage(...)`

Sends a `ChatRequest` to the server using the `SendMessage` RPC method. The request includes:

- `user="Alice"`: the sender's name.
- `message="Hello!"`: the message content.

**Step 7:** `print(response.response)`

Prints the server's reply to the console. Expected output: "Message received from Alice"

**Step 8:** `if __name__ == '__main__': run()`

Ensures the `run()` function is executed only when the script is run directly.

### Running the Project

1. Compile the `chat.proto` file.
2. Run the server:



**Run Server**

```
python server.py
```

## 3. Run the client:

**Run Client**

```
python client.py
```

Expected output:

**Sample Output**

```
Message received from Alice
```

## B. Remote Method Invocation

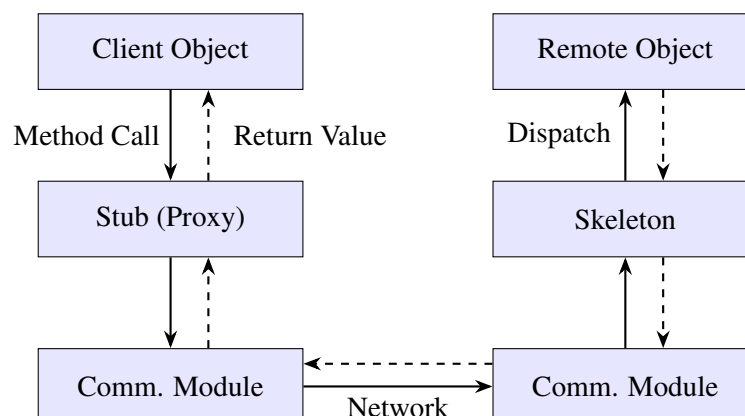
Remote Method Invocation (RMI) extends Remote Procedure Call (RPC) into the object-oriented domain. It allows objects on different machines to invoke each other's methods as if they were local, supporting distributed object-based systems.

Remote Method Invocation (RMI) is a Java API that allows an object running in one Java Virtual Machine (JVM) to invoke methods on an object running in another JVM. This is useful for building distributed applications.

### Key Concepts

| Concept                 | Description                                               |
|-------------------------|-----------------------------------------------------------|
| Remote Object           | An object whose methods can be invoked remotely.          |
| Remote Interface        | Interface defining remotely accessible methods.           |
| Proxy (Stub)            | Local representative of the remote object.                |
| Skeleton & Dispatcher   | Server-side components that handle incoming method calls. |
| Remote Reference Module | Translates between local and remote object references.    |

### RMI Control Flow Diagram



## Step-by-Step Example: Remote Calculator

### 1. Remote Interface

#### Calculator.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
}
```

#### Explanation:

- Defines a remote interface Calculator that extends Remote.
- Declares a method add() that can be invoked remotely.
- The method throws RemoteException to handle communication errors.

### 2. Remote Implementation

#### CalculatorImpl.java

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class CalculatorImpl extends UnicastRemoteObject
    implements Calculator {
    public CalculatorImpl() throws RemoteException {
        super();
    }

    public int add(int a, int b) {
        return a + b;
    }
}
```

#### Explanation:

- Implements the Calculator interface.
- Extends UnicastRemoteObject to enable remote communication.
- Constructor calls super() to export the object.
- Implements the add() method with basic arithmetic.

### 3. Server Setup

#### CalculatorServer.java

```
import java.rmi.Naming;

public class CalculatorServer {
    public static void main(String[] args) throws Exception {
        Calculator calc = new CalculatorImpl();
        Naming.rebind("CalculatorService", calc);
        System.out.println("CalculatorService is running...");
    }
}
```

**Explanation:**

- Instantiates the remote object CalculatorImpl.
- Binds it to the RMI registry under the name "CalculatorService".
- Prints a confirmation message to the console.

### 4. Client Code

#### CalculatorClient.java

```
import java.rmi.Naming;

public class CalculatorClient {
    public static void main(String[] args) throws Exception {
        Calculator calc = (Calculator) Naming.lookup("rmi://localhost/CalculatorService");
        System.out.println("Result: " + calc.add(5, 3));
    }
}
```

**Explanation:**

- Looks up the remote object from the RMI registry.
- Casts it to the Calculator interface.
- Invokes the add() method remotely and prints the result.

## 2.5 Exercises

### Section A: Conceptual Understanding

1. Define Remote Procedure Call (RPC). What distinguishes RPC from traditional local procedure calls?
2. Explain the role of the client stub in RPC. What are its responsibilities during a remote invocation?
3. List and describe the challenges in implementing RPC. Include at least three distinct challenges.
4. Differentiate between synchronous and asynchronous RPCs. How does each affect client-side execution?
5. What is parameter marshaling? Why is it necessary in distributed systems?

### Section B: Application & Analysis

6. **Scenario:** A client sends a reference parameter to a server via RPC.
  - What issues might arise?
  - How can they be resolved?
7. **Diagram Interpretation:** Based on the RPC flow diagram, describe the sequence of actions from client call to server response.
8. Why is failure handling more complex in RPC than in local procedure calls?
9. Explain how differences in architecture (e.g., little-endian vs. big-endian) affect RPC communication.
10. Compare RPC and RMI. What additional capabilities does RMI provide over RPC?

## 2.6 Answers

### Section A: Conceptual Understanding

1. **RPC Definition:** RPC allows a program to execute a procedure on a remote machine as if it were local, abstracting away the communication details.
2. **Client Stub Role:**
  - Acts as a proxy for the remote procedure.
  - Marshals parameters into a request message.
  - Sends the request via transport routines.
  - Unmarshals the reply and returns results to the caller.
3. **RPC Challenges:**
  - **Parameter marshaling:** Converting parameters into transmittable formats.
  - **Data representation:** Ensuring uniform formats across heterogeneous systems.
  - **Failure independence:** Handling crashes and network failures gracefully.
4. **Synchronous vs. Asynchronous RPCs:**
  - **Synchronous:** Client blocks until server responds.
  - **Asynchronous:** Client continues execution after sending request; server may respond later.
5. **Parameter Marshaling:** The process of packing parameters into a message for transmission. It ensures compatibility and correctness across different systems.

### Section B: Application & Analysis

6. **Reference Parameter Issues:**
  - **Invalid references:** Server cannot access client memory.
  - **No reflection of changes:** Updates on server don't affect client.
  - **Solution:** Use copy/restore semantics — copy data to server, and copy back results.
7. **RPC Flow Sequence:**
  - Client calls stub → stub marshals data → sends request → server stub receives → unmarshals → executes procedure → marshals result → sends response → client stub receives → returns result.
8. **Failure Independence Complexity:** Remote systems can fail independently. Clients must handle server crashes, network issues, and partial failures — unlike local calls where both components share fate.
9. **Architecture Differences:** Data types may vary in size and format. For example, integers may be 4 bytes on one system and 8 on another. Endianness affects byte order, requiring standardization in message formats.
10. **RPC vs. RMI:**
  - **RPC:** Works with procedures and basic data types.
  - **RMI:** Supports object references and method calls in object-oriented systems, enabling richer interactions.

## 2.7 Multiple-Choice Quiz

- Q1.** What is the primary purpose of Remote Procedure Calls (RPC)?
- A. To execute procedures locally
  - B. To abstract remote communication as local procedure calls
  - C. To manage memory across distributed systems
  - D. To provide direct access to remote hardware
- Q2.** Which of the following is NOT a challenge in implementing RPC?
- A. Parameter marshaling
  - B. Data representation differences
  - C. Failure independence
  - D. Direct memory access between client and server
- Q3.** What is the role of the client stub in RPC?
- A. Execute server-side logic
  - B. Marshal parameters and send request
  - C. Handle server failures
  - D. Manage server memory
- Q4.** What does marshaling refer to in RPC?
- A. Encrypting parameters
  - B. Packing parameters for transmission
  - C. Executing remote code
  - D. Managing memory allocation
- Q5.** What distinguishes synchronous from asynchronous RPC?
- A. Synchronous RPC blocks the client until response
  - B. Asynchronous RPC uses encryption
  - C. Synchronous RPC avoids marshaling
  - D. Asynchronous RPC is only used locally
- Q6.** What is the 'Copy/Restore' method used for?
- A. Encrypting reference parameters
  - B. Passing reference parameters safely
  - C. Avoiding marshaling
  - D. Improving performance
- Q7.** What is the role of the server stub?
- A. Execute client-side logic
  - B. Unmarshal parameters and invoke procedure
  - C. Handle client failures
  - D. Manage network connections
- Q8.** Why is data representation a challenge in RPC?
- A. All systems use identical formats
  - B. Different architectures represent data differently
  - C. RPC avoids data transmission
  - D. Data representation is irrelevant in RPC
- Q9.** What does the Remote Reference Module do in RMI?
- A. Encrypts remote calls
  - B. Translates object references across machines
  - C. Executes remote procedures
  - D. Handles marshaling
- Q10.** What advantage does RMI have over traditional RPC?
- A. Faster execution
  - B. Supports object references
  - C. Avoids marshaling

- D. Requires no stubs
- Q11.** Which scenario best illustrates synchronous RPC limitations?
  - A. Client receives immediate response
  - B. Client blocks, causing UI freeze
  - C. Server handles multiple requests
  - D. Client uses callback
- Q12.** Why choose asynchronous RPC in microservices?
  - A. Simplifies debugging
  - B. Reduces memory usage
  - C. Improves scalability and responsiveness
  - D. Avoids marshaling
- Q13.** What trade-off does 'Copy/Restore' introduce?
  - A. Performance vs correctness
  - B. Simplicity vs abstraction
  - C. Correctness vs overhead
  - D. Speed vs encryption
- Q14.** When does marshaling fail to preserve semantics?
  - A. Passing primitive types
  - B. Passing shared memory pointers
  - C. Passing strings
  - D. Passing binary blobs
- Q15.** What abstraction benefit of RPC can mislead developers?
  - A. Exposes network details
  - B. Hides latency and failure risks
  - C. Requires manual memory management
  - D. Forces synchronous calls
- Q16.** What happens if the server crashes during an RPC?
  - A. Client receives partial response
  - B. Client blocks indefinitely unless timeout is handled
  - C. Server automatically retries
  - D. Client switches to local execution
- Q17.** Which design choice improves fault tolerance in RPC systems?
  - A. Using synchronous calls only
  - B. Avoiding retries
  - C. Implementing timeouts and retries
  - D. Disabling marshaling
- Q18.** What is a drawback of hiding remote communication behind local call abstraction?
  - A. Developers overestimate latency
  - B. Developers ignore network failures
  - C. Developers avoid using RPC
  - D. Developers must manage memory manually
- Q19.** Which of the following best describes the role of interface definition in RPC?
  - A. It defines encryption protocols
  - B. It specifies procedure signatures for stub generation
  - C. It manages network connections
  - D. It handles marshaling logic
- Q20.** Which of the following best explains why RPC abstraction can lead to performance issues in distributed systems?
  - A. It forces developers to use synchronous calls only.
  - B. It hides network latency and failure risks, encouraging misuse of remote calls.

- C. It requires manual memory management across machines.
  - D. It prevents the use of asynchronous communication.
- Q21.** A developer notices that retrying a failed RPC causes duplicate entries in a database. What is the most appropriate solution?
- A. Use asynchronous RPC to avoid blocking.
  - B. Encrypt the RPC payload to prevent duplication.
  - C. Make the RPC operation idempotent.
  - D. Increase the server's timeout threshold.
- Q22.** Why is passing pointers between client and server problematic in RPC?
- A. Pointers are encrypted during transmission.
  - B. Pointers reference memory addresses that are invalid across address spaces.
  - C. Pointers are automatically marshaled by the stub.
  - D. Pointers are converted to strings before transmission.
- Q23.** In designing an RPC system for real-time applications, which factor is most critical?
- A. Minimizing interface definition complexity.
  - B. Guaranteeing low-latency and predictable response times.
  - C. Using synchronous calls for consistency.
  - D. Avoiding marshaling to reduce overhead.





### 3. Architectures Models in Distributed Systems

#### Course Map

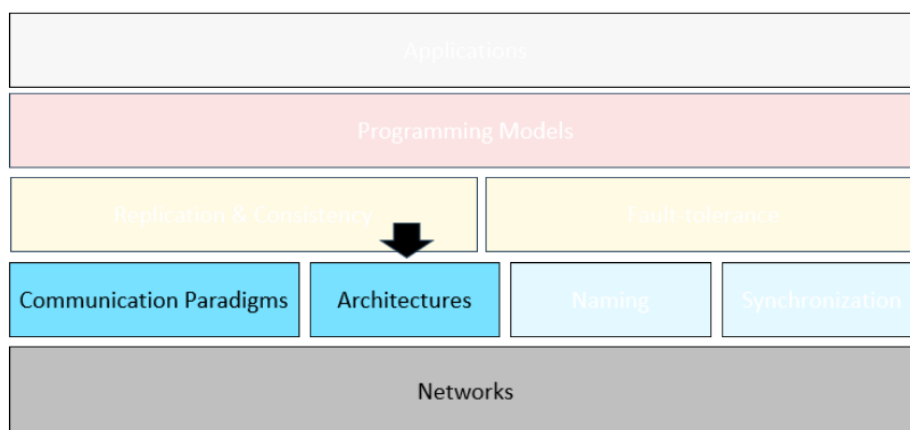


Figure 3.1: Architectures

Distributed systems consist of multiple independent computers that work together to perform tasks. The way these computers are organized—how they communicate, coordinate, and share responsibilities—is defined by **the system's architecture**.

Understanding architectural models is essential for designing systems that are scalable, fault-tolerant, and efficient. This chapter introduces key architectural paradigms, including Master-Slave, Peer-to-Peer, and Hybrid models, along with structural patterns like tiering and layering.

*Analogy:* Booking a flight online involves multiple systems—user interface, search engine, databases—all working together. Their organization is the architecture.

#### Characterizing Distributed Systems

Distributed systems can be described along three dimensions:

- **Entities:** Clients, servers, peers—who participates?
- **Communication Paradigms:** Sockets, RPC—how do they talk?

- **Roles and Responsibilities:** Coordination, computation, storage—*what do they do?*

*Analogy:* A team project with leaders, researchers, writers, and editors. The way tasks are assigned and coordinated is the architecture.

### 3.1 Architectural Models

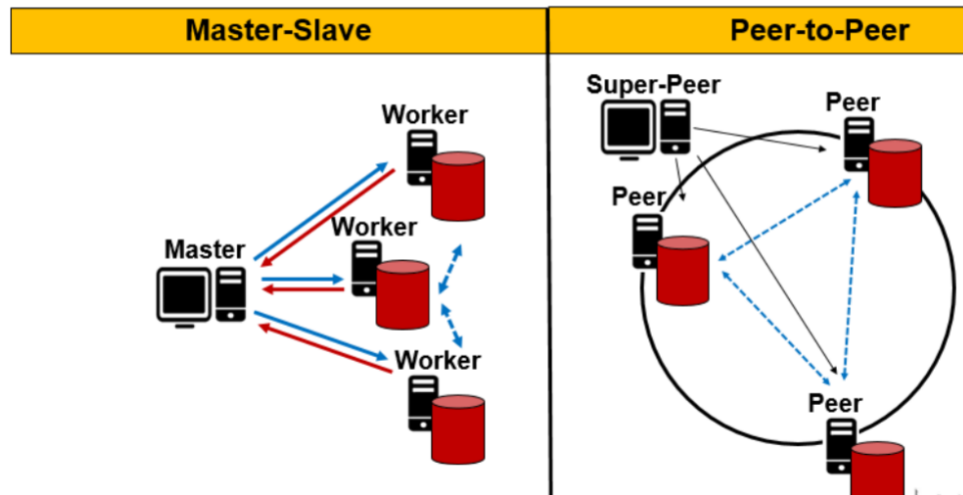


Figure 3.2: Architectural Models

#### 3.1.1 Master-Slave Architecture

The Master-Slave architecture is one of the most traditional and straightforward organizational models in distributed systems. In this setup, a single node—referred to as the *master*—acts as the central coordinator, while the remaining nodes—called *slaves* or *workers*—carry out tasks assigned by the master. This hierarchical structure simplifies decision-making, as all control and scheduling responsibilities are centralized.

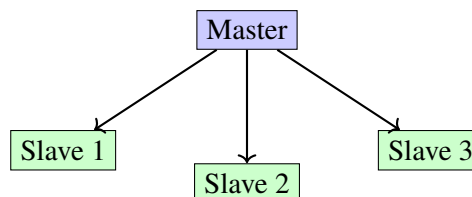
**Characteristics:**

- Hierarchical structure
- Centralized decision-making
- Vulnerable to single point of failure
- Limited scalability

**Example 1:** A teacher (master) assigns homework to students (slaves), who complete the tasks and report back.

**Example 2:** A web server handling multiple client requests. If the server fails, the system halts.

**Trend:** Still used in systems like database replication and job scheduling, but often replaced by more fault-tolerant models.



#### 3.1.2 Peer-to-Peer Architectures

Peer-to-Peer (P2P) architecture represents a paradigm shift from traditional client-server models by decentralizing control and distributing responsibilities across all participating nodes. In a P2P

system, each node—called a *peer*—can act both as a client and a server, initiating requests and responding to others. This design promotes scalability, fault tolerance, and resource sharing, making it ideal for applications like file sharing, blockchain, collaborative platforms, and decentralized storage. Unlike centralized systems, where a single server may become a bottleneck or point of failure, P2P networks thrive on dynamic participation and resilience. However, not all P2P systems are built the same. Depending on how peers are organized and how data is located, P2P architectures can be classified into several types, each with its own trade-offs in efficiency, complexity, and robustness.

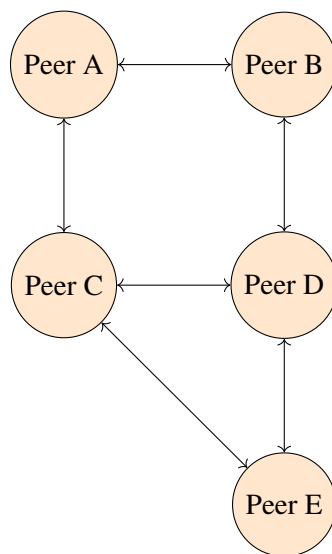
## 1. Unstructured P2P Architecture

Unstructured P2P systems are the most flexible and decentralized form of peer-to-peer architecture. In this model, peers connect randomly without any global organization or indexing. When a peer wants to find data, it typically floods the network with queries, asking neighboring nodes to forward the request until the data is found. This approach is simple to implement and highly resilient to peer churn (nodes joining or leaving), but it can be inefficient, especially for rare data.

**Example:** Gnutella file-sharing network.

**Analogy:** Asking everyone in a crowded room if they've seen your lost keys.

**Trend:** Less common today due to inefficiency, but useful in highly dynamic environments.



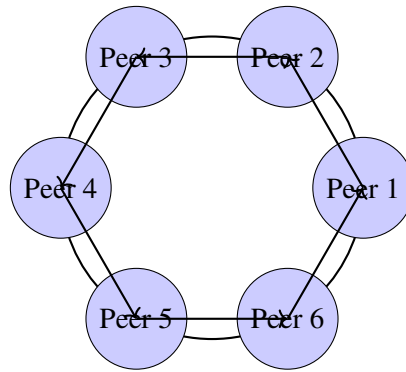
## 2. Structured P2P Architecture

Structured P2P systems introduce organization by using distributed hash tables (DHTs) to map data to specific nodes. Each peer is assigned a portion of the keyspace, and queries are routed deterministically based on the key. This makes lookups fast and efficient, often requiring only logarithmic time relative to the number of nodes.

**Example:** Chord, Kademlia, Pastry.

**Analogy:** A well-organized library with an index for every book.

**Trend:** Widely used in decentralized storage and blockchain systems.



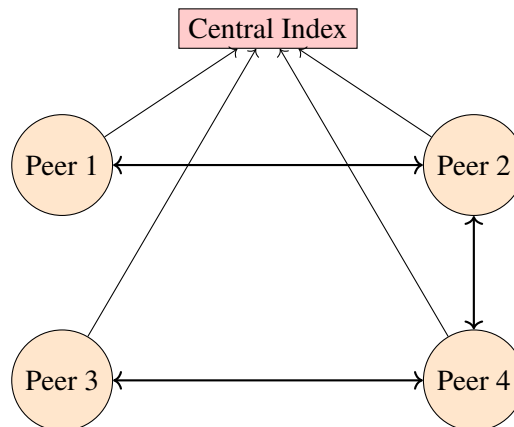
### 3. Hybrid P2P Architecture

Hybrid P2P architectures blend peer-to-peer principles with centralized elements to improve performance and usability. Typically, a central server helps with tasks like indexing or peer discovery, while actual data exchange happens directly between peers. This model reduces search overhead and improves scalability while retaining the decentralized nature of data transfer.

**Example:** Napster.

**Analogy:** A matchmaking service that connects people, but the conversation happens directly.

**Trend:** Common in media sharing, collaborative tools, and early P2P systems.



## Comparison of P2P Architecture Types

| Feature         | Unstructured P2P            | Structured P2P                    | Hybrid P2P                             |
|-----------------|-----------------------------|-----------------------------------|----------------------------------------|
| Topology        | Random connections          | Organized via DHT                 | Central index + peer links             |
| Search Method   | Flooding or random walk     | Deterministic lookup              | Centralized discovery, direct transfer |
| Efficiency      | Low for rare data           | High and predictable              | Moderate                               |
| Fault Tolerance | High                        | Moderate to high                  | Depends on central node                |
| Scalability     | Good, but inefficient       | Excellent                         | Good with central bottleneck risk      |
| Example Systems | Gnutella, Freenet           | Chord, Kademlia, Pastry           | Napster, early Skype                   |
| Analogy         | Asking everyone in a crowd  | Indexed library system            | Matchmaking service                    |
| Use Cases       | Dynamic networks, anonymity | Decentralized storage, blockchain | Media sharing, hybrid apps             |

Table 3.1: Comparison of Peer-to-Peer Architecture Types

Selecting the appropriate Peer-to-Peer architecture depends on the specific goals and constraints of the system being designed. If simplicity and resilience to frequent node changes are priorities, **Unstructured P2P** is a good fit—especially for applications where data availability is high and search precision is less critical. For systems requiring efficient and predictable data lookup, such as decentralized storage or blockchain platforms, **Structured P2P** offers strong performance through organized routing and indexing. Meanwhile, **Hybrid P2P** is ideal when a balance between centralized coordination and decentralized data exchange is needed, such as in media sharing or collaborative tools. Designers must weigh factors like scalability, fault tolerance, search efficiency, and maintenance complexity to choose the model that best aligns with their application’s requirements.

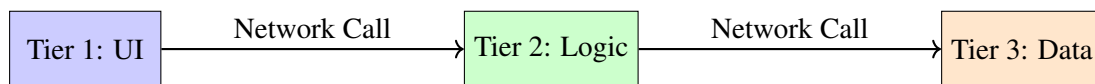
## 3.2 Architectural Patterns: Tiering vs. Layering

### 3.2.1 Tiering: Horizontal Splitting

Tiering is an architectural pattern that divides a system into distinct tiers based on functionality, with each tier deployed separately across machines or services. This horizontal splitting allows for clear separation of concerns and modular deployment. A classic example is an airline search application:

- **Tier 1:** User interface for flight search
- **Tier 2:** Application logic for filtering and processing
- **Tier 3:** Data access for retrieving flight schedules and prices

This structure enables teams to scale and maintain each tier independently. The benefits include easier maintenance and clearer responsibility boundaries. However, tiering introduces complexity in deployment and communication, and can increase latency due to network hops between tiers. It's like a relay race—each runner (tier) has a defined role, but passing the baton adds coordination overhead.

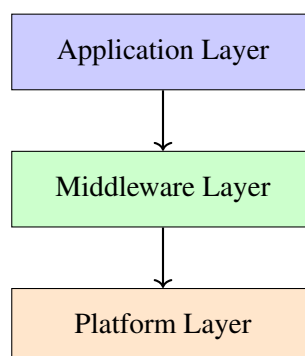


### 3.2.2 Layering: Vertical Organization

Layering organizes a system into stacked layers, where each layer builds on the services provided by the one below it. This vertical structure simplifies design by hiding complexity and promoting abstraction. Common layers include:

- **Applications:** End-user services like browsers or email clients
- **Middleware:** Communication tools like RPC, messaging, or service registries
- **Platform:** Operating system and hardware infrastructure

Layering is especially useful in distributed systems where abstraction helps manage complexity. For example, an application doesn't need to know how the OS schedules threads—it simply relies on the platform layer. This pattern promotes modularity and reusability, making it easier to swap or upgrade components. The analogy of a layered cake fits perfectly: each layer adds flavor and builds on the previous one, but the eater (user) only sees the top.



Comparison of Tiering and Layering

| Aspect     | Tiering (Horizontal)                                     | Layering (Vertical)                                   |
|------------|----------------------------------------------------------|-------------------------------------------------------|
| Definition | Divides system by functional roles across physical tiers | Organizes system by logical abstraction levels        |
| Deployment | Each tier may run on separate machines                   | Layers may reside on the same machine or across tiers |
| Example    | Airline search app: UI, logic, data tiers                | OS stack: application, middleware, platform           |
| Analogy    | Relay race with baton passing                            | Layered cake with stacked flavors                     |
| Pros       | Clear separation of concerns, modular scaling            | Simplifies design, hides complexity, promotes reuse   |
| Cons       | Added latency, network traffic, deployment complexity    | Can obscure performance issues, harder debugging      |
| Use Cases  | Web apps, enterprise systems, client-server models       | OS design, middleware platforms, layered protocols    |

Table 3.2: Comparison of Tiering and Layering Architectural Patterns



### 3.3 Exercises

- Scenario 1.** You are designing a distributed file storage system. You must choose between a Master-Slave and Peer-to-Peer architecture. The system must support high availability and scale to millions of users. Which architecture would you choose and why?
- Scenario 2.** A video streaming platform uses a hybrid P2P model. During peak hours, users complain about slow content delivery. What architectural bottlenecks might be causing this, and how could you improve the system?
- Scenario 3.** A search engine backend is structured as a three-tier architecture. The application logic tier is experiencing high latency. What strategies could you use to isolate and resolve the issue?
- Scenario 4.** A multiplayer game uses structured P2P for matchmaking. Players report frequent disconnects and slow game starts. How does churn affect this system, and what mitigation strategies would you recommend?
- Scenario 5.** A university deploys a distributed learning platform using tiering. The UI tier is hosted on a separate server from the application and data tiers. Students report delays in loading course materials. What architectural adjustments could improve performance?
- Scenario 6.** A blockchain-based voting system uses a pure P2P architecture. How would you ensure data integrity and availability during high churn and network partitioning?
- Scenario 7.** An e-commerce site uses a Master-Slave model for inventory management. The master node crashes during a major sale event. What are the consequences, and how could you redesign the system to avoid this failure?
- Scenario 8.** A distributed chat application uses unstructured P2P. Users complain that old messages are no longer accessible. What architectural limitation is causing this, and how could you redesign the system?
- Scenario 9.** A cloud-based analytics platform uses layering to separate data ingestion, processing, and visualization. A bug in the processing layer causes incorrect results. How does layering help isolate and fix the issue?
- Scenario 10.** A peer-to-peer file sharing system uses overlay networks. Users report that downloads are slow despite good internet connections. What might be causing this, and how could overlay routing be optimized?
- Scenario 11.** A distributed sensor network uses structured P2P to collect environmental data. Some sensors frequently go offline. How does this affect the system, and what architectural changes could improve robustness?
- Scenario 12.** A travel booking system uses a three-tier architecture. The database tier is under heavy load. What caching or replication strategies could be used to reduce pressure on the data tier?
- Scenario 13.** A social media platform uses middleware to abstract communication between services. A developer wants to add a new messaging feature. How does middleware simplify this integration?
- Scenario 14.** A distributed ledger system uses hybrid P2P with central nodes for coordination. One central node becomes overloaded. What architectural redesign could distribute the load more effectively?

### 3.4 Answers

- Answer 1.** Peer-to-Peer is preferable due to its decentralized nature, eliminating SPOF and enabling horizontal scalability across millions of users.
- Answer 2.** Bottlenecks may stem from overloaded central servers. Introduce more super-peers or decentralize coordination to reduce load and improve throughput.
- Answer 3.** Use profiling tools to monitor latency sources. Consider caching, load balancing, or scaling the application logic tier horizontally.
- Answer 4.** Churn disrupts routing and peer availability. Use redundancy, heartbeat protocols, and backup peers to maintain stability.
- Answer 5.** Introduce caching at the UI tier, optimize inter-tier communication, and consider edge servers to reduce latency.
- Answer 6.** Use replication, consensus algorithms (e.g., Raft), and distributed hash tables to maintain integrity and availability under churn.
- Answer 7.** The master node is a SPOF. Use leader election protocols or replicate the master role across multiple nodes for fault tolerance.
- Answer 8.** Unstructured P2P lacks guaranteed data persistence. Introduce structured indexing or hybrid models with backup nodes for message retention.
- Answer 9.** Layering isolates bugs to specific layers. Fixes in the processing layer won't affect ingestion or visualization, simplifying debugging.
- Answer 10.** Overlay routing may not align with physical paths. Optimize with proximity-aware routing or dynamic peer selection.
- Answer 11.** Offline sensors break routing paths. Use redundant paths, backup nodes, and dynamic reconfiguration to maintain data flow.
- Answer 12.** Use read replicas, caching layers (e.g., Redis), and load balancing to reduce direct pressure on the database tier.
- Answer 13.** Middleware abstracts communication, allowing the new messaging service to plug into existing APIs without modifying lower layers.
- Answer 14.** Distribute coordination across multiple central nodes or promote capable peers to super-peer roles to balance load.

### 3.5 Multiple-Choice Quiz

- Q1.** In a Master-Slave architecture, what is the primary drawback when scaling out the system?
- Increased latency due to peer routing.
  - The master becomes a performance bottleneck.
  - Lack of coordination among slaves.
  - Difficulty in maintaining consistency.
- Q2.** Why might a designer prefer a Peer-to-Peer architecture for a content-sharing platform?
- It simplifies decision-making.
  - It avoids single points of failure and scales well.
  - It centralizes control for better performance.
  - It reduces metadata overhead.
- Q3.** What is a key challenge in unstructured P2P networks?
- Maintaining consistent hashing.
  - Handling metadata replication.
  - Locating data efficiently.
  - Preventing centralization.
- Q4.** Which architectural pattern best supports modular debugging and abstraction?
- Tiering
  - Layering

- c. Master-Slave
  - d. Hybrid P2P
- Q5.** What is a consequence of high churn in structured P2P systems?
- a. Improved robustness.
  - b. Simplified routing.
  - c. Frequent metadata updates and instability.
  - d. Reduced overlay complexity.
- Q6.** Why might unpopular data disappear in a pure P2P system?
- a. It is stored on central servers.
  - b. Peers stop sharing it, making it inaccessible.
  - c. It is replicated across all nodes.
  - d. It is cached permanently.
- Q7.** What is the main benefit of hybrid P2P architectures?
- a. Eliminates all central coordination.
  - b. Combines scalability with centralized discovery.
  - c. Reduces the need for overlay networks.
  - d. Guarantees data consistency.
- Q8.** In a three-tier architecture, which tier typically handles ranking and business logic?
- a. Presentation tier
  - b. Application tier
  - c. Data tier
  - d. Middleware tier
- Q9.** What is a disadvantage of layering in distributed systems?
- a. Increased modularity
  - b. Simplified abstraction
  - c. Added latency due to vertical communication
  - d. Easier debugging
- Q10.** Which of the following best describes the role of middleware?
- a. It manages hardware resources.
  - b. It provides low-level networking protocols.
  - c. It abstracts heterogeneity and simplifies programming.
  - d. It stores application data.
- Q11.** What architectural strategy helps reduce load on a database tier?
- a. Adding more presentation servers
  - b. Using structured P2P
  - c. Implementing caching and replication
  - d. Switching to UDP
- Q12.** What does tiering primarily organize in a distributed system?
- a. Vertical abstraction of services
  - b. Horizontal placement of functionality across servers
  - c. Metadata replication
  - d. Peer routing
- Q13.** Which layer in a layered architecture typically interacts directly with hardware?
- a. Application layer
  - b. Middleware layer
  - c. Platform layer
  - d. Presentation layer

## 4. Naming Systems in Distributed Systems

### Course Map

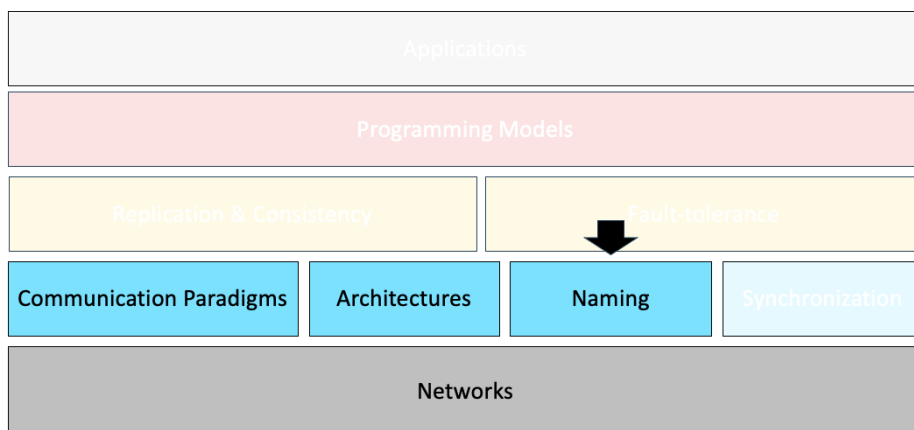


Figure 4.1: Naming

### 4.1 Introduction to Naming

#### Key Idea

Naming is a foundational concept in distributed systems. It enables entities—such as files, processes, services, or devices—to be identified and located across a network of machines. Unlike centralized systems, distributed environments span multiple nodes, networks, and administrative domains, making naming both more powerful and more complex.

#### Why Naming Matters

Naming provides a level of abstraction that decouples the identity of an entity from its physical location. This abstraction is essential for:

- **Mobility:** Entities can move across machines without changing their name.

- **Replication:** Multiple copies of an entity can share the same name.
- **Fault Tolerance:** Systems can redirect requests to backup entities if the primary fails.
- **Abstraction:** Hiding low-level details behind symbolic names.
- **Modularity:** Allowing components to be reused and independently managed.
- **Communication:** Enabling interaction between hardware and software layers.

Think of naming as the “address book” of a distributed system. Without it, every interaction would require low-level knowledge of where things are and how to reach them—an impossible task at scale.

## Types of References

To understand naming, we must distinguish between three types of references:

| Reference Type | Description                              | Example           |
|----------------|------------------------------------------|-------------------|
| Name           | Human-readable or system-generated label | earth.html        |
| Address        | Location-dependent reference             | 55.55.55.55:8888  |
| Identifier     | Unique, persistent reference             | 02:60:8c:02:b0:5a |

Table 4.1: Types of References in Distributed Systems

### Note

A robust naming system allows you to use a name like `earth.html` and resolve it to an address and identifier behind the scenes.

## Real-World Analogy

Imagine a university campus:

- Room 101 is a name for a physical space.
- Professor ID 12345 is a unique identifier.
- "Physics Lab" is a symbolic name used by students.

Each serves a different purpose—just like names in computer systems.

## Name Resolution

**Name resolution** is the act of translating a name into a usable reference. Consider the following example:

```
http://www.cdk5.net:8888/WebExamples/earth.html
```

**This involves several layers of resolution:**

1. **DNS Lookup:** `www.cdk5.net` → IP address `55.55.55.55`
2. **Port Resolution:** `:8888` → Specific service on the host
3. **Path Resolution:** `/WebExamples/earth.html` → File location
4. **MAC Resolution:** IP → MAC address `02:60:8c:02:b0:5a` (via ARP)

Each step abstracts away complexity, allowing users to interact with high-level names while the system handles the details.

## Name Resolution Pipeline

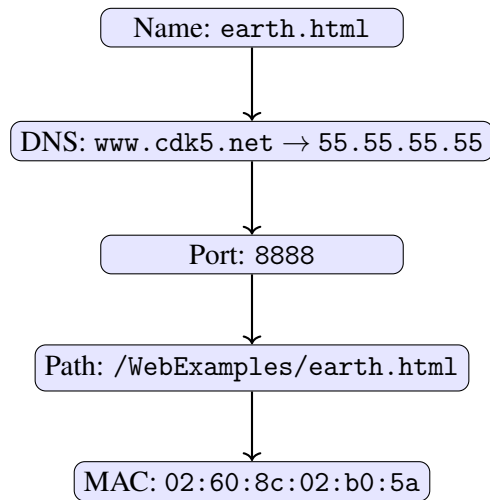


Figure 4.2: Name Resolution Pipeline in Distributed Systems

### Exercise: Trace the Resolution

Given the URL <http://files.example.org:8080/data/report.pdf>, identify:

- The name
- The address
- The identifier (assume MAC resolution is available)
- The steps required to resolve the full reference

## 4.2 Naming Systems in Distributed Systems

Distributed systems rely on naming schemes to identify and locate entities such as files, devices, or services. Three primary naming models are commonly used: **Flat Naming**, **Structured Naming**, and **Attribute-Based Naming**.

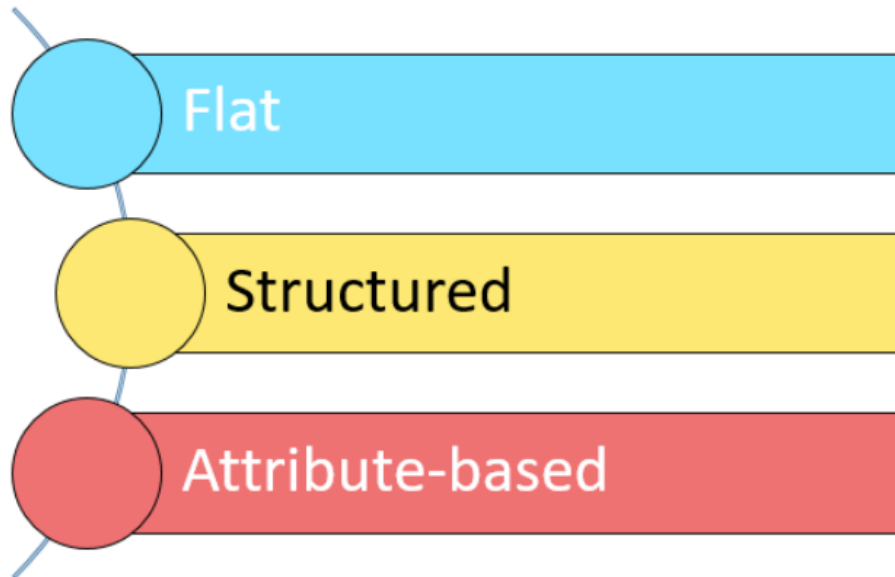


Figure 4.3: Naming Systems Types

### 4.2.1 Flat Naming

Flat naming assigns each entity a globally unique identifier without embedding any structural or location-based information. These identifiers are typically opaque and non-human-readable.

- **Example:** MAC addresses (e.g., 02:60:8c:02:b0:5a) or UUIDs.
- **Advantages:** Supports mobility and location independence; ideal for peer-to-peer and mobile systems.
- **Challenges:** Requires auxiliary mechanisms (e.g., broadcasting, forwarding pointers) for name resolution.

Name resolution is the process of mapping a name to the entity's current location or address. This is essential in flat naming systems where names lack embedded location data.

## Types of Flat Naming

### 1. Broadcasting

**Mechanism:** The system sends a query to all nodes asking who holds the name.

**Pros:** Simple, decentralized

**Cons:** Inefficient in large networks; high traffic

**Analogy:** Shouting a name in a crowded room



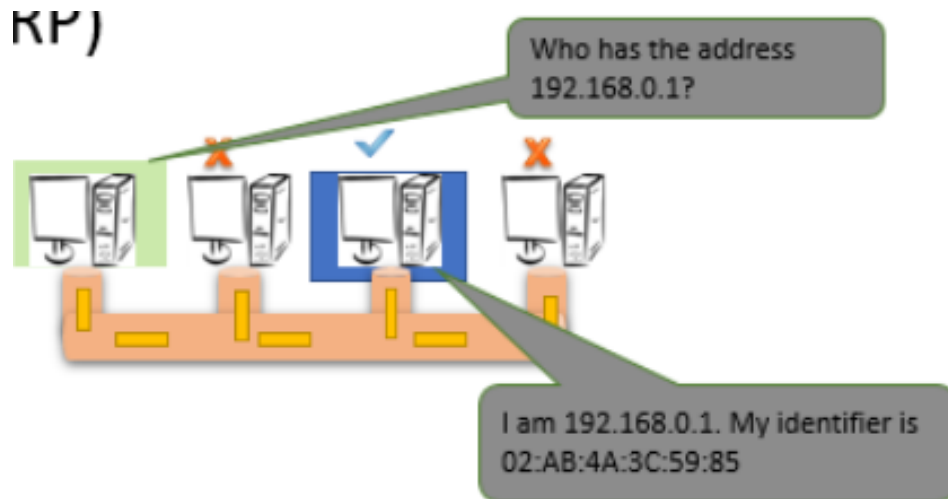


Figure 4.4: Broadcasting Name Resolution

## 2. Forwarding Pointers

**Mechanism:** When an entity moves, it leaves a pointer at its old location.

**Pros:** Efficient for mobile entities

**Cons:** Pointer chains can grow long; requires cleanup

**Analogy:** Leaving sticky notes at each previous location

When an object moves from A (e.g., P2) to B (e.g., P3),

- It leaves a client stub at A (i.e., P2)
- It installs a server stub at B (i.e., P3)

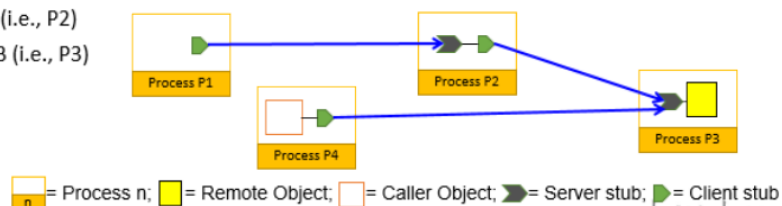


Figure 4.5: Forward Pointer Name Resolution

## 3. Home-Based Name Resolution Process

In home-based name resolution, each mobile entity is assigned a static "home node" that acts as a central authority for tracking its current location. When the entity moves to a new network or host, it updates its home node with its new foreign address. The following steps outline the sequence of communication in a home-based name resolution system for mobile IP:

### Step 1: Location Update

The mobile entity moves to a new network and registers its current *foreign address* with its designated *home node*.



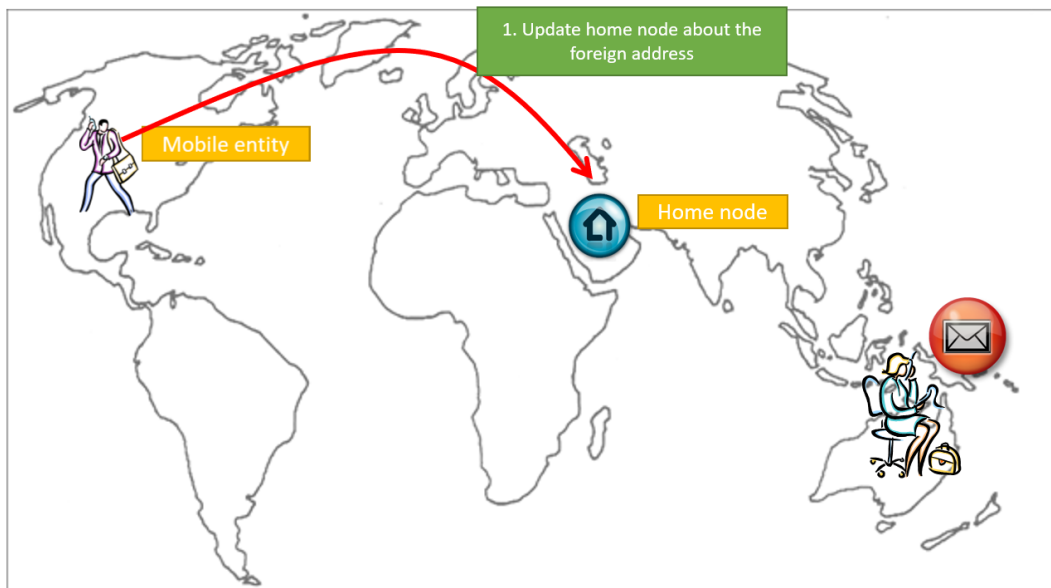


Figure 4.6: **Step 1:** Location Update Forward Pointer Name Resolution

### Step 2: Initial Packet Transmission

A client sends a packet to the mobile entity's *home address*, unaware that the entity has relocated.

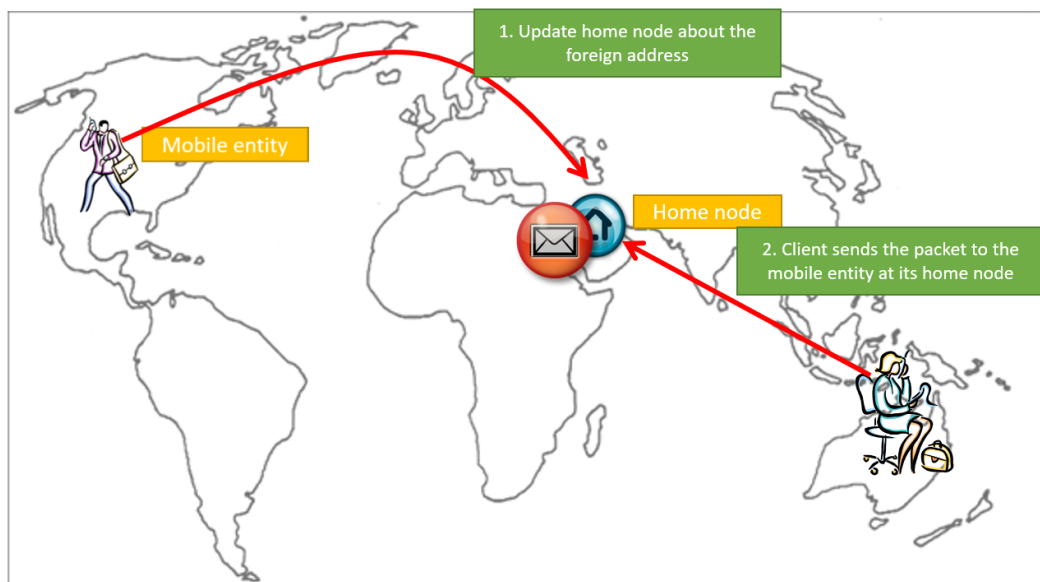


Figure 4.7: **Step 2:** Initial Packet Forward Pointer Name Resolution

### Step 3: Forwarding and Notification

- The home node intercepts the packet and forwards it to the mobile entity's *foreign address*.
- Simultaneously, the home node replies to the client with the mobile entity's updated IP address.

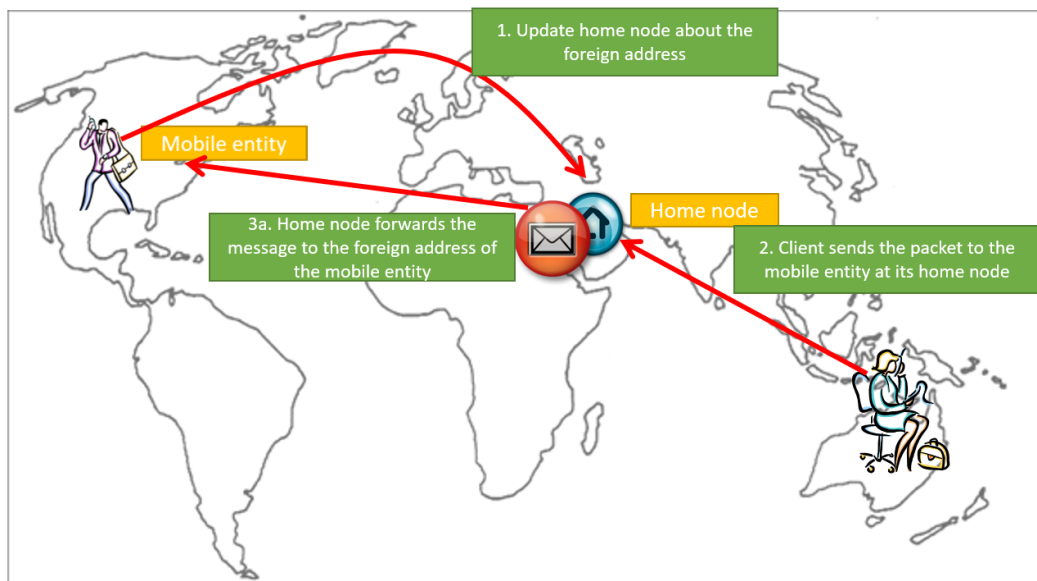


Figure 4.8: **Step 3:** Forwarding and Notification Forward Pointer Name Resolution

#### Step 4: Direct Communication

The client uses the foreign address to send all subsequent packets directly to the mobile entity, bypassing the home node.

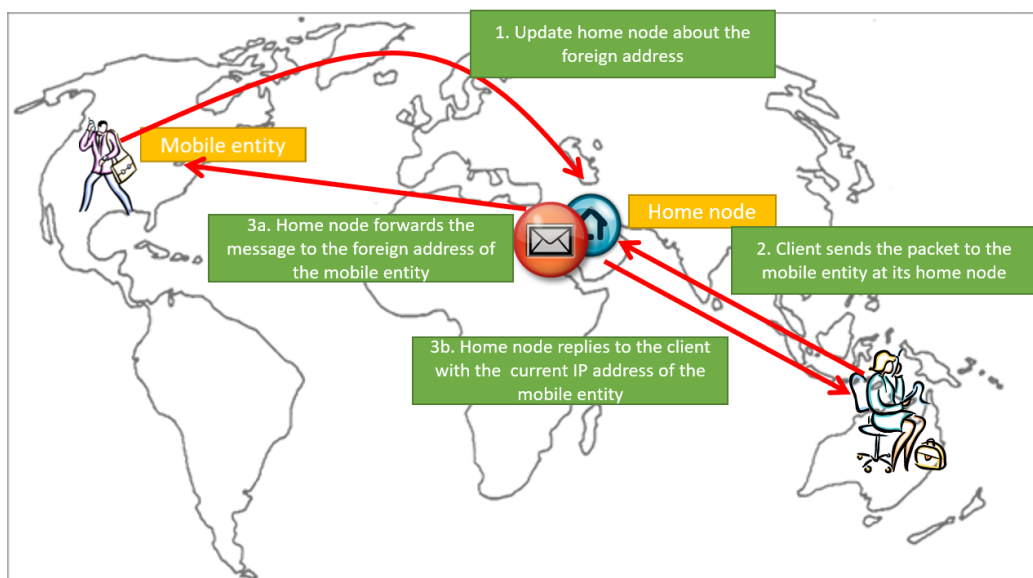


Figure 4.9: **Step 4:** Forwarding and Notification Forward Pointer Name Resolution

## 4. Distributed Hash Table DHT

A **Distributed Hash Table (DHT)** is a decentralized data structure that maps keys to values across a network of nodes. It enables scalable and fault-tolerant name resolution in distributed systems.

Each node is responsible for a portion of the keyspace, and queries are routed efficiently using structured overlays.

## Key Properties

- **Scalability:** Supports millions of nodes and keys.
- **Decentralization:** No central authority or single point of failure.
- **Efficiency:** Lookup operations typically require  $\mathcal{O}(\log n)$  hops.
- **Fault Tolerance:** Robust against node joins and failures.

## Chord Protocol

Chord is a well-known DHT protocol that organizes nodes in a circular keyspace of size  $2^m$ , where  $m$  is the number of bits in the hash function output (a ring). Each node maintains a finger table with pointers to other nodes at exponentially increasing distances.

- Each node has an ID:  $n_i = \text{hash}(IP_i)$
- Each key  $k$  is stored at the node whose ID is the first to equal or succeed  $k$  in the ring.
- Each node maintains a **finger table** for efficient routing.

## Finger Tables

- Store pointers at exponential distances
- Time complexity:  $\mathcal{O}(\log n)$

| Entry    | Finger Table FTp[i]    | Meaning      |
|----------|------------------------|--------------|
| 1        | $\text{succ}(p + 2^0)$ | Next node    |
| 2        | $\text{succ}(p + 2^1)$ | 2 hops ahead |
| $\vdots$ | $\vdots$               | $\vdots$     |

Table 4.2: Chord Finger Table Entries

For node  $n$ , the  $i$ -th entry in its finger table is:

$$FT_n[i] = \text{successor}(n + 2^{i-1}) \mod 2^m$$

This allows Chord to resolve lookups in  $\mathcal{O}(\log n)$  time.

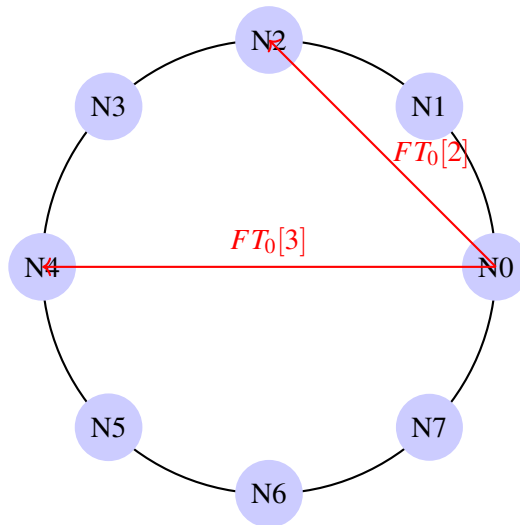


Figure 4.10: Chord Ring with Finger Table Routing

## Naïve Resolution Algorithm

The **Naïve Key Resolution Algorithm** is a simple method used in Distributed Hash Tables (DHTs) to locate the node responsible for a given key. It involves sequential traversal of nodes in the overlay network until the target is found.

### A. Algorithm Steps

Given a key  $k$  and a starting node  $n$ :

1. Check if  $k$  lies between  $n$  and its successor.
2. If yes, the successor is responsible for  $k$ .
3. If not, forward the query to the successor.
4. Repeat until the responsible node is located.

### B. Time Complexity

- Worst-case lookup time:  $\mathcal{O}(n)$  hops.
- Not scalable for large networks.

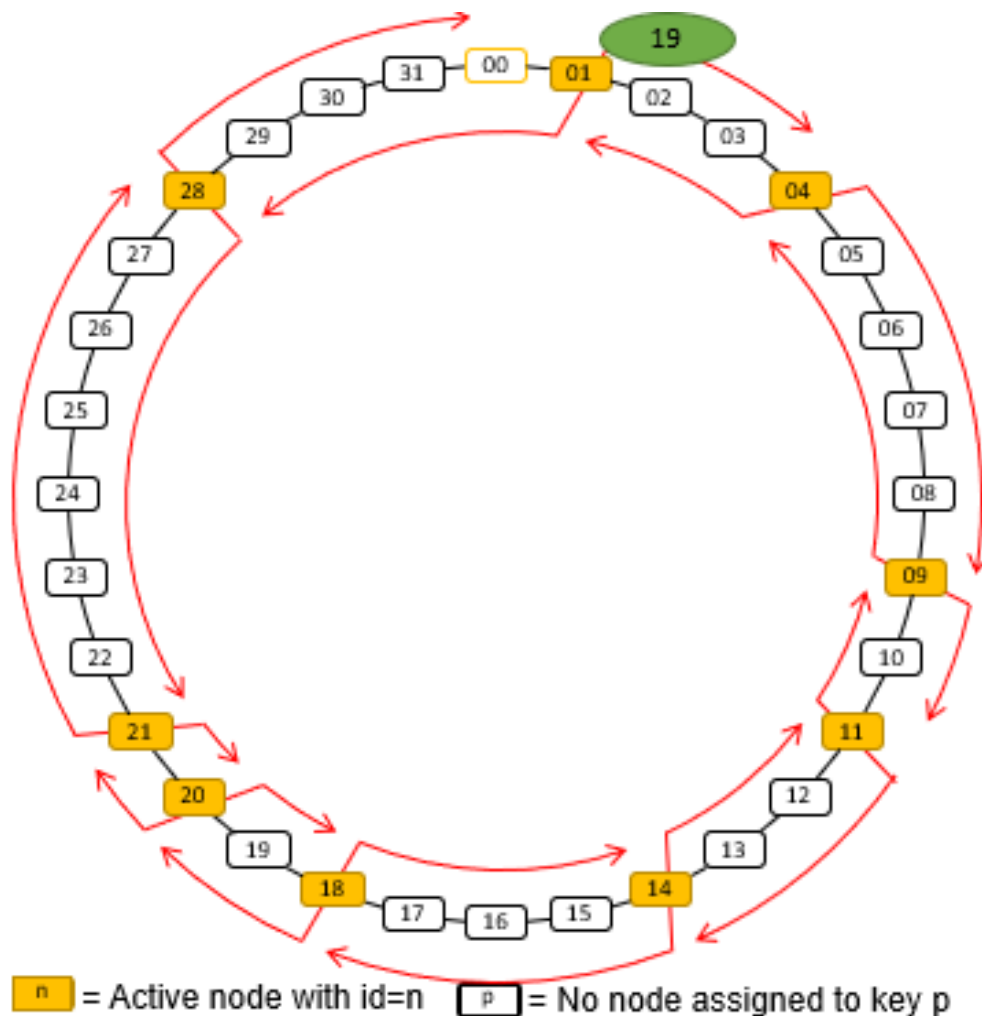


Figure 4.11: Naïve Resolution Algorithm

## Chord Key Resolution Algorithm

Chord improves key resolution by reducing the time complexity to  $\mathcal{O}(\log n)$ , where  $n$  is the number of nodes in the system. All nodes are arranged in a logical ring based on their hashed identifiers.

### Finger Table

Each node  $p$  maintains a **Finger Table**  $FT_p$  with at most  $m$  entries, where  $m$  is the number of bits in the identifier space.

$$FT_p[i] = \text{successor}(p + 2^{i-1}) \bmod 2^m$$

**Note:** The entries in  $FT_p$  increase exponentially with  $i$ .

### Key Resolution Procedure

When node  $p$  receives a request to resolve key  $k$ , it performs the following steps:

- Find the index  $j$  such that:  
 $FT_p[j] \leq k < FT_p[j+1]$
- Forward the request to node  $q = FT_p[j]$ .
- If  $k > FT_p[m]$ , then forward the request to  $FT_p[1]$ .
- If  $k < FT_p[1]$ , then forward the request to  $FT_p[1]$ .

This routing strategy ensures that each lookup moves closer to the target key exponentially, resulting in efficient resolution across the ring.

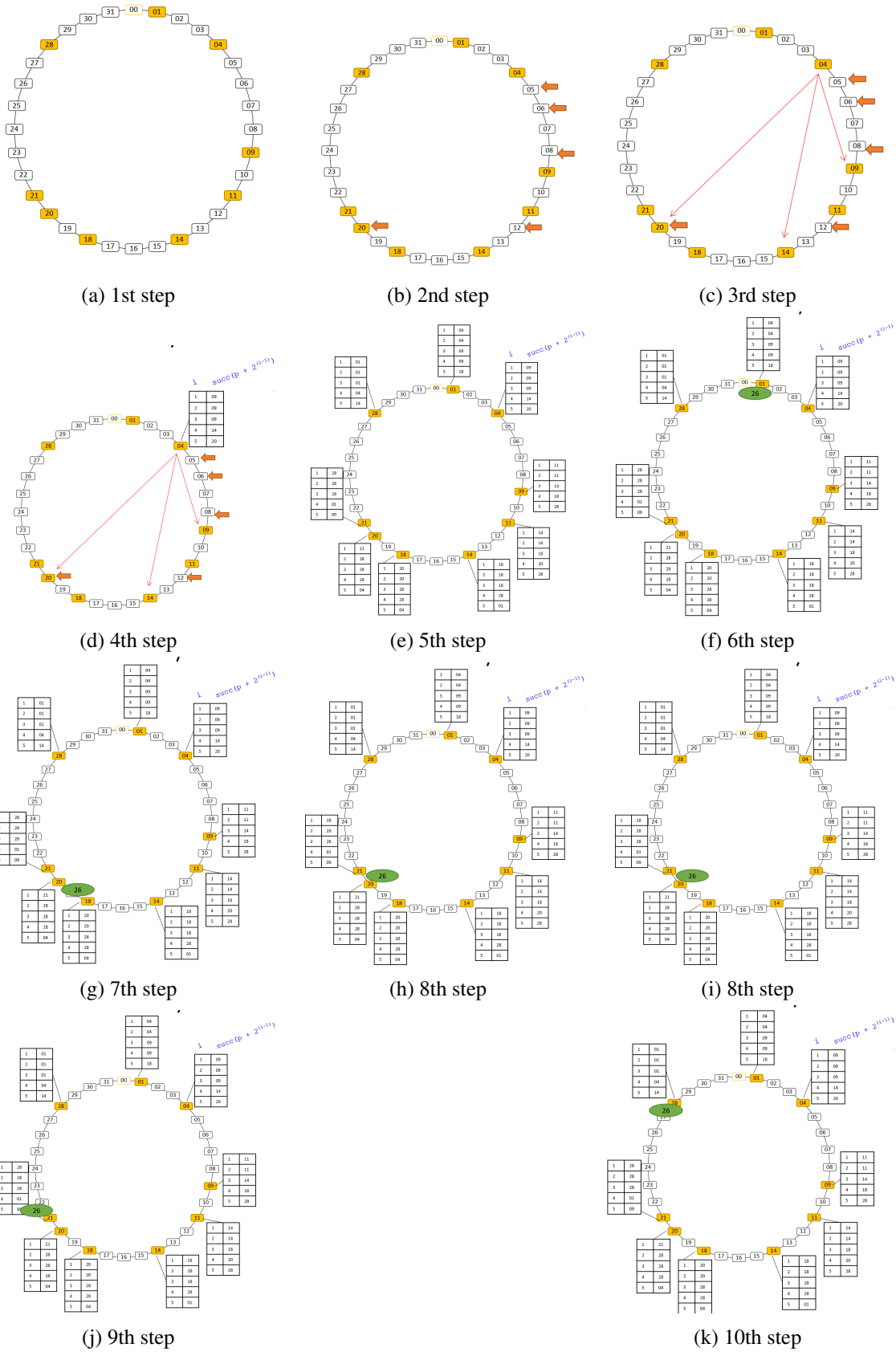


Figure 4.12: Chord Key Resolution Procedure Steps

### 4.2.2 Structured Naming and Naming Graphs

**Structured naming** refers to the use of hierarchical, human-readable names that reflect the organization of resources. *These names are typically composed of segments separated by delimiters (e.g., slashes in file paths or dots in domain names)*. Each segment represents a contextual level, allowing users to navigate through a logical hierarchy.

In distributed systems, *structured names are resolved using naming graphs—directed graphs where nodes represent entities and edges are labeled with name segments*. Leaf nodes correspond to *actual resources (e.g., files)*, while directory nodes serve as intermediaries that point to other nodes. This graph-based model supports efficient traversal and resolution of names, enabling systems to locate resources based on their hierarchical structure.

#### Examples of Structured Naming

**File System Context** In the context of a file system, the path:

```
/home/userid/work/dist-systems/naming.txt
```

is the structured name of the file `naming.txt`. Each segment in the path represents a directory level, forming a hierarchical structure that leads to the target file.

**Web Context** In the context of a website, the URL:

```
https://ci.suez.edu/ggg/15440-f23/
```

is the structured name of the 15440 course webpage. The URL segments reflect the domain, user directory, and course folder, forming a navigable hierarchy within the web server's namespace.

Structured names are organized into **name spaces**, which define the context and rules for interpreting names. A name space can be modeled as a **directed graph**, commonly referred to as a *naming graph*, consisting of two types of nodes:

#### Leaf Nodes

- Represent entities such as files, webpages, or services.
- Typically store:
  - **Addresses** (e.g., IP addresses in DNS),
  - **States** (e.g., contents of files in file systems),
  - **Absolute path names** to other nodes or resources.

#### Directory Nodes

- Refer to other leaf or directory nodes within the graph.
- Each outgoing edge is represented as a pair:

```
(edge label, destination node identifier)
```

- These edges define the traversal paths used during name resolution.

## Unix File System as a Naming Graph

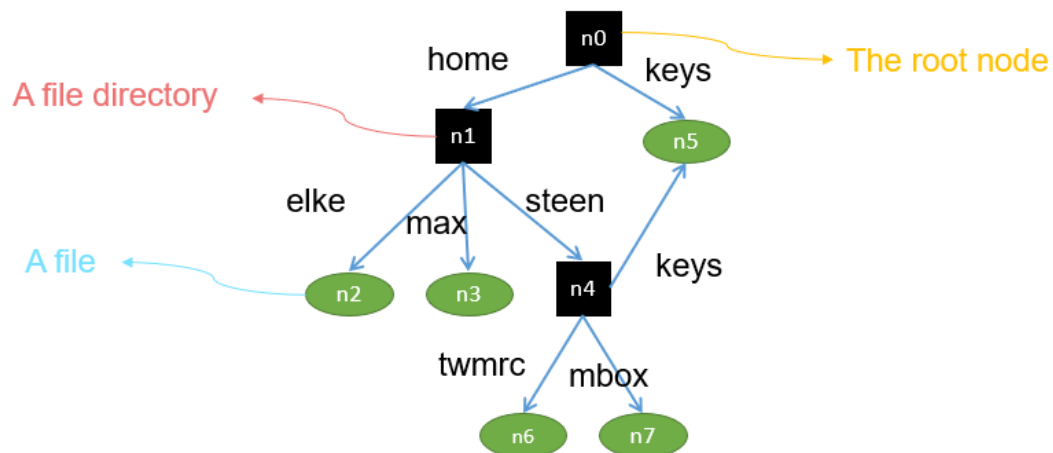


Figure 4.13: Unix Naming Graph Example

The Unix file system exemplifies structured naming through its use of inodes and data blocks. At the physical level, the file system is composed of several key components:

- **Boot Block:** is a special block of data and instructions that are automatically loaded into main memory when the system is booted, It is used to load the OS into main memory

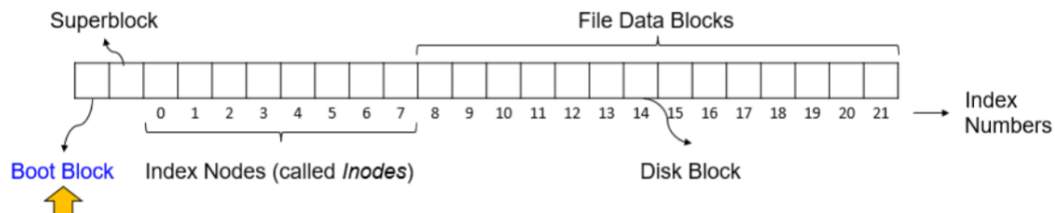


Figure 4.14: Boot Block

- **Superblock:** contains information on the entire file system (e.g., its size, which blocks on disk are not yet allocated, which Inodes are not yet used, etc.,)

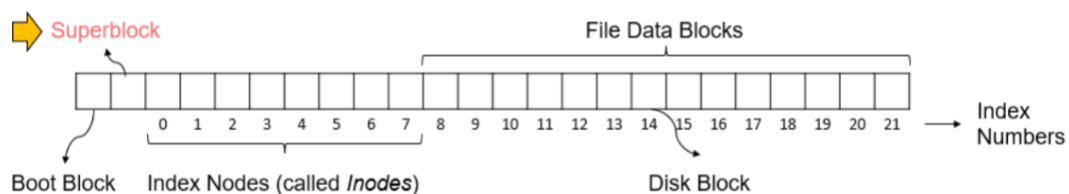


Figure 4.15: Super Block

- **Inodes:** The Inodes are referred to by index numbers, starting at zero, which is reserved for the Inode representing the root directory. Each Inode contains information on where the data of its associated file can be found on disk. Besides, an Inode contains information on its owner, time of creation and last modification, protection, etc.,



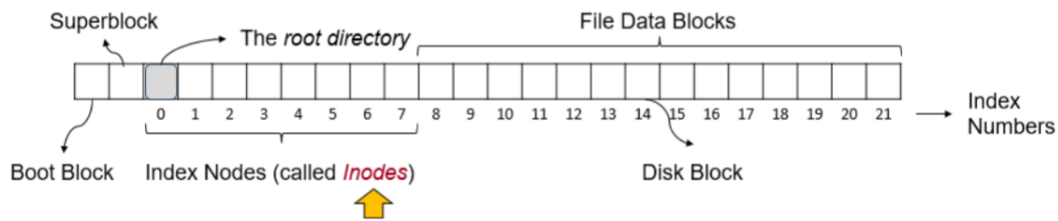


Figure 4.16: Inode

- **Data Blocks:** Hold the actual contents of files.

Name resolution in Unix involves traversing the naming graph by following directory entries. For example, resolving the path `/home/steen/mbox` requires reading the root inode, locating the “home” directory, then “steen,” and finally the “mbox” file. Each step involves accessing the corresponding inode and interpreting its directory entries to continue traversal.

## Linking Mechanisms: Hard and Symbolic Links

Unix supports two types of links that allow multiple names to refer to the same resource: hard links and symbolic links.

**Hard links** create multiple directory entries that point to the same inode. This means that different names can refer to the same file, and changes made through one name are reflected in all others. However, hard links are restricted to the same file system and cannot form cycles.

**Symbolic links**, on the other hand, store the path to another file as their content. When accessed, the system reads the symbolic link and resolves the referenced path. Symbolic links can span file systems and support more flexible referencing, but they introduce an additional layer of indirection during resolution.

## Mounting and Distributed Name Spaces

**Mounting** is a technique used to integrate multiple name spaces into a single unified hierarchy. In distributed systems, mounting allows directories from remote machines to appear as part of the local file system. This is commonly used in networked file systems like NFS.

For example, the path `/remote/vu/home/steen/mbox` may refer to a file stored on a remote server, but it is accessed as if it were part of the local directory structure. Mounting provides transparency and location independence, enabling users to interact with distributed resources seamlessly.

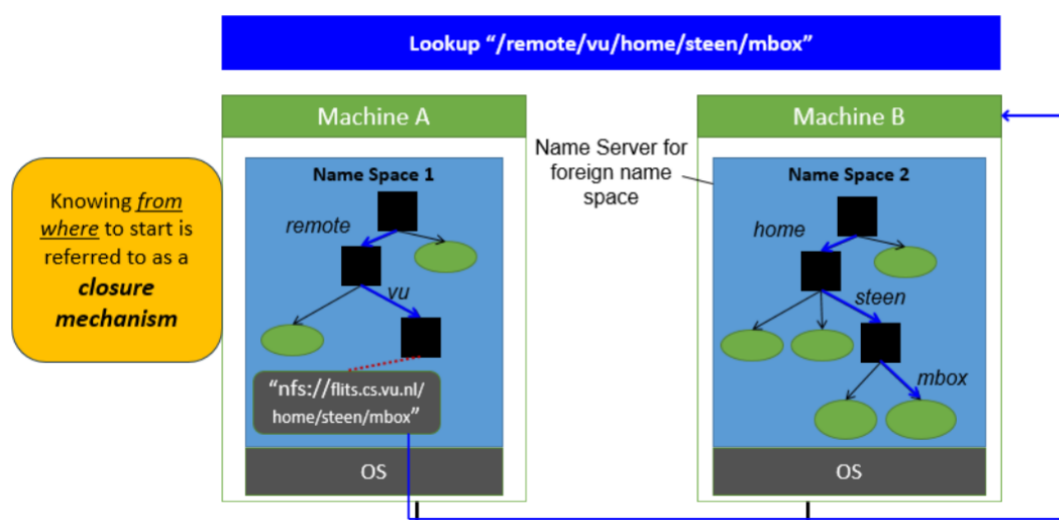


Figure 4.17: Example of Mounting Name space in NFS

## Distributing the Naming Graph

In large-scale systems, the naming graph itself may be distributed across multiple machines. A common approach is to use a master-worker model, where the master node stores the root and key directories, while worker nodes manage subdirectories and files. This distribution improves scalability and fault tolerance, but it also introduces challenges in consistency and coordination.

Alternative models, such as peer-to-peer distribution, offer decentralized control but require more complex resolution protocols. Understanding these models is essential for designing robust and efficient distributed naming systems.

### 4.2.3 Attribute-Based Naming and Directory Services

In many cases, it is more convenient to name and look up entities based on their **attributes**, rather than fixed hierarchical names. This approach is similar to traditional directory services, such as the Yellow Pages, where users search for entries by specifying descriptive properties (e.g., profession, location).

## Challenges of Attribute-Based Lookup

While attribute-based naming offers flexibility, the lookup operations can be computationally expensive. Matching requested attribute values against actual values may require inspecting all entities in the system, leading to high latency and resource consumption.

## Design Idea

To mitigate these costs, a practical approach is to:

- Implement the basic directory service as a **database**, where entities are indexed by their attributes.
- Combine this with **traditional structured naming**, allowing hierarchical resolution when attributes alone are insufficient or ambiguous.

This hybrid model supports both semantic queries and efficient traversal, balancing flexibility with performance.

This approach is exemplified by systems like LDAP (Lightweight Directory Access Protocol), which organize records into a Directory Information Tree (DIT). Each record consists of attribute-value pairs and is uniquely identified by a Distinguished Name. Queries are expressed using logical

filters, such as `&(C=NL)(O=Vrije Universiteit)(CN=Main server)`, which match records based on specified attributes.

Attribute-based naming supports flexible and semantic resource discovery, making it ideal for environments where resources are dynamic or loosely structured.

To consolidate the concepts, here’s a comparative overview of naming types:

| Naming Type            | Mechanism                               | Example                   |
|------------------------|-----------------------------------------|---------------------------|
| Flat Naming            | Unique identifiers, resolved via DHT    | MAC address, UUID         |
| Structured Naming      | Hierarchical names, resolved via graphs | File paths, URLs          |
| Attribute-Based Naming | Descriptive queries, attribute matching | LDAP, service directories |

Each naming approach serves different needs in distributed systems, balancing simplicity, flexibility, and scalability.

## 4.3 Questions

1. What is the difference between a name and an address in distributed systems?
2. Why is name resolution necessary, and how is it typically performed?
3. What are the advantages and disadvantages of flat naming?
4. How does hierarchical naming improve scalability and manageability?
5. What role do symbolic names play in user experience and system design?
6. Explain the concept of mounting in hierarchical naming systems.
7. How does the separation of naming and addressing support mobility?
8. Why might flat naming be insufficient in large-scale distributed systems, even though it guarantees uniqueness?
9. How does symbolic naming improve usability for humans, and what challenges does it introduce for machines?
10. In what ways does hierarchical naming support scalability and fault isolation in distributed systems?
11. What are the implications of separating naming from addressing in terms of system flexibility and mobility?
12. How does name resolution differ when using flat names versus hierarchical names?

## II. Scenario-Based Questions

1. **Scenario 1:** You're designing a distributed storage system for a multinational company. Each office has its own servers, and employees need to access files using intuitive names.  
*Question:* Would you use flat, symbolic, or hierarchical naming? Justify your choice based on scalability, usability, and resolution efficiency.
2. **Scenario 2:** A mobile device frequently changes networks and IP addresses. You need to ensure that services can still locate and communicate with it reliably.  
*Question:* How would you design the naming and resolution system to support mobility without breaking existing references?
3. **Scenario 3:** You're integrating two legacy systems: one uses symbolic names, the other uses flat identifiers.  
*Question:* How would you reconcile these naming schemes to create a unified resolution strategy?
4. **Scenario 4:** A user reports that symbolic links in their distributed file system are broken after a server migration.  
*Question:* What might have caused this issue, and how could hierarchical naming or mounting help prevent it?

## 4.4 Answers:

1. **Name vs. Address:** A name identifies an entity independent of its location, while an address specifies where the entity is located. Names are used for abstraction; addresses are used for routing.
2. **Name Resolution:** Name resolution maps a name to its corresponding address or object. It is typically performed via lookup tables, DNS-like systems, or distributed hash tables.
3. **Flat Naming Pros/Cons:** *Advantages:* Simplicity, uniqueness, and fast lookup via hashing. *Disadvantages:* Poor scalability, no structure for delegation, and hard to manage or interpret.
4. **Hierarchical Naming Benefits:** It supports delegation, scalability, and fault isolation. Names are easier to manage and interpret due to their structured format.
5. **Symbolic Names:** These improve usability by allowing human-readable identifiers. However, they require resolution mechanisms and may introduce ambiguity or collisions.
6. **Mounting:** Mounting allows integration of different name spaces by attaching one name space into another at a specific point, enabling modular and scalable system design.
7. **Naming vs. Addressing for Mobility:** Separating naming from addressing allows entities to retain their identity even when their location changes, supporting mobile and dynamic systems.
8. **Flat Naming Limitations:** Flat naming lacks structure, making it difficult to delegate authority or scale across administrative domains. It also complicates resolution and management.
9. **Symbolic Naming Trade-offs:** Symbolic names are intuitive for users but require resolution infrastructure. They may be ambiguous, non-unique, and slower to resolve than numeric identifiers.
10. **Hierarchical Naming Advantages:** Hierarchical naming enables scalable resolution, fault isolation, and administrative delegation. It mirrors organizational structures and supports modularity.
11. **Naming vs. Addressing Implications:** Decoupling naming from addressing allows entities to move or change location without losing identity. It enhances flexibility, mobility, and abstraction.
12. **Name Resolution Differences:** Flat names often use hashing or global lookup tables, while hierarchical names use recursive resolution through delegated name servers or mount points.

## II. Scenario-Based Questions Answers

1. **Scenario 1:** Hierarchical naming is ideal. It supports scalability across offices, intuitive access via structured paths (e.g., /cairo/hr/reports), and efficient resolution through delegation.
2. **Scenario 2:** Use symbolic naming with dynamic binding. A name server can map the symbolic name to the current address, updated via heartbeat or registration protocols. This preserves identity across network changes.
3. **Scenario 3:** Introduce a name resolution layer that maps symbolic names to flat identifiers. Use a translation service or aliasing mechanism to unify access while preserving legacy semantics.
4. **Scenario 4:** The links likely referenced absolute paths tied to the old server. Hierarchical naming with mounting allows abstraction over physical locations, so links remain valid if mount points are preserved.

## 4.5 Multiple Choice Quiz

- Q1.** Why might flat naming be insufficient in large-scale distributed systems?
- (A) It is too complex to implement.
  - (B) It lacks scalability and structure for delegation.
  - (C) It is incompatible with hierarchical naming.
  - (D) It requires symbolic links for resolution.
- Q2.** What is the primary advantage of separating naming from addressing in distributed systems?
- (A) It eliminates the need for name resolution.
  - (B) It allows entities to retain their identity despite location changes.
  - (C) It simplifies hierarchical naming.
  - (D) It avoids the use of symbolic names.
- Q3.** How does hierarchical naming support scalability in distributed systems?
- (A) By using flat identifiers for all resources.
  - (B) By delegating name resolution to different administrative domains.
  - (C) By eliminating the need for symbolic names.
  - (D) By avoiding the use of mounting.
- Q4.** What is a key challenge introduced by symbolic naming?
- (A) It is incompatible with hierarchical naming.
  - (B) It requires resolution infrastructure to map names to addresses.
  - (C) It eliminates the need for flat naming.
  - (D) It cannot be used in distributed systems.
- Q5.** What is the purpose of mounting in hierarchical naming systems?
- (A) To create flat identifiers for resources.
  - (B) To integrate multiple namespaces into a unified hierarchy.
  - (C) To resolve symbolic names to addresses.
  - (D) To eliminate the need for name resolution.
- Q6.** Why is name resolution more complex in flat naming systems compared to hierarchical naming systems?
- (A) Flat naming systems require recursive resolution.
  - (B) Flat naming systems lack structure for efficient lookup.
  - (C) Flat naming systems use symbolic names.
  - (D) Flat naming systems rely on mounting.
- Q7.** What is the main benefit of using a naming graph in distributed systems?
- (A) It eliminates the need for symbolic names.
  - (B) It provides a structured way to organize and locate resources.
  - (C) It simplifies memory allocation.
  - (D) It avoids the need for hierarchical traversal.
- Q8.** How does symbolic naming improve usability for users?
- (A) By using unique identifiers for resources.
  - (B) By providing human-readable names that are easier to understand.
  - (C) By eliminating the need for name resolution.
  - (D) By avoiding hierarchical paths.
- Q9.** What is the role of a directory service in distributed systems?
- (A) To store file contents.
  - (B) To manage memory allocation.
  - (C) To resolve names to addresses.
  - (D) To synchronize processes.
- Q10.** What is the main limitation of flat naming systems?
- (A) They are difficult to implement.
  - (B) They lack scalability and organization.

- (C) They are incompatible with distributed systems.
  - (D) They require complex algorithms for resolution.
- Q11.** What is the advantage of combining symbolic naming with hierarchical naming?
- (A) It eliminates the need for name resolution.
  - (B) It balances user-friendliness with scalability.
  - (C) It simplifies memory allocation.
  - (D) It avoids the need for hierarchical traversal.
- Q12.** What is the purpose of edge labels in a naming graph?
- (A) To store file contents.
  - (B) To define traversal paths between nodes.
  - (C) To manage memory allocation.
  - (D) To synchronize processes.
- Q13.** What is an example of symbolic naming in a distributed system?
- (A) `/home/user/documents/file.txt`
  - (B) MAC address: `00:1A:2B:3C:4D:5E`
  - (C) IP address: `192.168.1.1`
  - (D) inode `#12345`
- Q14.** What is the difference between symbolic and address-based naming?
- (A) Symbolic naming uses human-readable names, while address-based naming uses unique identifiers.
  - (B) Symbolic naming is faster than address-based naming.
  - (C) Address-based naming is hierarchical, while symbolic naming is flat.
  - (D) Address-based naming does not require name resolution.
- Q15.** What is the main challenge of name resolution in distributed systems?
- (A) It is difficult to implement.
  - (B) It requires efficient mapping of names to addresses.
  - (C) It is incompatible with hierarchical naming.
  - (D) It cannot handle symbolic names.



## 5. Synchronization in Distributed Systems

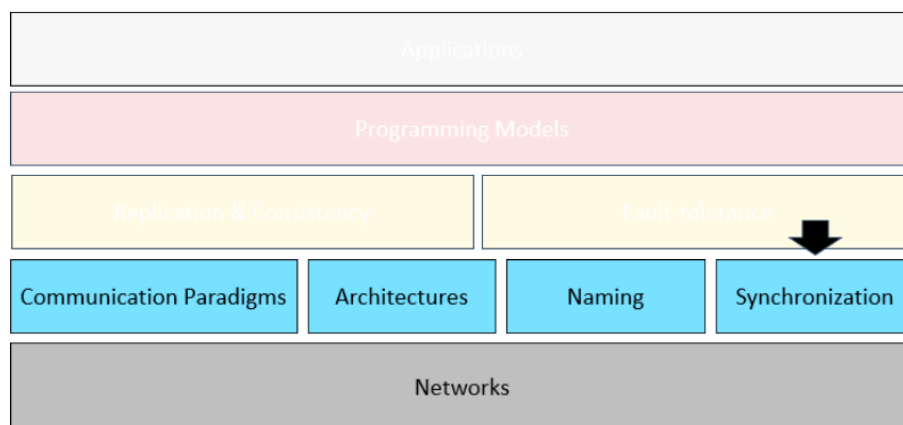


Figure 5.1: Synchronization

In distributed systems, **synchronization** refers to the coordination of time and events across multiple independent processes or machines. Unlike centralized systems, where a single clock governs execution, **distributed environments lack a global time reference, making it challenging to determine the order of events, maintain consistency, and ensure fairness.** Synchronization mechanisms—ranging from physical clock alignment to logical timestamping—are essential for enabling coherent interactions among nodes, whether it's updating shared data, managing concurrent access, or sequencing operations. This chapter introduces the foundational concepts of synchronization, explores key algorithms, and equips learners with the tools to reason about time and coordination in distributed computing.

### 5.1 Introduction: The Problem of Time in Distributed Systems

**In a centralized system, time is singular and consistent. But in distributed systems—where multiple nodes operate independently—time becomes fragmented.** Each machine maintains its own clock, and these clocks inevitably drift due to hardware imperfections and network latency.

#### **Analogy: Orchestra Without a Conductor**

Imagine an orchestra where each musician plays based on their own watch. Without a conductor or synchronized timing, harmony collapses. Distributed systems face a **similar challenge: coordinating**



actions across machines without a global clock.

So now to begin studying synchronization we will take an overview for the following subjects:

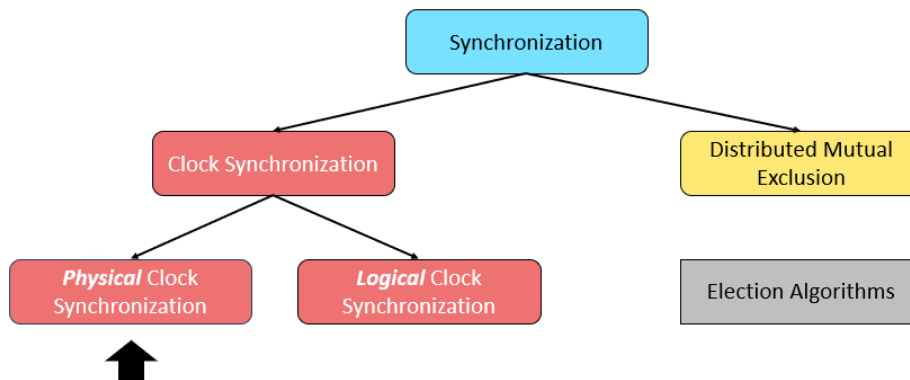


Figure 5.2: Physical clock Synchronization Overview

## 5.2 Physical Clock Synchronization

**Introduction** In distributed systems, each machine maintains its own local clock, which may drift over time due to hardware imperfections and environmental factors. To coordinate actions across machines, we need a consistent notion of time.

This is where **Universal Coordinated Time (UTC)** comes in—a globally agreed-upon time standard maintained by atomic clocks and disseminated via radio signals and satellite systems. However, computers do not inherently know UTC. Instead, they rely on their internal hardware clocks, which must be periodically synchronized with external time sources to stay accurate.

Tracking time on a computer involves reading from a quartz oscillator-based clock, which can drift by several milliseconds per day. In distributed systems, even small discrepancies can lead to inconsistent event ordering, stale data, or race conditions. To address this, various algorithms have been developed to synchronize clocks across machines, either by estimating network delays or by averaging time across nodes.

### Clock Drift and Skew

- **Drift:** The rate at which a clock deviates from real time.
- **Skew:** The difference between two clocks at a given moment.

### 5.2.1 Cristian's Algorithm

Cristian's algorithm is one of the earliest and simplest methods for synchronizing a client's clock with a time server. It assumes that the server's clock is accurate and that network delay is symmetric.

#### Steps:

1. The client sends a request to the time server at local time  $T_1$ .
2. The server receives the request and replies with its current time  $T_s$ .
3. The client receives the reply at local time  $T_2$ .
4. The client estimates the round-trip delay and adjusts its clock as follows:

$$T_{\text{client}} = T_s + \frac{T_2 - T_1}{2}$$

#### Limitations:

- Assumes symmetric network delay.
- Vulnerable to variable latency and server load.

### 5.2.2 Berkeley Algorithm

The Berkeley algorithm takes a decentralized approach to synchronization. Instead of relying on a single accurate server, it computes an average time across participating nodes.

**Steps:**

1. A master node polls all other nodes for their local time.
2. Each node replies with its current clock value.
3. The master calculates the average time, excluding outliers.
4. The master sends each node an adjustment value (not the new time).
5. Each node applies the adjustment to its local clock.

**Features:**

- Tolerates clock drift across nodes.
- More robust in environments without a trusted time source.

### 5.2.3 Network Time Protocol (NTP)

NTP is the most widely used protocol for synchronizing clocks over packet-switched networks like the Internet. It uses a hierarchical system of time servers organized into strata.

**Key Concepts:**

- **Stratum 0:** High-precision sources (e.g., atomic clocks, GPS).
- **Stratum 1:** Directly connected servers to stratum 0.
- **Stratum 2+:** Servers that synchronize with higher strata.

**Synchronization Process:**

1. A client exchanges four timestamps with a server:  $T_1, T_2, T_3, T_4$ .
2. It computes the offset and delay using:

$$\text{Offset} = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

3. The client adjusts its clock based on the calculated offset.

**Advantages:**

- Highly accurate and scalable.
- Accounts for asymmetric delays.
- Supports authentication and fault tolerance.

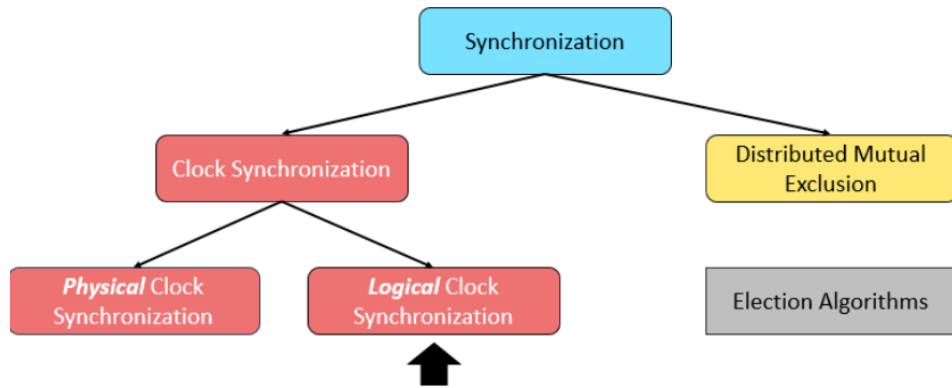


Figure 5.3: Logical Clock Synchronization

## 5.3 Logical Clock Synchronization

**Introduction** In distributed systems, physical clocks are prone to drift and cannot be perfectly synchronized due to unpredictable network delays and hardware limitations. To address this, systems use **logical clocks**, which provide a way to order events based on causality rather than actual time. Logical clock synchronization ensures that causally related events are recorded in a consistent sequence across processes, even when their physical timestamps differ. This is essential for maintaining consistency, coordinating actions, and detecting causality in environments where global time is unreliable or unavailable.

### 5.3.1 Lamport Timestamps

Lamport timestamps are a simple mechanism for ordering events in distributed systems. They do not capture causality precisely but ensure a consistent total ordering of events.

**Mechanism** Each process maintains a logical clock  $C$ , initialized to zero.

- **Local Event:** Before executing any local event, increment the clock:

$$C \leftarrow C + 1$$

- **Send Event:** When sending a message, include the current timestamp  $C$ .
- **Receive Event:** Upon receiving a message with timestamp  $T_m$ , update the clock:

$$C \leftarrow \max(C, T_m) + 1$$

**Example** If process  $P_1$  sends a message with timestamp 2 to process  $P_2$ , and  $P_2$ 's clock is at 0, then upon receiving the message:

$$C_2 \leftarrow \max(0, 2) + 1 = 3$$

**Limitation** Lamport timestamps cannot detect concurrent events. Two events may have different timestamps, but that does not imply a causal relationship unless one directly influenced the other.

### 5.3.2 Vector Clocks

Vector clocks extend Lamport timestamps by capturing the causal relationships between events more precisely.

**Structure** Each process  $P_i$  maintains a vector  $V_i = [v_1, v_2, \dots, v_N]$ , where  $v_j$  is the latest known logical time of process  $P_j$  as seen by  $P_i$ .

**Mechanism**

- **Local Event:** Increment the local entry:

$$V_i[i] \leftarrow V_i[i] + 1$$

- **Send Event:** Include the current vector  $V_i$  with the message.
- **Receive Event:** Upon receiving vector  $V_m$ , update:

$$V_i[k] \leftarrow \max(V_i[k], V_m[k]) \quad \text{for all } k$$

Then increment the local entry:

$$V_i[i] \leftarrow V_i[i] + 1$$

**Causal Comparison** Given two vector timestamps  $V_a$  and  $V_b$ :

- $V_a < V_b$  if  $V_a[i] \leq V_b[i]$  for all  $i$ , and  $V_a[j] < V_b[j]$  for at least one  $j$ .
- Events are **concurrent** if neither  $V_a < V_b$  nor  $V_b < V_a$ .

**Advantages Over Lamport Timestamps**

- Captures true causality between events.
- Detects concurrent events.
- Enables more precise conflict resolution in distributed systems.

**Trade-offs**

- Requires more memory: vector size grows with the number of processes.
- More complex to implement and compare.

**Applications**

- Conflict detection in distributed databases (e.g., Dynamo, Cassandra).
- Causal message delivery in distributed messaging systems.
- Version control in collaborative editing tools.

## 5.4 Mutual Exclusion in Distributed Systems

In distributed systems, multiple processes often need to access shared resources such as files, databases, or critical sections of code. To prevent race conditions, inconsistencies, and data corruption, it is essential to enforce **mutual exclusion**—ensuring that only one process accesses a shared resource at a time.

Unlike centralized systems, distributed environments lack shared memory and a global clock, making mutual exclusion significantly more complex. Processes must coordinate over unreliable networks, tolerate message delays, and handle failures gracefully. Several algorithms have been developed to address these challenges, each with its own trade-offs in terms of performance, fault tolerance, and message complexity.

### 5.4.1 Centralized Algorithm

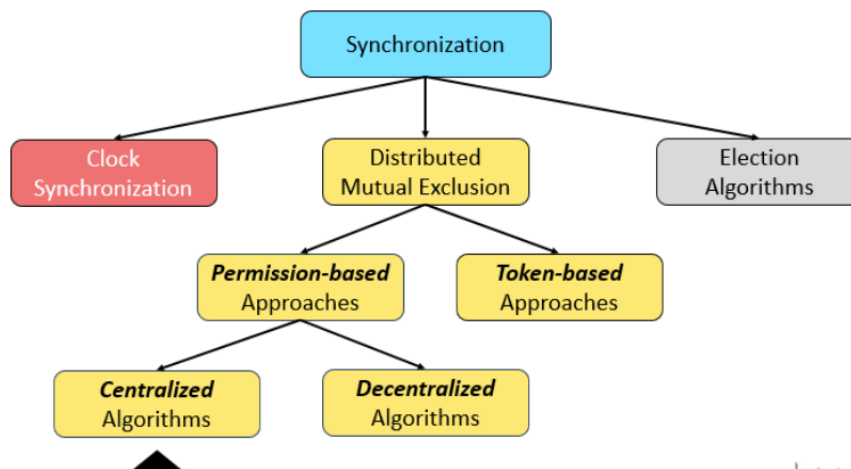


Figure 5.4: Mutual Exclusion Synchronization (Centralized Algorithms)

The centralized approach resembles a client-server model. A designated coordinator node acts as the gatekeeper for the critical section.

- **Mechanism:** A process sends a request to the coordinator. If no other process is in the critical section, the coordinator grants access. Once the process finishes, it sends a release message to the coordinator.

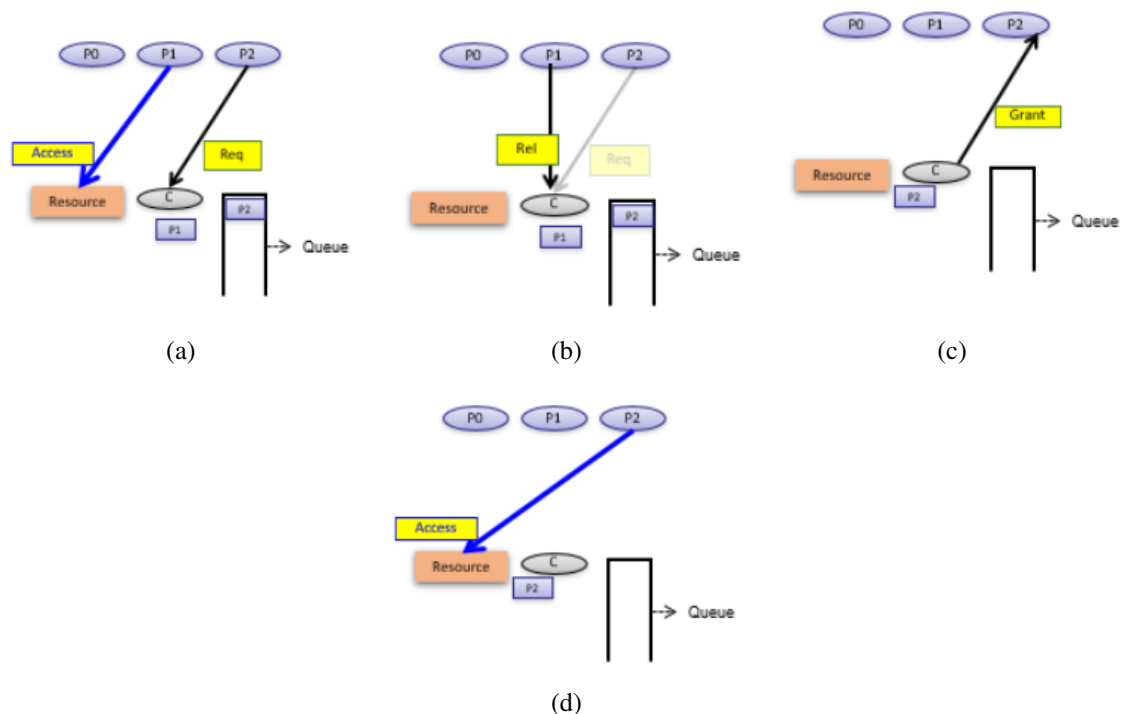


Figure 5.5: Centerlized Algorithm Mechanism

- **Advantages:**
  - Simple to implement.
  - Requires only 3 messages per entry (request, grant, release).

- **Disadvantages:**
  - Single point of failure: if the coordinator crashes, the system halts.
  - Bottleneck under high contention.

### 5.4.2 Decentralized Approach

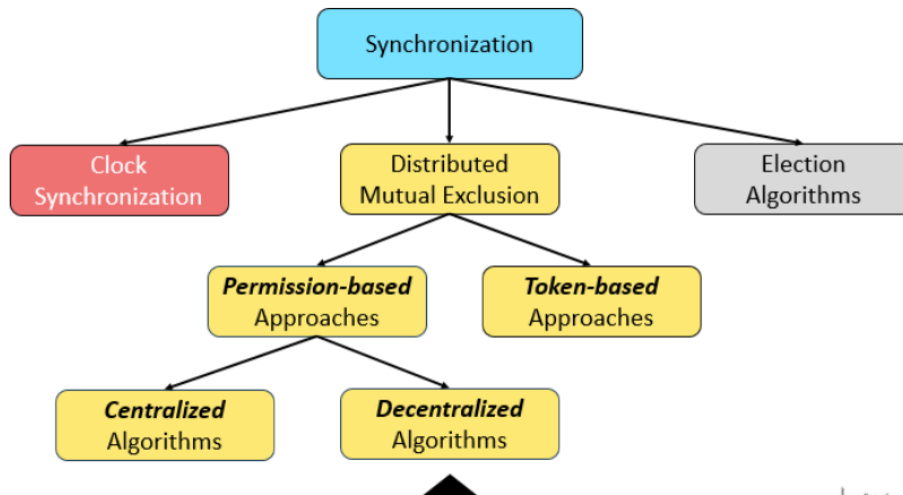


Figure 5.6: Mutual Exclusion Synchronization (Decentralized Algorithms)

The decentralized approach to mutual exclusion distributes the decision-making process across all participating nodes, avoiding reliance on a single coordinator. Each process plays an equal role in granting access to the critical section, typically through a voting mechanism.

- **Mechanism:**
  - When a process wants to enter the critical section, it sends a request to a predefined set of other processes (often a majority or quorum).
  - Each recipient checks whether it has already granted permission to another process. If not, it replies with a grant.
  - The requesting process enters the critical section once it receives enough grants (e.g., from a majority).
  - After exiting, it sends release messages to the same set of processes.
- **Advantages:**
  - No single point of failure—robust against coordinator crashes.
  - Load is distributed across multiple nodes.
  - Can be tuned for fault tolerance using quorum-based techniques.
- **Disadvantages:**
  - Requires more messages than centralized methods.
  - Complexity increases with dynamic membership or failures.
  - Risk of deadlock or starvation if quorum responses are delayed or lost.

#### 5.4.2.1 Quorum-Based Mutual Exclusion

A common implementation of the decentralized approach uses **quorums**, where each process contacts a subset of nodes such that any two quorums intersect. This ensures that no two processes can simultaneously enter the critical section.

- **Quorum Properties:**
  - Each process has a unique quorum set.
  - Any two quorum sets share at least one common node.

- Mutual exclusion is guaranteed by requiring permission from all nodes in the quorum.
- **Example:**
  - In a system with  $N$  nodes, each process may contact  $\sqrt{N}$  nodes.
  - This reduces message complexity compared to contacting all nodes.

#### 5.4.2.2 Design Considerations

When implementing a decentralized mutual exclusion algorithm, designers must consider:

- **Quorum selection:** How to choose quorum sets to balance load and ensure intersection.
- **Failure handling:** How to recover when quorum members crash or become unreachable.
- **Fairness and ordering:** Whether requests are granted in a fair and consistent order.
- **Scalability:** How the system performs as the number of nodes increases.

### 5.4.3 Token Ring Algorithm

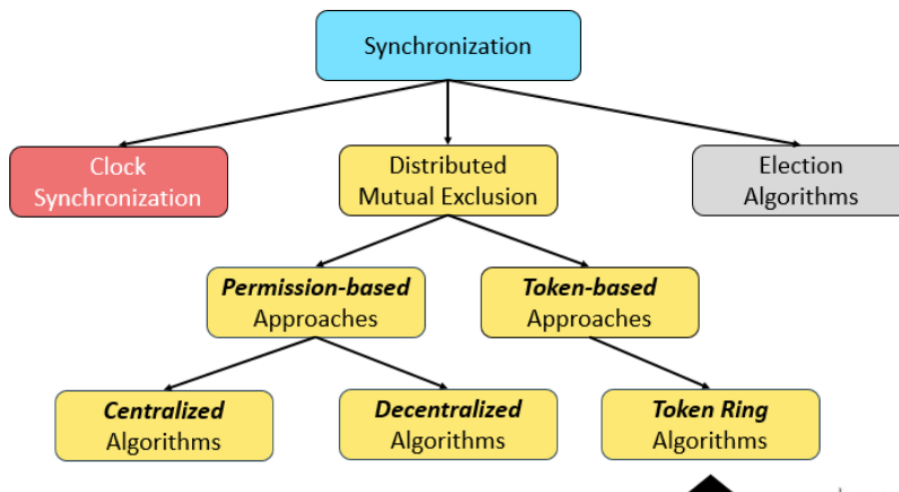


Figure 5.7: Mutual Exclusion Synchronization (Token-Based Approaches)

In this approach, processes are arranged in a logical ring. A unique token circulates around the ring, and possession of the token grants access to the critical section.

- **Mechanism:** The token is passed from one process to the next in a fixed order. A process can enter the critical section only when it holds the token. After exiting, it passes the token to its neighbor.
- **Advantages:**
  - Requires only 1 message per entry (token pass).
  - No need for timestamp comparison or voting.
- **Disadvantages:**
  - Token loss requires recovery mechanisms.
  - Ring reconfiguration is needed if processes join or leave.

#### Design Considerations

When choosing a mutual exclusion algorithm for a distributed system, several factors must be considered:

- **Fault tolerance:** Can the system recover from node or network failures?
- **Message complexity:** How many messages are required per critical section entry?
- **Fairness:** Are requests granted in the order they arrive?
- **Scalability:** Does performance degrade as the number of nodes increases?

## 5.5 Election Algorithms in Distributed Systems

In distributed systems, certain tasks—such as coordination, resource allocation, or mutual exclusion—may require a designated leader or coordinator. However, since nodes can fail or recover unpredictably, the system must be able to dynamically elect a new leader when needed. This is the role of an **election algorithm**: a protocol that allows nodes to agree on a single process to act as the coordinator.

Election algorithms are essential for fault tolerance and decentralized control. They ensure that even in the presence of failures, the system can continue operating by selecting a new leader without manual intervention.

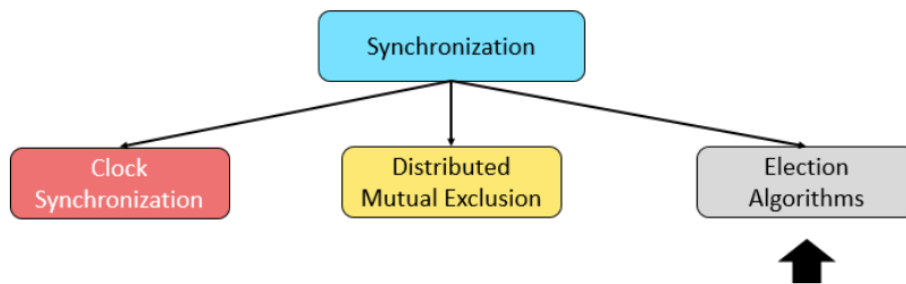


Figure 5.8: Election Algorithms

### Basic Requirements

A correct election algorithm must satisfy the following properties:

- **Uniqueness:** Only one coordinator is elected.
- **Agreement:** All nodes agree on the same coordinator.
- **Termination:** The election process eventually completes.
- **Fault tolerance:** The algorithm can handle node failures during the election.

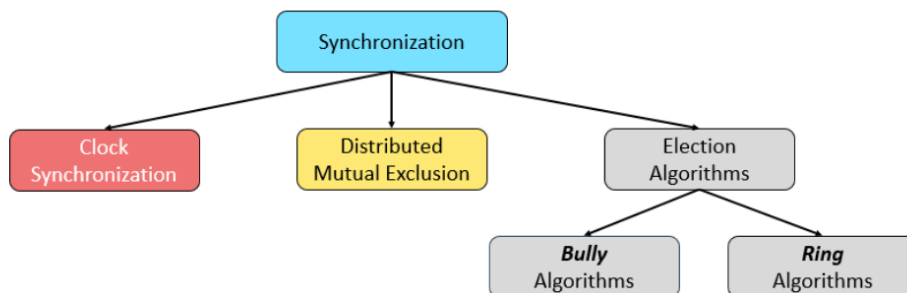


Figure 5.9: Election Algorithms Types

### 5.5.1 Bully Algorithm

The Bully Algorithm is a classic election protocol that assumes each process has a unique ID and that higher IDs have higher priority.

#### Mechanism

1. A process notices that the coordinator has failed.
2. It sends an ELECTION message to all processes with higher IDs.
3. If no one responds, it declares itself the coordinator.



4. If a higher-ID process responds, it takes over the election and repeats the process.
5. Eventually, the process with the highest ID becomes the coordinator and announces itself.

**Advantages**

- Simple and deterministic.
- Guarantees selection of the highest-ID process.

**Disadvantages**

- Generates many messages in large systems.
- Assumes reliable message delivery and knowledge of all process IDs.

### 5.5.2 Ring-Based Election Algorithm

This algorithm arranges processes in a logical ring and passes election messages around the ring.

**Mechanism**

1. A process detects coordinator failure and initiates an election by sending a message containing its ID to its neighbor.
2. Each process forwards the message, replacing the ID if its own is higher.
3. When the message returns to the initiator, the process with the highest ID is elected.
4. A COORDINATOR message is then circulated to inform all nodes.

**Advantages**

- Message complexity is linear:  $O(n)$ .
- No need for global knowledge of all process IDs.

**Disadvantages**

- Slower than the Bully algorithm in small systems.
- Requires ring topology or logical ordering.

**Design Considerations**

When choosing or designing an election algorithm, consider:

- **System size:** Message overhead grows with the number of nodes.
- **Failure model:** Can the algorithm tolerate crash failures or network partitions?
- **Topology:** Is the system organized in a ring, mesh, or arbitrary graph?
- **Performance:** How quickly can a new coordinator be elected?

## 5.6 Questions

1. What does synchronization mean in distributed systems, and why is it necessary?
2. How do mutual exclusion and deadlock prevention relate to synchronization?
3. Why is clock synchronization challenging in distributed systems?
4. Compare centralized and decentralized mutual exclusion algorithms. What are their trade-offs?
5. How do Lamport timestamps differ from vector clocks in terms of causality tracking?
6. What role does message ordering play in maintaining consistency across distributed nodes?
7. What properties must a good synchronization algorithm satisfy (e.g., fairness, fault tolerance)?
8. How does network latency affect the accuracy of clock synchronization algorithms?
9. Why might a system choose logical clocks over physical clocks?

## Scenario-Based Questions

1. **Scenario 1:** A distributed system uses physical clocks for timestamping events. Over time, nodes begin to show inconsistent timestamps due to drift.  
*Question:* How could the Berkeley Algorithm help resolve this issue? What are its limitations?
2. **Scenario 2:** A system maintains vector clocks for three processes. The vector at one node is [2, 3, 1], and another node has [2, 2, 2].  
*Question:* Can we determine which event happened first? Justify your answer using vector clock comparison.
3. **Scenario 3:** A centralized mutual exclusion system loses its coordinator due to a crash.  
*Question:* What are the immediate consequences? How could an election algorithm restore synchronization?
4. **Scenario 4:** A distributed database needs to maintain causal consistency across updates.  
*Question:* Would Lamport timestamps or physical clocks be more appropriate? Explain your reasoning.

## 5.7 Answers

1. **Synchronization** ensures that processes coordinate their actions when accessing shared resources, maintaining consistency and avoiding race conditions.
2. **Mutual exclusion** prevents multiple processes from entering critical sections simultaneously. **Deadlock prevention** ensures that processes do not wait indefinitely for resources, which is crucial for synchronization.
3. Clock synchronization is difficult due to **clock drift**, **network latency**, and the absence of a global clock in distributed systems.
4. **Centralized algorithms** are simple but suffer from a single point of failure. **Decentralized algorithms** are more fault-tolerant but often more complex and message-intensive.
5. **Lamport timestamps** provide a scalar value to order events logically. **Vector clocks** offer richer causality tracking by maintaining a vector of timestamps for each process.
6. Message ordering ensures that events are processed in a consistent sequence, which is essential for maintaining **causal consistency** across distributed nodes.
7. A good synchronization algorithm should ensure **mutual exclusion**, **fairness**, **freedom from deadlock**, and **fault tolerance**.
8. Network latency affects synchronization accuracy by introducing delays in message delivery, which can distort time comparisons and coordination.
9. Logical clocks are preferred when **event ordering** is more important than real-time accuracy, especially in systems that require causal consistency.

### Scenario-Based Questions Answers

1. **Scenario 1:** The Berkeley Algorithm helps by collecting time readings from all nodes, computing an average, and adjusting clocks accordingly. However, it assumes reliable communication and may not handle asymmetric delays well.
2. **Scenario 2:** The vectors [2, 3, 1] and [2, 2, 2] are **incomparable** because neither dominates the other in all dimensions. Therefore, the events are **concurrent**.
3. **Scenario 3:** The system loses its ability to manage mutual exclusion. An election algorithm (e.g., Bully or Ring) can be used to select a new coordinator and restore synchronization.
4. **Scenario 4:** Lamport timestamps are more appropriate because they maintain **causal ordering**, which is essential for consistency in distributed databases. Physical clocks may not reflect causality.

# Fast & Reliable or Efficient DS

|          |                                        |            |
|----------|----------------------------------------|------------|
| <b>6</b> | <b>Replication &amp; Caching</b>       | <b>85</b>  |
| 6.1      | Caching Architecture                   | 86         |
| 6.2      | Ordering of Updates                    | 99         |
| 6.3      | Questions on Replication               | 104        |
| 6.4      | Answers of Replication                 | 105        |
| 6.5      | Multiple Choice Quiz                   | 106        |
| <b>7</b> | <b>Fault Tolerance</b>                 | <b>109</b> |
| 7.1      | What is a Failure?                     | 110        |
| 7.2      | Types of Failures                      | 110        |
| 7.3      | Reliability vs. Availability           | 110        |
| 7.4      | Techniques for Fault Tolerance         | 111        |
| 7.5      | Two-Phase Commit Finite State Machines | 112        |
| 7.6      | Questions:                             | 116        |
| 7.7      | Answers:                               | 117        |



## 6. Replication & Caching

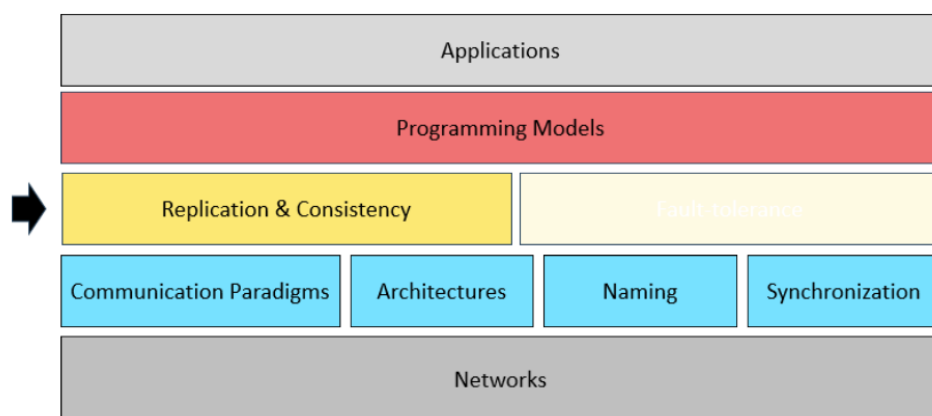


Figure 6.1: Replication & Consistency

### Part I: Caching

Caching is a fundamental technique used to reduce latency and bandwidth consumption in distributed systems. It involves storing frequently accessed data closer to the client, allowing faster access and reduced load on backend servers.

**Latency** refers to the time delay between a request and its response. **Bandwidth** is the amount of data that can be transmitted over a network per unit time. These two metrics are partially intertwined:

- If bandwidth is saturated, congestion occurs and latency increases.
- If bandwidth is underutilized, latency may still remain high.
- Bandwidth can be scaled more easily than latency, which is constrained by physical limits.

To mitigate latency and bandwidth bottlenecks, distributed systems employ two key techniques: **replication** and **caching**. **Part I** focuses on caching.

Caching is a special form of replication where data is stored at the client side. It allows clients to access frequently used data locally, reducing the need to contact remote servers. It is the process of storing copies of data objects near the client to improve access speed and reduce network traffic.

**Examples:**

- Caching webpages in browser memory.
- Caching API responses on mobile devices.
- Caching media content in Content Delivery Networks (CDNs).

## 6.1 Caching Architecture

Caching typically occurs at multiple layers:

[Client]  $\rightleftharpoons$  [Edge Cache/CDN]  $\rightleftharpoons$  [App Server Cache]  $\rightleftharpoons$  [Database]

Each layer can cache data independently, forming a multi-tier caching strategy.

**Three main questions must be answered to complete the caching process:**

### 6.1.1 Fetching Policy: What Data Should Be Cached and When?

Caching systems must decide which data to store and when to retrieve it. This decision is governed by the **fetching policy**, which directly affects performance, consistency, and resource usage.

#### Goals of Fetching

The fetching policy aims to:

- Identify which data objects are worth caching.
- Determine when to fetch data from the original source.
- Balance freshness, latency, and bandwidth efficiency.

#### Two Main Strategies

##### 1. Push-Based Fetching

The server proactively sends updates to the cache whenever the data changes.

###### **Mechanism:**

- The server tracks which clients have cached which objects.
- When an object is updated, the server pushes the new version to all relevant caches.

**Analogy:** A teacher emails updated lecture notes to all students immediately after editing them.

Students always have the latest version without needing to ask.

###### **Advantages:**

- High consistency: clients always have fresh data.
- No need for clients to check for updates.

###### **Disadvantages:**

- Requires the server to maintain state about clients.
- High network overhead, especially with many clients.

**Example:** Dropbox uses push-based replication to synchronize files across devices.

##### 2. Pull-Based Fetching

The client requests data from the server only when needed.

###### **Mechanism:**

- The client checks whether the data is present and valid in the cache.
- If not, it pulls the latest version from the server.

**Analogy:** A student downloads lecture notes only when they are about to study. If they suspect the notes are outdated, they check the website for a newer version.

###### **Advantages:**

- Stateless server: no need to track clients.
- Lower network load.

**Disadvantages:**

- Clients may use stale data.
- Requires logic to detect and handle staleness.

**Example:** Web browsers often use pull-based caching for web pages and assets.

**Comparison Table**

| Strategy   | Consistency | Server Load | Client Complexity | Example                 |
|------------|-------------|-------------|-------------------|-------------------------|
| Push-Based | High        | High        | Low               | Dropbox, real-time apps |
| Pull-Based | Eventual    | Low         | Medium            | Web browsers, AFS       |

Table 6.1: Comparison of Push-Based and Pull-Based Fetching Policies

**Push-Based:** [Server] → (sends updates) → [Client Cache]

**Pull-Based:** [Client Cache] ← (requests data) ← [Server]

**Scenario:** You are designing a caching system for a weather app.

- Would you use push or pull-based fetching?
- How would your choice change if the app is used in areas with poor connectivity?
- **Push-based fetching:** Data is proactively sent to clients. Implies strict consistency.
- **Pull-based fetching:** Data is fetched on demand. Implies eventual consistency.

**Examples:**

- Dropbox uses push-based replication.
- Andrew File System (AFS) uses pull-based fetching.

**6.1.2 Consistency Policy: How are Updates Propagated to Cached Copies?**

Once data is cached, it may become outdated if the original source changes. The **consistency policy** defines how and when updates are propagated to cached copies to ensure correctness and coherence.

**Goals of Consistency**

The consistency policy aims to:

- Ensure cached data reflects the latest version.
- Minimize stale reads and data anomalies.
- Balance freshness with performance and scalability.

**Consistency Models****A. Strong Consistency**

Every read returns the most recent write.

**Analogy:** Like checking a live scoreboard—every time you look, you see the latest score.

**Mechanism:**

- Updates are immediately propagated to all caches.
- Reads may be delayed until the latest data is available.

**Pros:**

- Guarantees correctness.



- Ideal for financial or critical systems.

**Cons:**

- High latency and overhead.
- Poor scalability in distributed environments.

## B. Eventual Consistency

Cached copies may be temporarily stale, but will eventually reflect the latest version.

**Analogy:** Like syncing your notes across devices—changes may take a few seconds to appear, but they eventually do.

**Mechanism:**

- Updates are propagated asynchronously.
- Conflicts may be resolved using timestamps or version vectors.

**Pros:**

- High availability and performance.
- Scales well across distributed systems.

**Cons:**

- Temporary inconsistencies.
- Requires conflict resolution logic.

## C. Weak Consistency

No guarantees about when updates will be visible.

**Analogy:** Like reading a newspaper—by the time you read it, the news may already be outdated.

**Use Case:** Suitable for non-critical data like logs or analytics.

## Cache Consistency Approaches

Beyond traditional consistency models, distributed systems often employ creative or pragmatic strategies to manage cached data. Below are seven notable approaches, each with unique trade-offs.

### I. Broadcast Invalidation

Broadcast invalidation is a cache consistency strategy where updates to a data item trigger a system-wide notification. All caches that may hold the outdated copy receive an invalidation message, prompting them to discard the stale data.

#### Motivation

In distributed systems, multiple clients may cache the same data. When the original data changes, cached copies must be invalidated to prevent stale reads. Broadcast invalidation provides a simple mechanism to ensure consistency across all nodes.

#### Mechanism

- The server maintains no knowledge of which clients have cached the data.
- Upon a write or update, the server broadcasts an invalidation message to all clients.
- Each client checks whether it holds the affected data and invalidates it if present.

Imagine a teacher announcing to the entire class: *“Please ignore page 42 in your textbook—it contains an error.”* Even if only some students had read that page, everyone hears the message and discards the faulty information.

[Server]  $\xrightarrow{\text{Broadcast}}$  [Client A]       $\xrightarrow{\text{Broadcast}}$  [Client B]       $\xrightarrow{\text{Broadcast}}$  [Client C]

Now as a practical example:

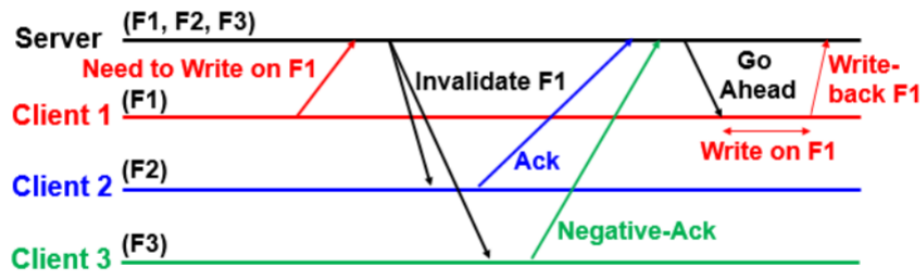


Figure 6.2: Write process in Broadcast Invalidation

If a site is caching the object, it invalidates it; otherwise, it sends a negative Ack message to the server.

If invalidated, next reference to this object at this site will cause a miss

Each client independently checks its cache and invalidates the stale entry if present.

### Advantages

- **Simplicity:** No need to track which clients hold which data.
- **Timeliness:** All clients are notified immediately.
- **Stateless Server:** Reduces server-side complexity.

### Disadvantages

- **Network Overhead:** Every update triggers a broadcast to all clients.
- **Scalability Issues:** Performance degrades as the number of clients grows.
- **Redundant Messages:** Clients may receive invalidations for data they don't cache.

### Use Cases

- Small-scale distributed systems.
- Systems with frequent updates and moderate client count.
- Situations where simplicity is prioritized over scalability.

**Scenario:** You are designing a distributed caching system for a real-time multiplayer game.

- Would broadcast invalidation be suitable?
- How would you mitigate the network overhead?

## II. Callback Consistency

The **callback** approach to cache consistency allows clients to register interest in specific data. When the data is updated, the server sends a notification (callback) to all registered clients, prompting them to invalidate or refresh their cached copy.

### Motivation

Polling the server repeatedly to check for updates is inefficient. Callback mechanisms shift the responsibility to the server, which proactively informs clients when data changes.

### Mechanism

- Clients request data and register a callback with the server.
- The server maintains a list of clients interested in each data item.
- When the data is modified, the server sends a callback to all registered clients.

- Clients invalidate or update their cached copy upon receiving the callback.

Imagine subscribing to a newsletter. Instead of checking the website every day, you receive an email whenever a new article is published. You stay up-to-date without lifting a finger.

```
[Client A] → Register Callback → [Server]
[Client B] → Register Callback → [Server]

[Server]  $\xrightarrow{\text{Update}}$  Notify [Client A, B]
```

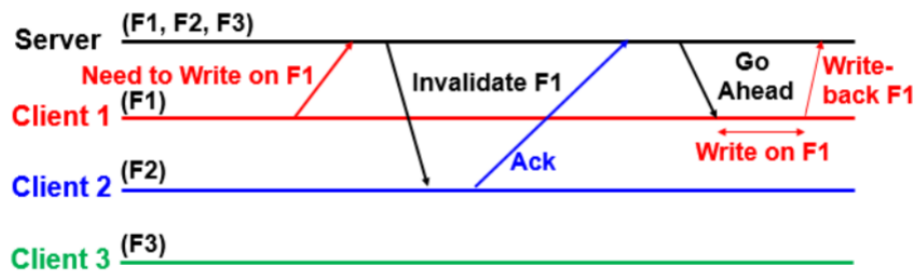


Figure 6.3: Write process in Call Back

### Advantages

- **Efficient:** Reduces unnecessary polling and bandwidth usage.
- **Timely Updates:** Clients are notified immediately upon changes.
- **Selective Consistency:** Only interested clients are notified.

### Disadvantages

- **Stateful Server:** Must track client registrations.
- **Callback Failures:** If a client is unreachable, it may retain stale data.
- **Complexity:** Requires robust error handling and retry logic.

### Use Cases

- Distributed file systems (e.g., Coda).
- Subscription-based APIs.
- Collaborative editing platforms.

**Scenario:** You are designing a caching system for a shared document editor.

- How would you implement callback consistency?
- What strategies would you use to handle unreachable clients?

## III. Check on Use Consistency

The **Check on Use** approach defers consistency validation until the moment a cached item is accessed. Instead of proactively invalidating or updating cached data, the system checks whether the data is still valid only when it is needed.

### Motivation

In many systems, cached data is accessed infrequently or unpredictably. Constantly updating or invalidating such data wastes resources. Check on Use offers a lazy, demand-driven strategy that balances performance and correctness.

## Mechanism

- Clients cache data without receiving updates or invalidations.
- When a client attempts to use cached data, it first checks with the server to confirm validity.
- If the data is stale, the client fetches the latest version before proceeding.

Imagine checking the expiration date on a carton of milk only when you're about to pour a glass. You don't monitor it daily—you verify freshness only when you need it.

[Client Cache]  $\xrightarrow{\text{Use Request}}$  [Server]  $\rightarrow$  Valid?  $\rightarrow$  Use or Refresh

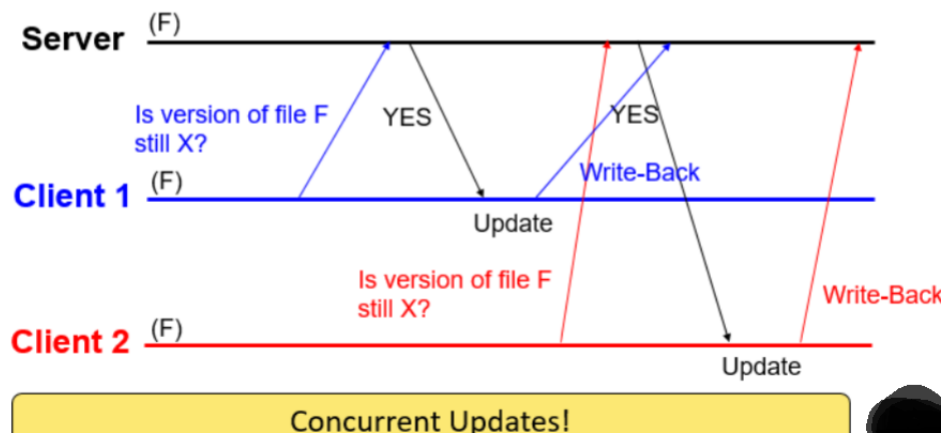


Figure 6.4: Write process in Check in use

## Advantages

- **Efficient:** Avoids unnecessary updates for rarely accessed data.
- **Scalable:** Reduces server-side tracking and network traffic.
- **Simple:** Easy to implement with minimal coordination.

## Disadvantages

- **Latency Spike:** First access may be delayed due to validation.
- **Stale Reads:** If validation fails or is skipped, clients may use outdated data.
- **Inconsistent Behavior:** Different clients may observe different versions of the same data.

## Use Cases

- Web browsers validating cached pages with HTTP headers.
- Distributed file systems like AFS (Andrew File System).
- Systems with read-heavy, low-update workloads.

## Variants

- **Soft Validation:** Use cached data optimistically, but refresh in the background.
- **Hard Validation:** Block access until freshness is confirmed.

**Scenario:** You are designing a caching system for a news aggregator app.

- Would Check on Use be appropriate for article summaries?
- How would you handle validation failures or server delays?

## IV. Leases in Cache Consistency

In distributed systems, **leases** are a time-bound mechanism for maintaining cache consistency. A lease allows a client to cache a data item for a fixed duration without revalidating with the server.

Once the lease expires, the client must check with the server before using the cached data again.

Imagine renting a car: during the rental period, you can drive it freely. Once the rental expires, you must return the car or renew the agreement. Similarly, a lease grants temporary permission to use cached data without contacting the server.

## Lease Protocol

1. Client requests data from the server.
2. Server responds with the data and a lease duration  $T$ .
3. Client caches the data and uses it freely for time  $T$ .
4. After  $T$  expires, the client must revalidate or renew the lease.
5. Server may revoke leases early if the data changes.

## Benefits

- Reduces server load compared to frequent polling.
- Provides bounded staleness: data is guaranteed fresh within lease duration.
- Supports fault tolerance: expired leases naturally invalidate stale data.

## Trade-Offs

- Requires lease tracking and timers on clients.
- Updates may be delayed if leases are still active.
- Clients must implement lease renewal logic.

## Use Cases

- Distributed file systems (e.g., NFS with leases).
- Cloud storage APIs with time-bound access.
- Systems requiring eventual consistency with bounded staleness.

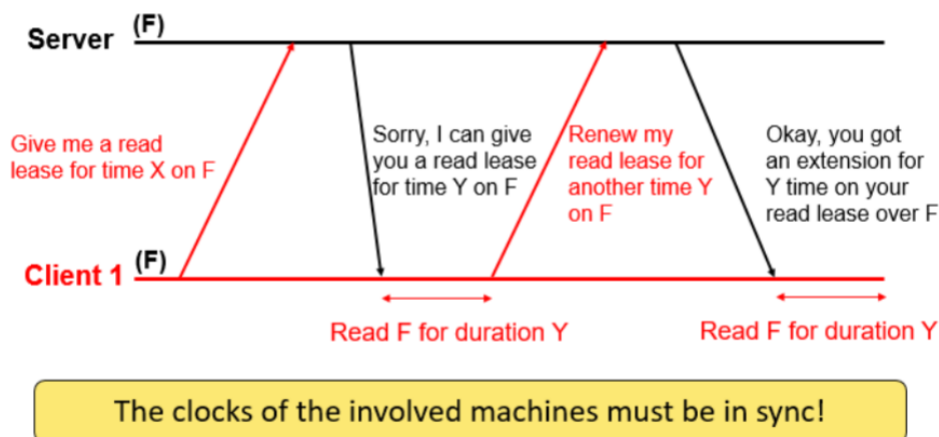


Figure 6.5: Read process in Lease

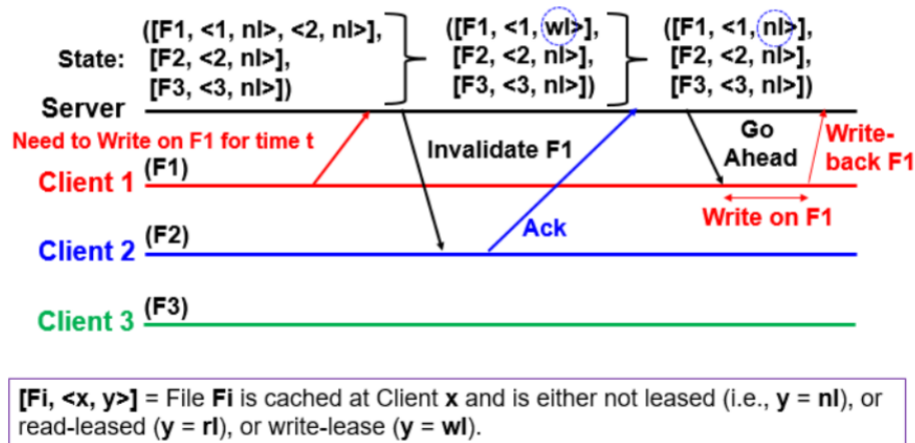


Figure 6.6: Write process in Lease

## V. Skip Scary Parts Strategy

In distributed systems, the **Skip Scary Parts** strategy avoids blocking or revalidation when cached data may be stale. Instead, the system skips operations that depend on uncertain data, allowing continued execution with reduced fidelity.

### Mechanism

- Cached data is used optimistically.
- If consistency is uncertain, dependent operations are skipped.
- Fallbacks or defaults may be used in place of missing data.

### Benefits

- High availability and responsiveness.
- Graceful degradation of functionality.
- Reduced server load and latency.

### Trade-Offs

- Potential loss of completeness or accuracy.
- Silent failures may affect user experience.
- Requires careful design to avoid cascading inconsistencies.

### Example Use Cases

- UI rendering with optional metadata.
- Analytics pipelines tolerating partial input.
- Mobile apps with intermittent network access.

## VI. Faith-Based Caching

Faith-based caching is an optimistic strategy in which clients use cached data without revalidation. The system assumes the data remains valid, trading consistency guarantees for performance and simplicity.

### Mechanism

- Data is cached locally.
- No lease, expiration, or callback is used.
- Cached data is used until explicitly refreshed or invalidated.

## Benefits

- High performance and low latency.
- Minimal protocol complexity.
- Reduced server load.

## Risks

- Potential for stale or inconsistent data.
- Silent failures may affect correctness.
- Difficult to debug without validation traces.

## Use Cases

- Read-heavy systems with stable data.
- Non-critical UI metadata.
- Offline-first applications.

## VII. Pass the Buck: Shifting Responsibility

**Pass the buck** is an idiomatic expression meaning to shift responsibility or blame to another person or group. It implies avoidance of accountability and is often used in contexts involving leadership, decision-making, or system design.

## Origin

The phrase originates from 19th-century American poker games, where a marker—often a knife with a buckhorn handle—was used to indicate the dealer. Players who did not wish to deal could pass the marker, or "buck," to the next player. Over time, the phrase evolved to mean avoiding responsibility.

## Modern Usage

- In management: Delegating difficult decisions without resolution.
- In politics: Avoiding blame by redirecting it to other departments or officials.
- In distributed systems: Deferring consistency or fault-handling to another node or layer.

## Contrasting Phrase

U.S. President Harry S. Truman famously displayed a sign on his desk reading: “*The buck stops here.*” This phrase signifies taking full responsibility and refusing to pass the blame.

## Example

When the system failed, the manager passed the buck to the technical team, claiming it was outside his scope.

### 6.1.3 Replacement Policy: Which Cached Data Should Be Evicted and When?

Caches have limited storage. When new data needs to be cached but the cache is full, the system must decide which existing data to remove. This decision is governed by the **replacement policy**.

## Goals of Replacement

The replacement policy aims to:

- Maximize cache hit rate.
- Minimize latency and bandwidth usage.
- Adapt to changing access patterns.

## Common Replacement Strategies

### A. Least Recently Used (LRU)

**Definition:** Evict the item that has not been accessed for the longest time.

**Analogy:** Like cleaning out your closet—if you haven’t worn a shirt in a year, it’s probably time to donate it.

**Mechanism:**

- Maintain a timestamp or access order for each item.
- On eviction, remove the oldest accessed item.

**Pros:**

- Simple and intuitive.
- Performs well when access patterns are temporal.

**Cons:**

- Requires tracking access history.
- May evict frequently used but recently idle items.

### B. Least Frequently Used (LFU)

**Definition:** Evict the item with the lowest access count.

**Analogy:** Like pruning your playlist—songs you rarely listen to get removed.

**Mechanism:**

- Maintain a counter for each item.
- On eviction, remove the item with the smallest count.

**Pros:**

- Captures long-term popularity.

**Cons:**

- May retain stale but historically popular items.
- Requires periodic counter resets or decay.

### C. First-In First-Out (FIFO)

**Definition:** Evict the item that was added earliest.

**Analogy:** Like a queue at a bakery—the first customer served is the first to leave.

**Mechanism:**

- Track insertion order.
- Remove the oldest item on eviction.

**Pros:**

- Simple to implement.

**Cons:**

- Ignores access patterns.
- May evict frequently used items.

### D. Random Replacement

**Definition:** Evict a randomly chosen item.

**Use Case:** Suitable for systems with unpredictable access patterns or low overhead requirements.

**LRU:** [A, B, C] → Access B → [A, C, B]



| Policy | Tracks Usage? | Complexity | Adaptability | Use Case                        |
|--------|---------------|------------|--------------|---------------------------------|
| LRU    | Yes           | Medium     | High         | Web caches, OS page replacement |
| LFU    | Yes           | High       | Medium       | CDN, database caching           |
| FIFO   | No            | Low        | Low          | Embedded systems                |
| Random | No            | Very Low   | Low          | Hardware caches                 |

Table 6.2: Comparison of Cache Replacement Policies

Evict A (least recently used)

**LFU:** A(5), B(2), C(1)  
Evict C (least frequently used)

**FIFO:** [A, B, C]  
Evict A (first-in)

**Random:** [A, B, C]  
Evict any item randomly

**Scenario:** You are designing a cache for a video streaming platform.

- Which replacement policy would you choose?
- How would you handle trending content vs. long-tail content?

## Part II: Replication

Replication is the process of maintaining multiple copies of data across different servers or nodes in a distributed system. These copies, called **replicas**, allow clients to access data from the nearest or most available source.

### Motivations for Replication

- **Performance:** Reduce latency by serving data from nearby replicas.
- **Availability:** Mask failures such as server crashes or network partitions.
- **Scalability:** Distribute load across multiple servers to avoid bottlenecks.
- **Security and Fault-Tolerance:** Protect against malicious or faulty replicas by relying on correct majority.

### Examples of Replication

#### Content Delivery Networks (CDNs)

Replicate static content (e.g., videos, software) across geographically distributed servers to reduce latency and server load.

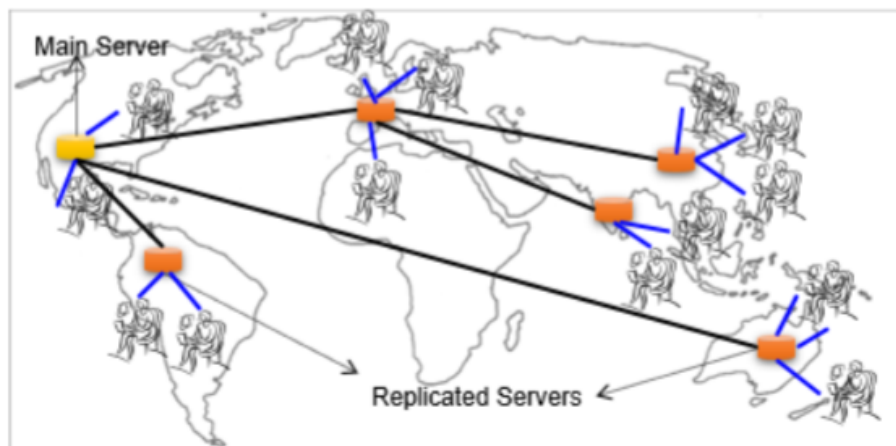


Figure 6.7: Content Delivery Network

#### Google File System

Replicates data blocks across racks, clusters, and data centers to ensure high availability and fault tolerance.

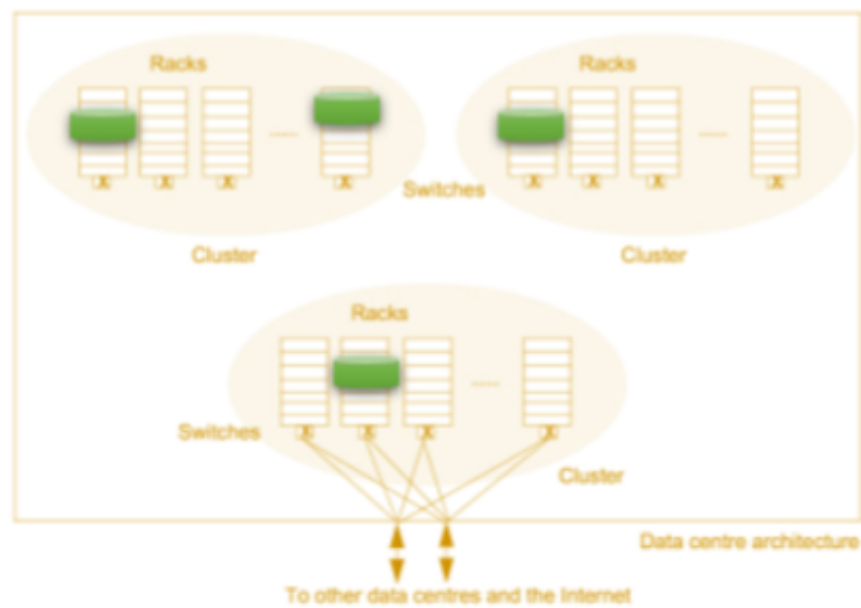


Figure 6.8: Google File System

## Peer-to-Peer Systems

Replicas can outvote faulty or malicious nodes to ensure correct data delivery.

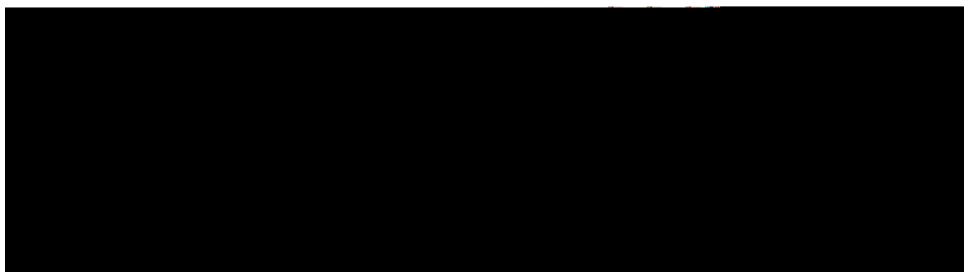


Figure 6.9: Peer to Peer Network

## The Challenge of Consistency

While replication improves performance and reliability, it introduces the challenge of maintaining **consistency** across replicas. Updates must be applied in a coordinated manner to ensure that all replicas reflect a coherent view of the data.

### Example: Bank Database

- Initial balance: \$1000
- Event 1: Add \$1000
- Event 2: Add 5% interest
- Different update orders lead to different final balances

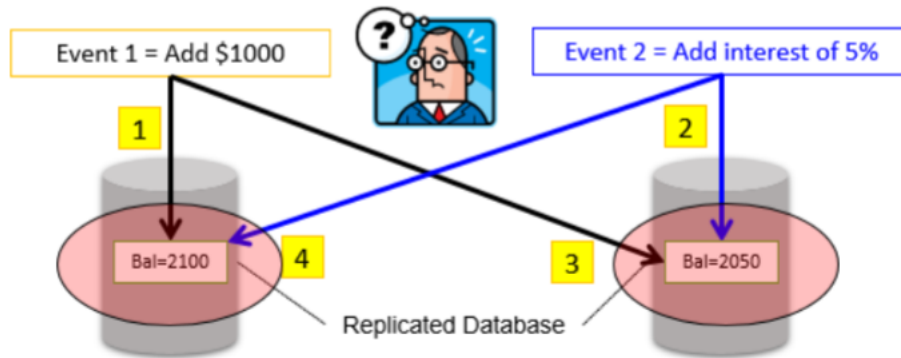


Figure 6.10: Consistency Example

## 6.2 Ordering of Updates

To maintain consistency, replicas must agree on the order in which updates are applied. Three major types of ordering are:

### 6.2.1 Total Ordering in Distributed Systems

One of the strongest forms of ordering is known as **Total Ordering**.

#### Definition

A system satisfies **total ordering** if:

If process  $P_i$  sends a message  $m_i$  and process  $P_j$  sends a message  $m_j$ , and if one correct process delivers  $m_i$  before  $m_j$ , then **every other correct process must also deliver  $m_i$  before  $m_j$** .

This ensures that all replicas apply updates in the same global order, regardless of the order in which messages were originally sent or received.

#### Motivation

Total ordering is essential when operations depend on each other or when the system must emulate a single-copy semantic. Without it, different replicas may diverge in state due to inconsistent update sequences.

#### Example

Consider three processes  $P_1$ ,  $P_2$ , and  $P_3$  maintaining replicas of a variable  $x$ .

- $P_1$  issues message  $m_{(1,1)}$ :  $x = x + 1$ ;
- $P_3$  issues message  $m_{(3,1)}$ :  $\text{print}(x)$ ;

If any process delivers  $m_{(3,1)}$  before  $m_{(1,1)}$ , then all processes must deliver the messages in that same order:

```
print(x);
x = x + 1;
```

This guarantees that the output of  $\text{print}(x)$  is consistent across all replicas.

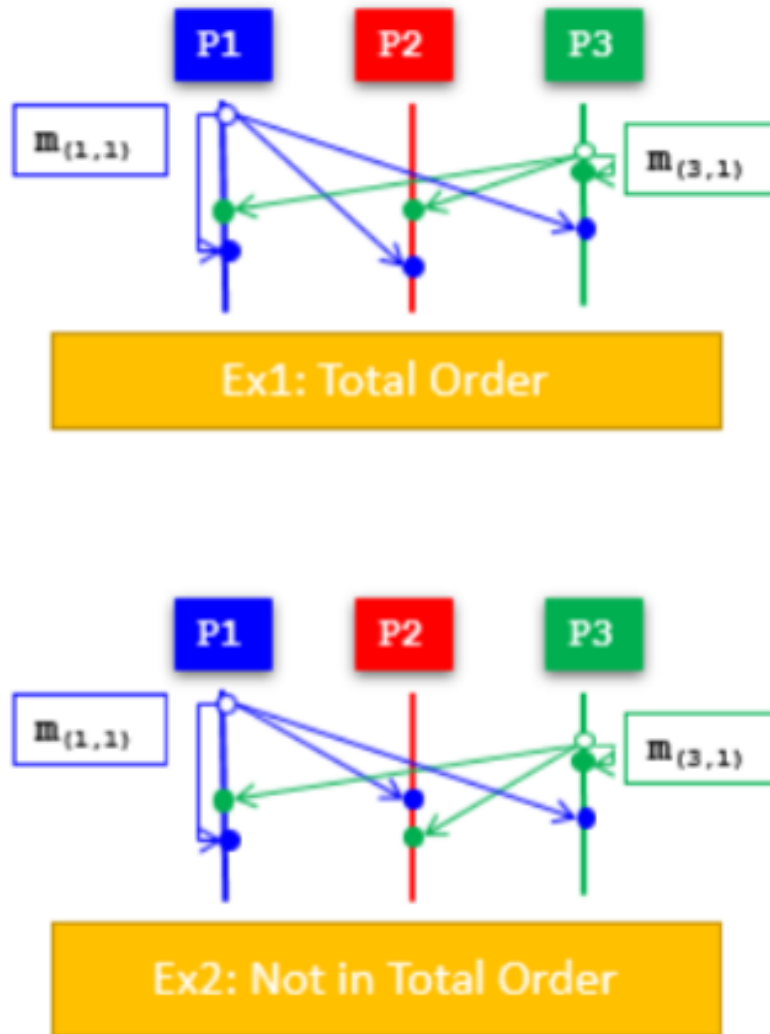


Figure 6.11: Total Ordering System Different Examples

In Ex2, the messages are delivered in different orders across processes, violating total ordering.

### Implications

- Guarantees strong consistency across replicas.
- Requires coordination mechanisms such as sequencers or consensus protocols.
- May introduce latency due to synchronization overhead.

### Use Cases

- Distributed databases requiring strict serializability.
- Replicated state machines (e.g., Paxos, Raft).
- Financial systems where operation order affects correctness.

## 6.2.2 Sequential Ordering

While total ordering ensures a global agreement on message delivery order across all processes, **sequential ordering** (also called *per-process consistency*) relaxes this constraint slightly.

### Definition

A system satisfies **sequential ordering** if:

The messages sent by each individual process are delivered by all correct processes in the order they were sent by that process.

This means that while messages from different processes may be interleaved differently across replicas, the local order of each sender's messages is preserved globally.

### Example

Suppose process  $P_1$  sends two messages:

- $m_{(1,1)}: x = x + 1;$
- $m_{(1,2)}: \text{print}(x);$

Sequential ordering guarantees that all processes deliver  $m_{(1,1)}$  before  $m_{(1,2)}$ . However, messages from other processes (e.g.,  $P_2, P_3$ ) may be interleaved differently across replicas.

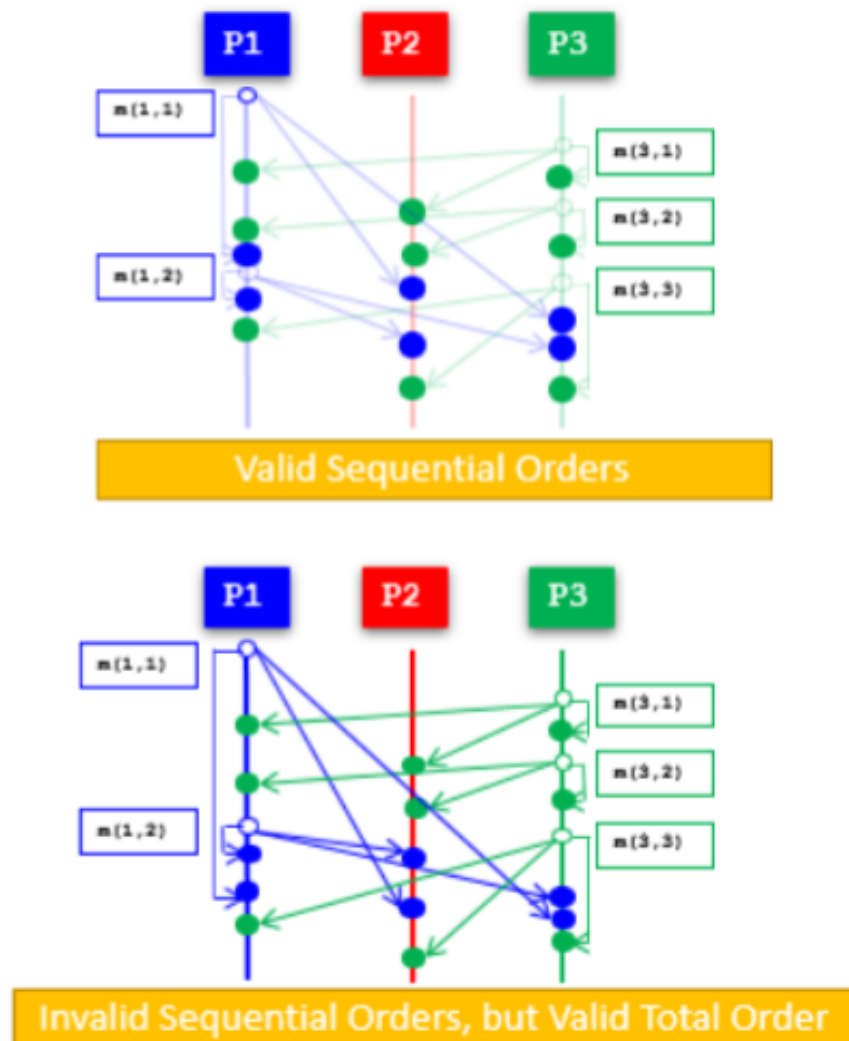


Figure 6.12: Sequential Ordering System Different Examples

Each process's messages are delivered in the order they were sent, but the interleaving across processes may vary.

## Comparison with Total Ordering

| Property                            | Total Ordering | Sequential Ordering |
|-------------------------------------|----------------|---------------------|
| Global agreement on message order   | Yes            | No                  |
| Per-process message order preserved | Yes            | Yes                 |
| Stronger consistency guarantee      | Yes            | No                  |
| Lower coordination overhead         | No             | Yes                 |

## Implications

- Sequential ordering is easier to implement and incurs less overhead.
- Suitable for applications where per-process causality is sufficient.
- May lead to divergent states if operations are not commutative.

### 6.2.3 Causal Ordering

In distributed systems, operations may have causal relationships—where one event influences or depends on another. **Causal Ordering** ensures that these dependencies are respected during message delivery.

## Definition

A system satisfies **causal ordering** if:

If message  $m_i$  causally precedes message  $m_j$  (denoted  $m_i \rightarrow m_j$ ), then every correct process delivers  $m_i$  before  $m_j$ .

This does not require a total order of all messages, only that causally related messages are delivered in the correct order.

## Motivation

Causal ordering preserves the "happens-before" relationships between events, as defined by Lamport. It is crucial in collaborative applications, replicated data stores, and systems where user actions depend on prior context.

## Example

Consider three processes:

- $P_1$  sends  $m_1$ :  $x = 5$ ;
- $P_2$  receives  $m_1$  and sends  $m_2$ : `print(x)`;

Here,  $m_1 \rightarrow m_2$  because  $m_2$  was triggered by the receipt of  $m_1$ . Causal ordering ensures that all processes deliver  $m_1$  before  $m_2$ , so that `print(x)` correctly outputs 5.

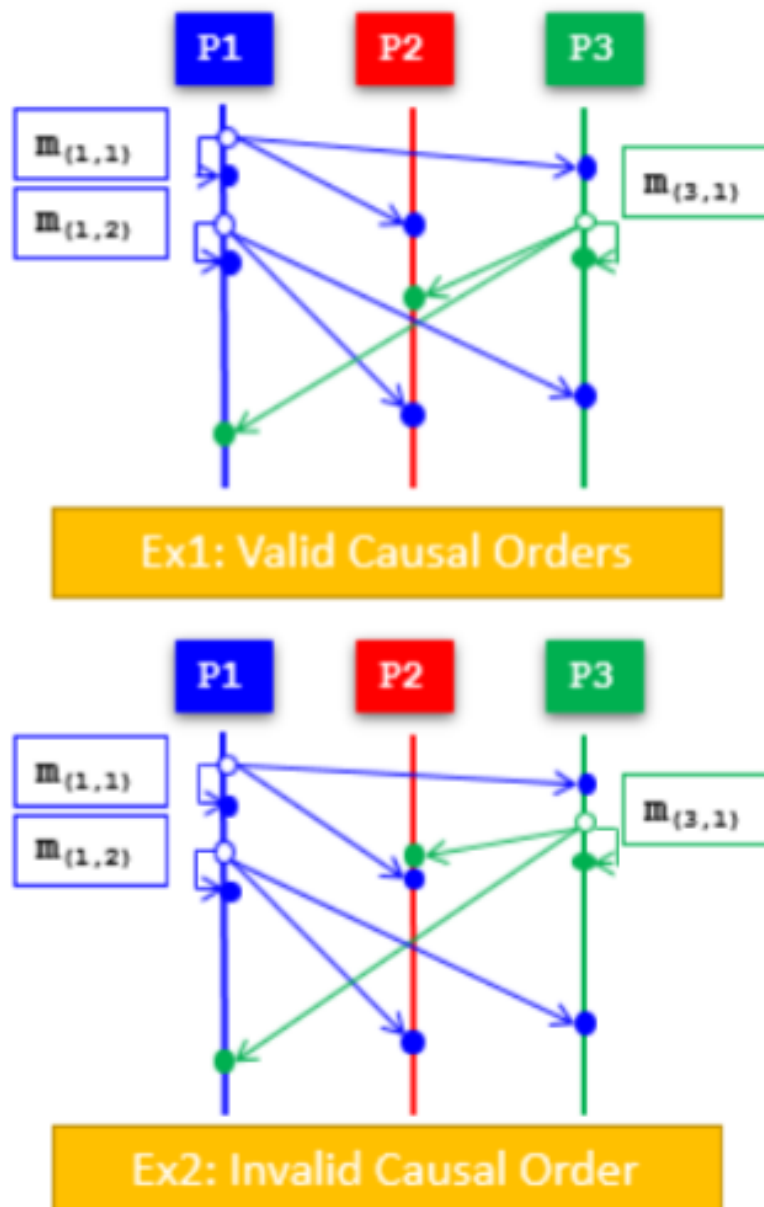


Figure 6.13: Casual Ordering System Examples

### Comparison with Other Orderings

| Property                              | Total Ordering | Sequential Ordering | Causal Ordering |
|---------------------------------------|----------------|---------------------|-----------------|
| Global agreement on all message order | Yes            | No                  | No              |
| Per-process message order preserved   | Yes            | Yes                 | Yes             |
| Causal dependencies respected         | Yes            | No                  | Yes             |
| Coordination overhead                 | High           | Low                 | Moderate        |



### 6.3 Questions on Replication

1. How does replication improve performance, availability, scalability, and security—and what trade-offs does it introduce?
2. In what scenarios might replication actually reduce system efficiency or consistency?
3. How does quorum-based replication help balance consistency and availability?
4. What are the implications of using asynchronous replication in financial systems?
5. How does replication interact with load balancing in distributed architectures?
6. How do total ordering, sequential consistency, and causal consistency differ in terms of guarantees and implementation complexity?
7. Why might a system choose causal consistency over sequential consistency?
8. How does the choice of consistency model affect the correctness of a replicated bank database?
9. What challenges arise when concurrent updates are applied to replicated data without a global ordering?
10. What are the pros and cons of using version vectors for conflict detection?
11. How can application-level semantics guide conflict resolution in replicated systems?

### Questions on Caching

1. What are the key differences between replication and caching, and how do they affect system behavior?
2. Why is caching considered a latency optimization rather than a reliability mechanism?
3. How does cache locality (temporal and spatial) influence system performance?
4. How do fetching, consistency, and replacement policies interact to shape cache behavior?
5. What are the trade-offs between push-based and pull-based fetching strategies?
6. What is the “one-copy semantic,” and why is it difficult to achieve in real-world caching systems?
7. How do different cache consistency approaches (e.g., broadcast invalidations, callback, check-on-use) balance strictness and scalability?
8. How do distributed caches maintain consistency across nodes?
9. What challenges arise when caching dynamic content in web applications?
10. What metrics would you use to evaluate the effectiveness of a caching strategy?

## 6.4 Answers of Replication

1. Replication improves fault tolerance, load distribution, and read performance. Trade-offs include consistency overhead, update latency, and conflict resolution complexity.
2. Over-replication can lead to stale data, increased write latency, and synchronization bottlenecks, especially in systems with frequent updates.
3. Quorum-based replication ensures consistency by requiring a majority of nodes to agree on updates, balancing availability and correctness.
4. Asynchronous replication improves performance but risks data loss during failures. It is unsuitable for systems requiring strong consistency.
5. Replication can distribute load across nodes, improving scalability. However, it may complicate routing and consistency enforcement.
6. Total ordering ensures all replicas apply updates in the same order. Sequential consistency preserves program order per client. Causal consistency respects causality but allows more concurrency.
7. Causal consistency offers a balance between performance and correctness, making it suitable for collaborative applications where strict ordering is unnecessary.
8. A bank database requires strong consistency to prevent anomalies like double withdrawals. Weak models may violate transactional integrity.
9. Without global ordering, concurrent updates may conflict or be applied inconsistently, requiring conflict resolution mechanisms.
10. Version vectors track causality and detect conflicts but require metadata overhead and careful interpretation.
11. Application semantics (e.g., last-write-wins, merge functions) guide resolution by prioritizing domain-specific correctness over strict consistency.

## Answers on Caching

1. Replication maintains multiple authoritative copies; caching stores temporary, often stale data for performance. Caching is read-optimized, replication is fault-tolerant.
2. Caching reduces latency by serving data locally but does not guarantee correctness or durability, unlike replication.
3. Temporal locality refers to re-accessing recently used data; spatial locality refers to accessing nearby data. Both improve cache hit rates.
4. Fetching policies determine when data is loaded; consistency policies ensure correctness; replacement policies manage eviction. Their interaction defines cache behavior.
5. Push strategies proactively update caches, increasing freshness but consuming bandwidth. Pull strategies reduce overhead but may serve stale data.
6. One-copy semantics ensures cached data behaves as if from a single source. Achieving this requires strict consistency, which is costly and complex.
7. Broadcast invalidations scale poorly; callbacks reduce traffic but require state; check-on-use is simple but may delay access.
8. Distributed caches use coherence protocols, versioning, or TTLs to maintain consistency across nodes.
9. Dynamic content changes frequently, making caching difficult. Solutions include edge computing, cache busting, and content-aware policies.
10. Metrics include hit ratio, latency, bandwidth savings, and freshness. These guide cache tuning and evaluation.

## 6.5 Multiple Choice Quiz

- Q1.** What is the primary benefit of caching in distributed systems?
- A. Ensures strong consistency
  - B. Reduces latency and backend load
  - C. Guarantees fault tolerance
  - D. Simplifies replication
- Q2.** Which caching strategy writes data to both cache and backend simultaneously?
- A. Write-around
  - B. Write-through
  - C. Write-back
  - D. Lazy write
- Q3.** What does a high cache hit ratio typically indicate?
- A. Frequent cache invalidations
  - B. Efficient cache usage
  - C. Poor replacement policy
  - D. High write latency
- Q4.** Which of the following is a drawback of caching?
- A. Increased consistency
  - B. Serving stale data
  - C. Reduced scalability
  - D. Guaranteed availability
- Q5.** What does “check-on-use” consistency imply?
- A. Data is always fresh
  - B. Cache is updated periodically
  - C. Validity is checked at access time
  - D. Backend pushes updates
- Q6.** Which eviction policy removes the least recently accessed item?
- A. FIFO
  - B. LRU
  - C. LFU
  - D. MRU
- Q7.** Why might write-around caching be preferred over write-through?
- A. It reduces backend writes
  - B. It avoids polluting the cache with infrequent data
  - C. It guarantees consistency
  - D. It improves fault tolerance
- Q8.** What is a key challenge in cache invalidation?
- A. Reducing latency
  - B. Ensuring consistency with backend
  - C. Improving fault tolerance
  - D. Avoiding replication
- Q9.** Which caching strategy is best for read-heavy workloads?
- A. Write-through
  - B. Write-around
  - C. Read-through
  - D. No caching
- Q10.** What does edge caching primarily optimize?
- A. Backend synchronization
  - B. Static content delivery
  - C. Write latency

- D. Conflict resolution
- Q11.** Which method actively notifies clients of stale data?
  - A. Polling
  - B. Callback invalidation
  - C. Lazy invalidation
  - D. Check-on-use
- Q12.** What is the role of a cache eviction policy?
  - A. To replicate data
  - B. To remove stale data
  - C. To manage cache capacity
  - D. To enforce consistency
- Q13.** Which metric best reflects cache effectiveness?
  - A. Write latency
  - B. Cache hit ratio
  - C. Backend throughput
  - D. Replication factor
- Q14.** What is a downside of write-through caching?
  - A. Data loss
  - B. Increased write latency
  - C. Stale reads
  - D. Reduced fault tolerance
- Q15.** Which caching model delays writing to the backend until eviction?
  - A. Write-through
  - B. Write-around
  - C. Write-back
  - D. Read-through
- Q16.** What is the primary goal of replication?
  - A. Reduce latency
  - B. Improve fault tolerance and availability
  - C. Enforce access control
  - D. Simplify caching
- Q17.** Which consistency model allows temporary divergence among replicas?
  - A. Strong consistency
  - B. Eventual consistency
  - C. Linearizability
  - D. Sequential consistency
- Q18.** What is a trade-off introduced by replication?
  - A. Improved consistency
  - B. Increased fault tolerance with consistency overhead
  - C. Reduced latency with strict ordering
  - D. Simplified conflict resolution
- Q19.** Which replication strategy updates all replicas synchronously?
  - A. Lazy replication
  - B. Eager replication
  - C. Partial replication
  - D. Asynchronous replication
- Q20.** What does quorum-based replication help achieve?
  - A. Faster reads
  - B. Strong consistency via majority agreement
  - C. Cache invalidation

- D. Stateless replication
- Q21.** Which scenario best illustrates replication's benefit?
  - A. Serving static content
  - B. Accessing data during node failure
  - C. Evicting stale cache
  - D. Reducing backend load
- Q22.** What is a challenge in replicated systems?
  - A. Reducing latency
  - B. Ensuring consistency across replicas
  - C. Avoiding cache pollution
  - D. Simplifying access control
- Q23.** Which model guarantees that all replicas reflect the same state instantly?
  - A. Eventual consistency
  - B. Strong consistency
  - C. Causal consistency
  - D. Read-your-writes consistency
- Q24.** What is a benefit of asynchronous replication?
  - A. Immediate consistency
  - B. Reduced write latency
  - C. Guaranteed ordering
  - D. Conflict-free updates
- Q25.** Which replication model tracks causal relationships between operations?
  - A. Strong consistency
  - B. Sequential consistency
  - C. Causal consistency
  - D. Linearizability
- Q26.** What does replication improve besides fault tolerance?
  - A. Write latency
  - B. Data locality and read performance
  - C. Cache invalidation
  - D. Backend throughput
- Q27.** Which replication method delays updates until needed?
  - A. Eager replication
  - B. Lazy replication
  - C. Quorum replication
  - D. Synchronous replication
- Q28.** What is a risk of replication without proper conflict resolution?
  - A. High latency
  - B. Data loss
  - C. Divergent replicas
  - D. Cache pollution
- Q29.** Which consistency model ensures operations appear instantaneous?
  - A. Eventual consistency
  - B. Linearizability
  - C. Causal consistency
  - D. Read-your-writes

## 7. Fault Tolerance

### Course Map

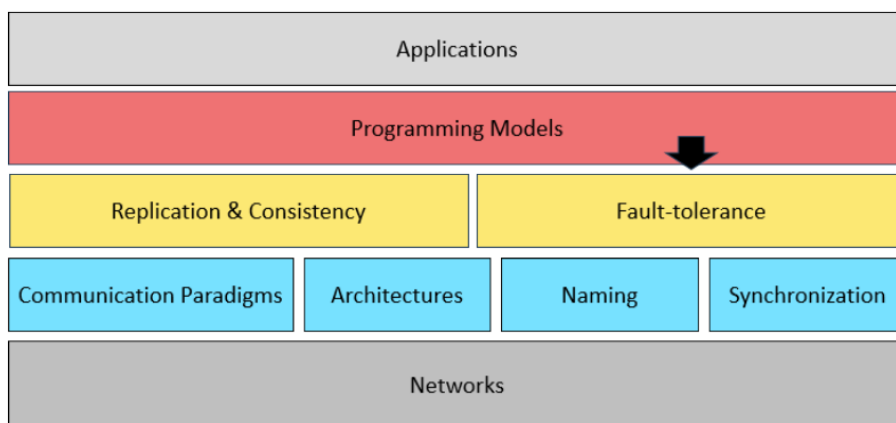


Figure 7.1: Fault Tolerance

Distributed systems are inherently prone to partial failures due to their reliance on multiple components, networks, and processes. **Fault tolerance** is the property that enables a system to continue operating correctly even when some of its components fail.



Figure 7.2: Fault Tolerance Concept

**Example:** TCP is designed to maintain reliable communication over unreliable networks by retransmitting lost packets and reordering out-of-sequence data.

## 7.1 What is a Failure?

A **failure** is a deviation from the system's specified behavior.

- **Catastrophic failure:** Brake pedal does not stop the car.
- **Non-catastrophic failure:** Disk read returns incorrect data.

Failures often result from unanticipated interactions or overlooked scenarios in complex systems.

## 7.2 Types of Failures

Distributed systems can experience various types of failures. Understanding these helps in designing systems that can detect, recover from, or tolerate faults effectively. Below, we explore two fundamental categories: transient and persistent failures.

### A. Transient Failures (Soft Failures / Heisenbugs)

**Definition:** Transient failures are temporary malfunctions that disappear without any permanent damage or intervention.

**Characteristics:**

- Caused by rare timing conditions, race conditions, or temporary resource unavailability.
- Often resolved by retrying the operation or rebooting the system.
- Difficult to reproduce consistently and may not leave behind logs.

**Examples:**

- A network packet dropped due to momentary congestion.
- A GUI freezes but becomes responsive after a few seconds.
- A thread deadlocks only under specific timing conditions.

**Analogy:** Like a flickering light bulb—it works fine most of the time, but occasionally blinks for no clear reason. Tap it, and it's back to normal.

**Design Implication:** Systems should implement time redundancy, such as automatic retries with exponential backoff, to mitigate transient failures.

### B. Persistent Failures (Hard Failures)

**Definition:** Persistent failures are long-lasting faults that require manual repair, replacement, or intervention to resolve.

**Characteristics:**

- Caused by hardware faults, software bugs, or permanent resource loss.
- Retry attempts do not resolve the issue.
- Easier to detect and log than transient failures.

**Examples:**

- A disk drive crashes and becomes unreadable.
- A server shuts down due to a power supply failure.
- A corrupted file system that consistently returns incorrect data.

**Analogy:** Like a flat tire—you can't keep driving until you fix or replace it. No amount of retrying will make it roll again.

**Design Implication:** Fault-tolerant systems use redundancy—such as backup servers or replicated data—to mask persistent failures and maintain availability.

## 7.3 Reliability vs. Availability

- **Availability:** Probability that a system is operational at a given moment.
- **Reliability:** Probability that a system continues to operate without failure over time.



**Formula:**

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

| System Type              | Availability (%) | Annual Downtime |
|--------------------------|------------------|-----------------|
| Workstation              | 99.0             | 3.6 days        |
| High-Availability System | 99.9             | 8.4 hours       |
| Fault-Resilient System   | 99.99            | 1 hour          |
| Fault-Tolerant System    | 99.999           | 5 minutes       |

## 7.4 Techniques for Fault Tolerance

### A. Masking Failure (Redundancy)

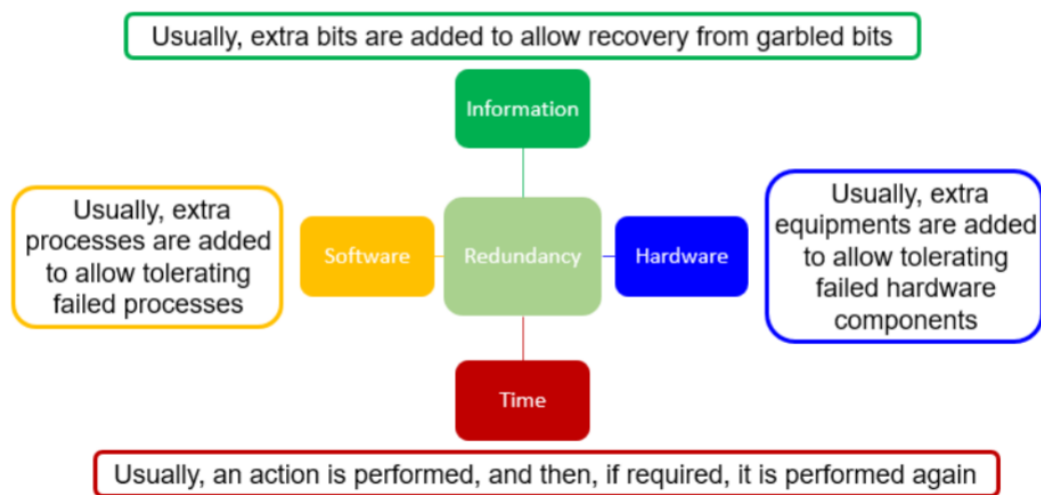


Figure 7.3: Redundancy types for Masking Failure

- **Information Redundancy:** Extra bits (e.g., parity, checksums) to detect and correct errors.
- **Hardware Redundancy:** Duplicate components (e.g., backup servers).
- **Software Redundancy:** Multiple processes or threads performing the same task.
- **Time Redundancy:** Retry operations upon failure.

### B. Failure Detection

- Use heartbeats or periodic messages to detect unresponsive components.
- Distinguish between server and network failures by cross-checking with other processes.

#### 1. Atomic Multicast:

Atomic multicast ensures:

- **Atomicity:** A message is delivered to either all or none of the recipients.
- **Total Ordering:** All recipients deliver messages in the same order.

#### 2. Distributed Atomic Transactions

A key protocol for fault-tolerant transactions is the **Two-Phase Commit (2PC)**.



Phase I: Voting

- 1. Coordinator sends VOTE\_REQUEST to all participants.
- 2. Each participant replies with VOTE\_COMMIT or VOTE\_ABORT.



Figure 7.4: Phase I: Voting Steps

Phase II: Decision

- 1. If all votes are COMMIT, coordinator sends GLOBAL\_COMMIT.
- 2. Otherwise, coordinator sends GLOBAL\_ABORT.
- 3. Participants act accordingly.

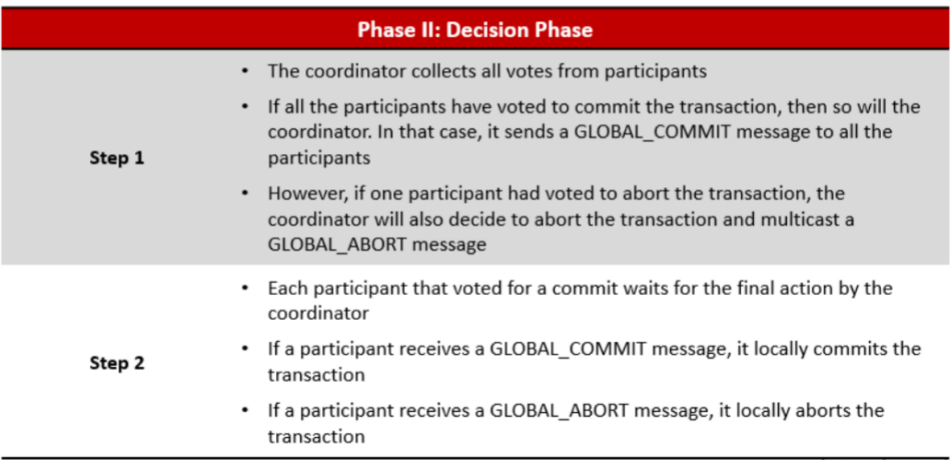


Figure 7.5: Phase II: Decision Steps

Recovery Scenarios

- **Coordinator Recovery:** Uses log to retransmit decision.
- **Participant Recovery:** Uses log to determine next action or request decision from peers.

7.5 Two-Phase Commit Finite State Machines

The Two-Phase Commit (2PC) protocol ensures atomic commitment across distributed systems. It involves two roles:

- **Coordinator:** Initiates the transaction and decides its final outcome.
- **Participants:** Vote on whether to commit or abort and act based on the coordinator’s decision.

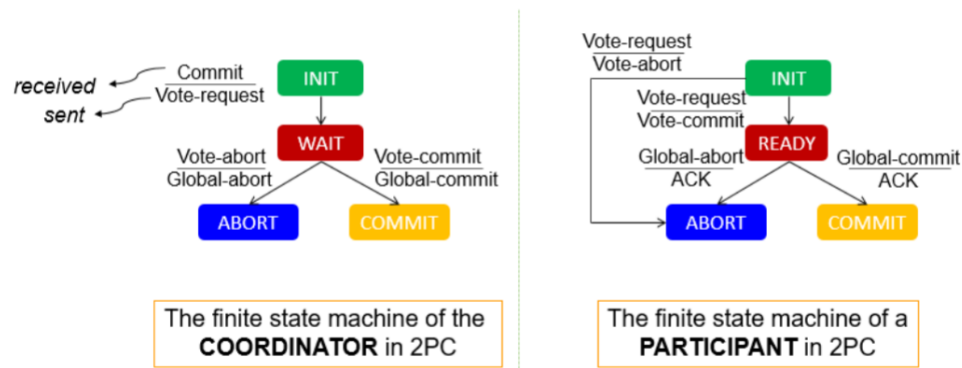


Figure 7.6: 2PC Finite State Machines

## 1. Coordinator State Transitions

### 2PC Coordinator Algorithm

```

write START_2PC to local log;
multicast VOTE_REQUEST to all participants;

while not all votes have been collected {
    wait for any incoming vote;
    if timeout {
        write GLOBAL_ABORT to local log;
        multicast GLOBAL_ABORT to all participants;
        exit;
    }
    record vote;
}

if all participants sent VOTE_COMMIT and coordinator votes COMMIT {
    write GLOBAL_COMMIT to local log;
    multicast GLOBAL_COMMIT to all participants;
} else {
    write GLOBAL_ABORT to local log;
    multicast GLOBAL_ABORT to all participants;
}

```

- **INIT:** Initial state before any messages are sent.
- **WAIT:** After sending VOTE\_REQUEST, the coordinator waits for votes.
- **COMMIT:** If all participants vote VOTE\_COMMIT, the coordinator sends GLOBAL\_COMMIT.
- **ABORT:** If any participant votes VOTE\_ABORT, the coordinator sends GLOBAL\_ABORT.

#### Coordinator Transitions:

- INIT → WAIT: Upon sending VOTE\_REQUEST.
- WAIT → COMMIT: If all votes are VOTE\_COMMIT.
- WAIT → ABORT: If any vote is VOTE\_ABORT.
- COMMIT → COMMIT: Retransmit GLOBAL\_COMMIT if needed.
- ABORT → ABORT: Retransmit GLOBAL\_ABORT if needed.

## 2. Participant State Transitions

### 2PC Participant Algorithm

```

write INIT to local log;
Wait for VOTE_REQUEST from coordinator;

if timeout {
    write VOTE_ABORT to local log;
    exit;
}

if participant votes COMMIT {
    write VOTE_COMMIT to local log;
    send VOTE_COMMIT to coordinator;
    wait for DECISION from coordinator;

    if timeout {
        multicast DECISION_REQUEST to other participants;
        wait until DECISION is received; /* remain blocked */
        write DECISION to local log;
    }

    if DECISION == GLOBAL_COMMIT {
        write GLOBAL_COMMIT to local log;
    } else if DECISION == GLOBAL_ABORT {
        write GLOBAL_ABORT to local log;
    }
} else {
    write VOTE_ABORT to local log;
    send VOTE_ABORT to coordinator;
}

```

- **INIT:** Initial state before receiving any message.
- **READY:** After sending VOTE\_COMMIT, the participant waits for the coordinator's decision.
- **COMMIT:** Upon receiving GLOBAL\_COMMIT, the participant commits locally.
- **ABORT:** Upon sending VOTE\_ABORT or receiving GLOBAL\_ABORT.

#### Participant Transitions:

- INIT → READY: Upon receiving VOTE\_REQUEST and sending VOTE\_COMMIT.
- INIT → ABORT: Upon sending VOTE\_ABORT.
- READY → COMMIT: Upon receiving GLOBAL\_COMMIT.
- READY → ABORT: Upon receiving GLOBAL\_ABORT.
- COMMIT → COMMIT: Acknowledge commit if requested.
- ABORT → ABORT: Acknowledge abort if requested.

### State Descriptions

- **INIT:** No decision made; waiting for initial messages.
- **WAIT (Coordinator):** Awaiting votes from participants.
- **READY (Participant):** Voted to commit; awaiting coordinator's decision.
- **COMMIT / ABORT:** Final decision states; transaction is either committed or aborted.

## Recovery Considerations

### Coordinator Recovery:

- INIT: Abort the transaction.
- WAIT: Retransmit VOTE\_REQUEST.
- COMMIT: Retransmit GLOBAL\_COMMIT.
- ABORT: Retransmit GLOBAL\_ABORT.

### Participant Recovery:

- INIT: Locally abort and notify coordinator.
- READY: Contact other participants to learn the final decision.
- COMMIT: Retransmit GLOBAL\_COMMIT.
- ABORT: Retransmit GLOBAL\_ABORT.

## Limitations

- Participants in the READY state may block indefinitely if the coordinator crashes before sending a decision.
- Logging is essential for recovery and consistency.

## 7.6 Questions:

1. What distinguishes availability from reliability, and how can a system be high in one but low in the other?
2. Why is masking failures with redundancy not always the optimal solution?
3. How do MTTF and MTTR influence system design decisions?
4. How does the CAP theorem relate to fault tolerance in distributed systems?
5. What are the trade-offs between fail-stop and fail-safe models in system design?
6. How can fault detection mechanisms be validated for correctness and completeness?
7. What are the key limitations of the Two-Phase Commit (2PC) protocol, and how do they impact distributed transaction safety?
8. How does the decision request handler help resolve indefinite blocking in 2PC?
9. How do coordinator and participant recovery strategies differ in 2PC, and what assumptions do they rely on?
10. What role does logging play in enabling fault-tolerant recovery?
11. In a distributed database, how would you handle a Byzantine failure in one node?
12. What strategies can be used to recover from partial failures without restarting the entire system?

## 7.7 Answers:

1. Availability refers to the system being operational and accessible, while reliability refers to the system performing correctly. A system can be available but unreliable if it returns incorrect results.
2. Redundancy may mask failures but can also hide underlying issues, leading to silent data corruption or increased recovery complexity.
3. MTTF (Mean Time to Failure) and MTTR (Mean Time to Repair) help quantify system robustness and recovery speed, guiding design decisions for fault-tolerant architectures.
4. The CAP theorem states that a distributed system can only guarantee two of the following three: Consistency, Availability, and Partition Tolerance. Fault tolerance often requires sacrificing consistency or availability.
5. Fail-stop systems halt upon failure, simplifying recovery. Fail-safe systems continue operating in degraded mode, which may introduce complexity and risk.
6. Fault detection mechanisms must be tested against known failure modes and validated through simulation or formal verification to ensure accuracy.
7. 2PC suffers from blocking if the coordinator crashes, lacks built-in recovery, and requires extensive logging, making it unsuitable for highly dynamic systems.
8. The decision request handler responds to blocked participants by consulting the log and sending the final decision (commit or abort), enabling recovery.
9. Coordinators must recover and resend decisions; participants must consult logs to determine their last known state. Recovery depends on durable logging.
10. Logging ensures that decisions and votes are persisted, enabling recovery after crashes. Without logs, the protocol cannot guarantee atomicity.
11. Byzantine failures involve arbitrary or malicious behavior. Handling them requires Byzantine Fault Tolerant protocols like PBFT, which use quorum and message authentication.
12. Partial recovery strategies include checkpointing, rollback recovery, and state replication. These allow subsystems to recover independently.

## Multiple Choice Quiz

- Q1.** A distributed system uses quorum-based replication. During a network partition, only a minority of nodes remain reachable. What is the most likely consequence?
- (A) The system continues with degraded performance
  - (B) The system halts write operations to preserve consistency
  - (C) The system switches to eventual consistency
  - (D) The system promotes a new leader automatically
- Q2.** Which of the following best explains why idempotent operations are critical in fault-tolerant systems?
- (A) They reduce memory usage during retries
  - (B) They ensure correctness when operations are retried after failure
  - (C) They improve throughput under load
  - (D) They prevent network congestion
- Q3.** In a replicated key-value store, one replica becomes unreachable. The system continues serving reads and writes. Which consistency model is most likely in use?
- (A) Strong consistency
  - (B) Eventual consistency
  - (C) Linearizability
  - (D) Causal consistency
- Q4.** Which fault-tolerant protocol is designed to handle arbitrary (Byzantine) failures?
- (A) Raft
  - (B) Paxos
  - (C) PBFT
  - (D) Two-Phase Commit
- Q5.** A system uses active-passive replication. The primary crashes, and the backup takes over. What is a potential risk during this failover?
- (A) Data loss due to delayed synchronization
  - (B) Increased latency due to load balancing
  - (C) Duplicate requests from clients
  - (D) Deadlock between replicas
- Q6.** Which metric best reflects the reliability of a fault-tolerant system?
- (A) MTTR (Mean Time to Repair)
  - (B) MTTF (Mean Time to Failure)
  - (C) CPU utilization
  - (D) Network throughput
- Q7.** Why might a system choose availability over consistency during a network partition?
- (A) To reduce memory usage
  - (B) To maintain user experience and responsiveness
  - (C) To simplify protocol design
  - (D) To avoid leader election
- Q8.** Which of the following is a drawback of using replication for fault tolerance?
- (A) Reduced fault isolation
  - (B) Increased consistency complexity
  - (C) Lower throughput
  - (D) Inability to scale horizontally
- Q9.** A distributed system uses consensus to elect a leader. During a failure, no leader is elected. What is the likely impact?
- (A) The system switches to read-only mode
  - (B) The system continues with degraded consistency
  - (C) The system halts operations until consensus is restored

(D) The system promotes a random node

**Q10.** Which technique helps a system recover from transient faults without human intervention?

(A) Manual failover

(B) Load shedding

(C) Self-healing mechanisms

(D) Static routing